

QFF 使用手册

版本: 0.5.18



作者: 徐海江

March 9, 2025

目 录

1	量化	1
1.1	什么是量化	1
1.2	量化的价值	2
1.3	量化的步骤	3
1.4	做量化交易需要什么	5
1.5	Python 语言的学习资源	6
2	概览	7
2.1	主要功能	7
2.2	主要特点	8
2.2.1	与其他量化框架比较	8
2.3	声明	8
2.4	鸣谢	9
3	安装说明	11
3.1	安装库	11
3.2	数据准备	12
3.2.1	数据库安装	12
3.2.2	数据库配置	12
3.2.3	数据下载	13
3.2.4	数据自动更新	15
3.2.5	数据库维护	15
4	Docker 部署	17
4.1	安装 Docker	17
4.1.1	安装指导	17
4.1.2	配置国内镜像	17
4.2	部署 MongoDB 镜像	18
4.2.1	拉取 mongo 镜像	18
4.2.2	运行 mongo 容器	18
4.3	部署 QFF 镜像	19
4.3.1	拉取 QFF 镜像	19

4.3.2	运行 QFF 容器	19
4.3.3	镜像使用	19
5	快速入门	21
5.1	开始编写第一个策略	21
5.1.1	下载策略模板	21
5.1.2	编写自己的策略代码	22
5.2	回测自己编写的策略	24
5.2.1	命令行运行	24
5.2.2	集成开发环境中运行	24
5.2.3	查看策略回测结果	24
5.3	实盘模拟编写的策略	25
5.3.1	启动实盘模拟	25
5.3.2	执行过程中交互	25
6	策略文件编写	27
6.1	框架接口函数	27
6.1.1	框架流程图	28
6.1.2	运行时间	29
6.1.3	运行频率	29
6.1.4	定时运行	30
6.2	数据获取函数	31
6.2.1	历史数据	31
6.2.2	当前数据	32
6.2.3	全局对象	33
6.3	交易执行函数	34
6.3.1	交易函数接口	34
6.3.2	订单交易流程	34
7	策略文件运行	37
7.1	回测和模拟	37
7.1.1	回测	37
7.1.2	模拟交易	38
7.2	多种方式运行策略	38
7.2.1	命令行运行	38
7.2.2	在代码中运行	39
7.3	运行过程中交互	40
7.3.1	命令总览	40
7.3.2	查询策略运行数据	41
7.3.3	查看股票 K 线图	46

7.4	暂停和恢复	48
7.4.1	暂停	48
7.4.2	恢复	49
8	策略运行评价	51
8.1	策略评价报告	51
8.1.1	收益概述页面	51
8.1.2	交易分析页面	52
8.1.3	交易详情页面	53
8.1.4	持仓详情页面	53
8.1.5	账户详情页面	54
8.1.6	日志输出页面	54
8.2	风险指标说明	55
8.2.1	策略收益	55
8.2.2	年化收益率	55
8.2.3	最大回撤	56
8.2.4	阿尔法	56
8.2.5	贝塔	56
8.2.6	夏普比率	57
8.2.7	索提诺比率	58
8.2.8	卡玛比率	58
8.2.9	欧米茄比率	59
8.2.10	信息比率	60
8.2.11	策略波动率	60
8.2.12	基准波动率	60
9	实盘操作指南	61
9.1	安装和设置	61
9.1.1	客户端安装	61
9.1.2	客户端设置	61
9.2	命令行指令	63
9.3	实盘交易函数	64
9.4	trace 交互接口	65
9.5	实盘交易注意事项	66
9.5.1	实盘与模拟交易的区别	66
9.5.2	股票实盘下单失败的可能原因	67
10	策略示例	69
10.1	双均线策略	69
10.2	MACD 策略	71

10.3	小市值策略	72
10.4	海龟策略	74
11	数据字典	83
11.1	市值数据	83
11.2	融资融券数据	84
11.3	年报 (季报) 数据	84
11.3.1	1. 每股指标	84
11.3.2	2. 资产负债表 BALANCE SHEET	85
11.3.3	3. 利润表	89
11.3.4	4. 现金流量表	89
11.3.5	5. 偿债能力分析	92
11.3.6	6. 经营效率分析	92
11.3.7	7. 发展能力分析	93
11.3.8	8. 获利能力分析	93
11.3.9	9. 资本结构分析	94
11.3.10	10. 现金流量分析	94
11.3.11	11. 单季度财务指标	95
11.3.12	12. 股本股东	95
11.3.13	13. 机构持股	95
11.3.14	14. 预告快报数据	97
11.3.15	15. 增补数据	98
11.3.16	16. 资产负债表新增指标	98
11.3.17	17. 未分类财务数据	100
12	基础 API	103
12.1	策略程序接口	103
12.1.1	initialize - 策略初始化	103
12.1.2	handle_data - 盘中策略运行函数 (可选)	104
12.1.3	before_trading_start - 开盘前运行策略 (可选)	105
12.1.4	after_trading_end - 收盘后运行策略 (可选)	105
12.1.5	on_strategy_end - 策略运行结束时调用函数 (可选)	106
12.1.6	process_initialize - 策略恢复启动时运行函数 (可选)	106
12.2	策略执行函数	107
12.2.1	run_file - 执行指定的策略文件	107
12.3	策略设置函数	108
12.3.1	run_daily - 定时运行策略设置函数 (可选)	108
12.3.2	set_benchmark - 设置指数基准	109
12.3.3	set_order_cost - 设置交易税费	109

12.3.4	set_slippage - 设置固定滑点	110
12.3.5	set_universe - 定股票值 (history 函数专用)	110
12.3.6	pass_today - 跳过当日 (回测专用)	110
12.4	股票交易函数	111
12.4.1	order - 按股票数量下单	111
12.4.2	order_value - 按股票价值下单	112
12.4.3	order_target - 按股票目标数量下单	112
12.4.4	order_target_value - 按股票目标价值下单	113
12.4.5	order_cancel - 撤回已下的订单	114
12.4.6	get_orders - 获取当天的所有订单	114
12.4.7	get_open_orders - 获得当天的所有未完成的订单	115
12.5	实盘操作函数	115
12.5.1	trader_connect - 连接交易软件客户端	115
12.5.2	trader_balance - 获取当前账户资金股票信息	115
12.5.3	trader_position - 获取当前账户股票持仓信息	116
12.5.4	trader_today_entrusts - 获取当日委托记录	116
12.5.5	trader_today_deal - 获取当日成交记录	116
12.5.6	trader_order - 实盘交易股票	117
12.5.7	trader_cancel - 撤销当日委托	117
12.6	策略分析函数	118
12.6.1	stats_report - 策略运行结果分析	118
12.6.2	strategy_eval - 择时策略评价	118
12.7	日期时间函数	119
12.7.1	get_real_trade_date - 获取真实的交易日期	119
12.7.2	is_trade_day - 判断是否交易日期	120
12.7.3	get_date_gap - 获取向前或向后间隔天数的交易日日期	120
12.7.4	get_next_trade_day - 获取下一个交易日的日期	120
12.7.5	get_pre_trade_day - 获取前几个交易周期的日期	121
12.7.6	get_trade_days - 获取指定范围交易日	121
12.7.7	get_trade_gap - 获取两个交易日之间的交易天数	122
12.7.8	run_time - 计算函数运行时间的装饰函数	122
12.8	其他函数	122
12.8.1	log - 日志 log 信息	122
12.8.2	kshow - 生成网页版 K 线图	123
12.8.3	send_message - 发送消息提醒	124
12.9	类和对象	124
12.9.1	g - 全局变量对象	124
12.9.2	context - 策略上下文全局对象	125
12.9.3	Portfolio - 股票账户	126

12.9.4	Position - 持仓	127
12.9.5	Order - 订单对象	128
12.9.6	SecurityUnitData - 当前数据对象	129
12.9.7	Tick - tick 对象	130
12.10	枚举常量	131
12.10.1	RUN_TYPE - 策略运行模式	131
12.10.2	RUN_STATUS - 策略运行状态	131
12.10.3	ORDER_TYPE - 订单类型	132
12.10.4	ORDER_STATUS - 订单状态	132

13 数据 API 133

13.1	行情	133
13.1.1	get_price - 获取历史数据, 可查询多个标的多个数据字段	133
13.1.2	history - 获取历史数据, 可查询多个标的单个数据字段	135
13.1.3	attribute_history - 获取历史数据, 可查询单个标的多个数据字段	136
13.1.4	get_bars - 获取历史数据 (包含快照数据), 可查询多个标的多个数据字段	137
13.1.5	get_current_data - 获取当前时刻标的的数据	138
13.1.6	fetch_ticks - 获取实时 tick 数据	139
13.1.7	fetch_today_transaction - 获取当日实时分笔成交信息	139
13.1.8	fetch_today_min_curve - 实时获取当天的 1 分钟曲线	140
13.2	股票	140
13.2.1	get_all_securities- 获取平台支持的所有股票信息	140
13.2.2	get_security_info- 获取股票的信息	141
13.2.3	get_stock_name- 获取股票名称	142
13.2.4	get_block_stock- 根据板块名称检索对应的股票代码	142
13.2.5	get_stock_block- 根据股票代码检索所属的板块名称	145
13.2.6	get_mtss- 获取股票的融资融券信息	145
13.2.7	get_st_stock- 获取指定日期 ST 股票代码和名称	146
13.2.8	get_paused_stock- 获取停牌的股票代码	146
13.3	指数	147
13.3.1	get_index_name- 获取指数名称	147
13.3.2	get_index_stocks- 获取一个指数给定日期的成分股列表	147
13.3.3	get_industry_stocks- 获取申万一级行业给定日期的成分股列表	148
13.4	财务	149
13.4.1	get_fundamentals- 根据 mongodb 语法查询财务数据	149
13.4.2	get_financial_data- 查询多只股票给定日期的财务数据	150
13.4.3	get_stock_reports- 获取单个股票多个报告期发布的财务数据	151

13.4.4	<code>get_fundamentals_continuously</code> - 查询单个股票连续多日的财务数据	152
13.4.5	<code>get_history_fundamentals</code> - 获取多只股票多个季度（年度）的历史财务数据	153
13.4.6	<code>get_stock_forecast</code> - 获取股票业绩预告数据	154
13.4.7	<code>get_stock_express</code> - 获取股票业绩快报数据	154
13.4.8	<code>get_valuation</code> - 获取多个股票在指定交易日范围内的市值表数据 .	155
13.4.9	<code>query_valuation</code> - 查询满足条件的市值信息数据	156

Python Module Index

159

在金融最为发达的美国，量化交易已大行其道，占据了 70% 以上的股市成交量。其中最为著名的量化私募 大奖章基金，平均年收益率高达 35%，远超同时代的股神巴菲特。

国内量化投资目前阶段相当于海外几年前的状态。2010 年 4 月 16 日，中国第一个股指期货沪深 300 股指期货挂牌上市，量化人有了对冲工具之后，量化对冲策略得以实现，2010 年也被视为是中国量化元年。近几年，国内量化私募快速发展，根据中基协数据统计，2022 年 7 月国内百亿量化私募已达 34 家，数量创下历史新高，可以说量化交易逐渐成为发展的趋势。



1.1 什么是量化

简单来说，量化就是利用计算机技术，并采用一定的数学模型、现代统计学原理或者人工智能等方法，去实现投资理念，进行证券投资的过程。

传统的股票投资和量化投资就类似中医和西医，中医讲究经验积累和定性分析，一位优秀的股票投资者来说，需花费大量的时间及试错的成本，才能积累丰富的投资经验，成功建立稳定的交易系统。西医就不同了，依托于医学仪器，对病人进行化验、拍片，得出结论对症下药

药。量化投资就像西医一样，对投资策略模型先用整个市场进行一次全面的扫描和检验，再根据检验结果反复优化，并指导我们做出正确的投资决策。

1.2 量化的价值

1. 可以利用大量历史数据检验策略，效率提升百倍。当我们想验证交易策略的时候，一个基本的想法是想知道它在历史上表现如何，这往往需要大量的历史数据与计算量，量化交易做一次回测可能几分钟就可以得到结果了，相比于传统人工做法效率的提升是成百倍的。

2. 更科学更客观的衡量交易策略的效果。比如一个关于某技术指标的策略，人工的进行了 10 个交易日的验证，效果都不错，但这就能说明这指标不错吗？不，10 次太少了，你需要更多的验证，比如 1000 个交易日，人工验证不可行，量化交易则又快又准。而且量化交易还可以利用数学与统计学自动给出客观的结果，比如年化收益率、最大回撤率、夏普比率等。

3. 全市场实时捕捉交易机会。当你知道一个盈利条件，当股价一旦满足这条件，你就可以操作盈利。问题是，市场几千个股票，股价时时刻刻都在变动，你能盯住几个，你会错失多少个机会。但量化交易可以利用计算机全市场实时盯盘，可以不错过任何交易机会，加倍你的盈利能力。

4. 更多的盈利机会。量化交易可以利用计算机对海量数据分析得到常人难以发现的盈利机会，而且有些机会只有量化交易才能利用。比如你发现一种交易方法，其特点是盈亏的额度相等，但盈利的概率是 55%，亏损概率 45%。首先这种小差距的概率规律，非量化交易不能发现，其次，要利用这个规律盈利需要大量次数的交易才能稳定盈利，这也非量化交易不可。

5. 有助于抵御情绪对投资的影响。情绪是投资的一大天敌，量化可以很大程度剔除冲动或优柔寡断情绪的干扰，客观理性的通过数据进行分析，指导你果断的进行交易。

1.3 量化的步骤

1. 从一个灵感开始

灵感就是指那些你想验证的可能会盈利的方法，比如净利润增长率可能是良好的指标、一旦跨过 20 日均线后股价会继续涨等等。灵感获取的方式可以是阅读、听人说、自己悟等等。

这里我们以一个简单的情况为例进行讲解。比如你的灵感是这样的：

如果股价显著低于近几日的平均价，则买入
如果股价显著高于近几日的平均价，则卖出

现在，你想知道这样操作究竟会不会赚钱？

2. 把灵感细化成明确的可执行的交易策略

一般灵感都很模糊，需要将其细化成明确的可执行的策略，目的是为了能得到确定的结果，以及为后续程序化准备。需要明确什么条件下买卖，买卖什么品种，买卖多少量等，从而形成一个明确的交易策略，让不同人根据你的描述在相同情形下都能做出相同的操作。

继续以之前那个关于平均价的灵感为例。显然它是不够明确的。比如多低叫显著低于？多高叫显著高于？近几日究竟是几日？买入卖出是买卖多少？我们把它细化：

每个交易日监测是除 `st` 股外的国内 `A` 股的所有股票的股价
如果股价低于近 20 日平均价 10%，则用全部可用资金买入该股票
如果股价高于近 20 日平均价 10%，则卖出全部所持有的该股票

现在我们基本已经把之前的灵感细化成明确的可执行的交易策略。当然，可能还有些地方不够明确，也可能有些细节还不确定要改动，这些可以随时想到随时再改，不必一次做到完美。

3. 把策略转成程序

就是把明确后的策略通过编程转成程序，好让计算机能根据历史数据模拟执行该策略，以及能根据实际行情进行反应并模拟交易或真实交易。

简言之，就是把刚刚的策略翻译成计算机可识别的代码语言，写成类似这样的代码（下面的代码并不完全符合，只是展示下大概的样子）：

```
1 def initialize():
2     g.security = ['002043.XSHE', '002582.XSHE']
3 def handle_data():
4     for i in g.security:
5         data = get_current_data(i)
6         last_price = data.close
7         average_price = data.mavg(20, 'close')
8         cash = context.portfolio.cash
9         if last_price > average_price:
10             order_value(i, cash)
11         elif last_price < average_price:
12             order_target(i, 0)
```

这样一来，就把刚才细化好策略转成了代码程序，计算机就能运行了。这个过程你可以理解成用计算机能听懂的语言（代码），把你的策略告诉给计算机了。

4. 检验策略效果

现在计算机理解了你的策略，你现在可以借助计算机的力量来验证你的策略了。基本的检验策略方法有回测和模拟交易两种方法。

4.1 回测

回测是让计算机能根据一段时间的历史数据模拟执行该策略，根据结果评价并改进策略。继续之前的那个均价的策略例子的话就是这样的：

设定初始的虚拟资产比如 500000 元、一个时期比如 20060101 到 20160101，把这一时期的各种数据如估计股价行情等发给计算机，计算机利用这些数据模仿真实的市场，执行你刚才告诉它的策略程序。最后计算机给你一份报告，根据这个报告你就会知道，在 20060101 的 500000 元，按照你的策略交易到 20160101，会怎样？一般包括盈亏情况，下单情况，持仓变化，以及一些统计指标等，从而你能据此评估交易策略的好坏。

如果结果不好，则需要分析原因并改进。如果结果不错，则可以考虑用模拟交易进一步验证。

4.2 模拟交易

模拟交易是让计算机能根据实际行情模拟执行该策略一段时间，根据结果评价并改进策略。与回测不同，回测是用历史数据模拟，模拟交易使用实际的实时行情来模拟执行策略的。举例就是这样：

设定初始的虚拟资产比如 500000 元，选择开始执行模拟交易的时间点，比如明天。那么从明天开始，股市开始交易，真实的行情数据就会实时地发送到计算机，计算机利用真实的数据模仿真实的市场，执行你的策略程序。同时，你会得到一份实时更新的报告。这报告类似于回测得到的报告，不同的是会根据实际行情变化更新。同样你能据此评估交易策略的好坏。可见，回测是用历史数据模拟执行策略，模拟交易是用未来的实际行情模拟执行策略。如果策略在回测与模拟交易的表现都非常好，你可以考虑进行完全真实的真金白银的实盘交易。

5. 进行实盘交易并不断维护修正

实盘交易就是让计算机能自动根据实际行情，用真金白银自动执行策略，进行下单交易。注意，这时不再是用虚拟资产模拟交易，亏损和盈利都是真钱。实盘交易一般也会给出一份类似模拟交易的会不断更新的报告，从而不断要观察策略的实盘表现并及时调整与改进策略，使之持续平稳盈利。

1.4 做量化交易需要什么

- 要有证券市场各种数据。要有能方便使用的各种投资相关的数据。这要考虑到各种数据的收集、存储、清洗、更新，以及数据取用时的便捷、速度、稳定。
- 要有一套量化交易的系统。要有能编写策略、执行策略、评测策略的系统。这要考虑到系统对各种策略编写的支持、系统进行回测与模拟的高仿真、系统执行策略的高速、系统评测策略的科学可靠全方面。
- 要能基本掌握 **Python** 编程。**Python** 语言简单易懂，适合非计算机专业人员学习，不但入门容易，而且将来深入下去，可以编写那些非常非常复杂的程序。**Python** 提供非常完善的基础代码库，许多功能不必从零编写，直接使用现成的即可。

Note: QFF 框架提供了一整套用于量化交易所需要的数据和系统，使您量化交易的成本极大降低。

1.5 Python 语言的学习资源

1. [python 官方教程中文版](#)
2. [廖雪峰的 Python 新手教程](#)
3. [【视频】Python 入门 | 慕课网](#)
4. [菜鸟教程](#)

QFF 是基于 Python 语言开发的一个量化金融框架库，提供数据获取-数据存储-策略编写-策略分析-策略回测-模拟交易等一站式解决方案，用于为个人用户提供本地的回测和模拟交易环境，使用户更加专注于交易策略的编写。

提示

请访问 [Github](#) 网站获取 QFF 源代码，如果有任何问题请访问 [Github Issues](#) 查询或提交，或加入 QQ 群 775477625 参与讨论。

2.1 主要功能

- 1. 提供免费的金融数据接口。**提供国内股票、指数、基金等金融产品的基本面数据、实时和历史行情数据、技术指标等。所有数据均从互联网上抓取，并经过专业清洗保存在用户自己搭建的数据库服务器中。
- 2. 提供灵活优雅的策略编写接口。**提供类似聚宽的策略程序框架，及灵活优雅的策略编写接口。策略文件中可以使用所有的 Python 标准库和用户自己常用的第三方库，用户可以很方便的实现自己的交易思路，编写自己的策略。
- 3. 提供本地运行的回测和模拟交易环境。**支持本地编写和运行用户的策略文件，进行回测或模拟交易。通过选择回测日期区间，测试用户策略在历史数据下的收益情况，如果回测结果良好，可以进一步利用实时数据进行模拟交易，以验证策略的可用性，从而帮助用户做出理性的投资决策。
- 4. 提供丰富的策略分析报告。**在对策略进行回测和模拟交易时，计算策略运行的各种风险指标和绩效指标，以帮助用户识别策略的优劣，并输出内容丰富的策略分析报告。另外，针对股票择时策略（附加的策略模板），可以单独输出策略分析报告。

5. 提供实用的策略编写辅助功能。为方便用户编写策略，QFF 收集了常用的公共函数，包括技术指标计算，同花顺公式基础函数等等。另外，针对仓位管理、风险管理、止盈止损策略等，提供了一个交易系统框架，可提供给用户编写策略时使用，用户也可方便的扩展，构建自己的交易系统。

2.2 主要特点

易于使用	让您专注于策略的开发，一行简单的命令就可以执行您的策略
接口丰富	提供优雅的策略编写接口，方便用户快速入门实现自己的策略。
运行高效	提供本地的运行环境，相较其他量化框架，提升策略运行效率。
数据免费	提供丰富的数据接口，以获取免费的股票和市场数据。
策略安全	用户策略在本地编辑、运行、评估，充分保障策略安全。
文档完善	提供最佳的文档支持，每个接口均提供详细的说明和示例。
代码简单	使用 Python 语言开发，避免使用复杂的数据结构，代码容易理解。

2.2.1 与其他量化框架比较

框架	QFF	JoinQuant	rqalpha	QUANTAXIS	Backtrader
运行环境	本地	网络	本地	本地	本地
数据获取	免费	收费	收费	免费	无
学习难度	简单	未开源	复杂	中等	复杂
文档支持	完善	完善	完善	一般	完善

2.3 声明

- QFF 当前仅支持国内 A 股数据及交易，暂不支持其他金融产品如：期货、外汇、基金、债券、加密货币等。
- QFF 提供的所有数据仅用做学术研究，不得用于任何商业用途。
- QFF 提供的所有数据和运行结果仅作为参考意见，不构成任何投资建议。
- 任何基于 QFF 研究的投资者需关注数据风险。
- QFF 将持续提供开源的金融数据及研究框架。

- 基于某些无法控制的原因，QFF 的某些数据接口未来可能会被移除。
- 请用户遵守 QFF 相关开源协议。

2.4 鸣谢

首先要特别感谢QUANTAXIS在项目开发上对本人提供的借鉴和学习的机会。

特别感谢JoinQuant提供的对于本项目框架和接口方面的学习和借鉴的机会。

特别感谢AKShare和rqalpha提供的对于本项目文档方面的学习和借鉴的机会。

注解

- 如果执行 `pip install` 安装依赖库网络速度比较慢的话，推荐使用 `pip install -i https://pypi.douban.com/simple` 国内镜像来加速。
 - 为避免因为环境问题出现安装失败，建议您使用虚拟环境安装，Python 虚拟环境有 `venv`, `conda`, `pipenv` 等，请参考[相关教程](#)。
 - 推荐使用 `python -m venv venv` 创建虚拟环境，并执行 `venv\Scripts\activate` 激活运行。
-

3.1 安装库

以下命令将安装 QFF 及其依赖项。

```
$ pip install qff
```

QFF 依赖于许多其他 Python 包，使用软件包安装工具像 PIP 或 Conda 等将确保安装必要的依赖软件包，您可以使用包管理器命令 `pip show qff` 查看依赖项，更多信息可以在其 [PyPI](#) 页面上找到。

P.S. 由于目前版本更新迭代频繁，且国内镜像同步时间不一致，请在[使用](#)以下命令进行 QFF 升级：

```
$ pip install -U qff -i https://pypi.python.org/simple/
```

查看 QFF 是否安装成功可以通过如下方式：

```
$ qff --version
```

3.2 数据准备

注解

QFF 框架提供所有数据均从互联网上抓取，并经过专业清洗保存在用户自己搭建的数据库服务器中，这样设计的优点是增强数据的可靠性，数据永久保存在用户自己手上，避免后期因为网站更新造成数据缺失或格式不匹配等问题。

因此，在 QFF 正常使用前，需进行数据库的安装配置，以及股票数据的下载和更新。

3.2.1 数据库安装

QFF 使用的是 MongoDB 数据库，它是一个高性能、无模式、文档型的 NoSQL 数据库。

先进入 [mongodb](#) 的官网[MongoDB](#)下载，选择要下载的版本以及系统，MongoDB 数据库安装教程参考如下：

操作系统平台
Windows
Linux
Mac OS

我们强烈推荐您直接拉取[Docker](#) 镜像安装使用 **MongoDB** 数据库。本人选择 **4.4.2** 版本镜像进行安装。

3.2.2 数据库配置

MongoDB 数据库安装成功后，需设置 QFF 与数据库的连接参数，设置命令如下：

```
$ qff config set MONGODB.uri=mongodb://xxx.xxx.xxx.xxx:27017
```

如果 MongoDB 数据库设置了用户名密码，则需配置相关认证信息：

```
$ qff config set MONGODB.uri=mongodb://admin:*****@xxx.xxx.xxx.xxx:27017/?  
←authSource=admin&authMechanism=SCRAM-SHA-256
```

另外，可使用命令 `qff config list` 查看 QFF 所有配置信息。

Important: 注：如果 **MongoDB** 数据库安装在本机，则无需配置连接参数，QFF 使用默认连接参数。

3.2.3 数据下载

QFF 在正常使用前需下载历史数据，执行方法如下所示：

```
$ qff save all
```

执行该命令将从互联网下载所有股票、指数、ETF 基金的历史行情数据和基本面数据等，并保存至您安装的 **MongoDB** 数据库中，根据您的计算机和网络性能，下载时间将超过 5 个小时。

您也可以通过执行 `qff save` 命令选择下载您所需要的数据类别。

qff save all	保存/更新所有数据
qff save day	保存/更新股票日数据、指数日数据、ETF 日数据
qff save min	保存/更新股票分钟数据、指数分钟数据、ETF 分钟数据
qff save stock_list	保存/更新股票列表数据
qff save stock_day	保存/更新股票日线数据
qff save index_day	保存/更新指数日线数据
qff save etf_day	保存/更新 ETF 日线数据
qff save stock_min	保存/更新股票分钟数据
qff save index_min	保存/更新指数分钟数据
qff save etf_min	保存/更新 ETF 分钟数据
qff save stock_xdxr	保存/更新日除权除息数据
qff save stock_block	保存/更新板块股票数据
qff save report	保存/更新股票财务报表
qff save valuation	保存/更新股票市值数据
qff save mtss	保存/更新融资融券数据
qff save index_stock	保存/更新指数成分股信息
qff save indus-try_stock	保存/更新行业成分股信息
qff save init_info	初始化股票列表、指数列表、ETF 列表
qff save init_name	初始化股票历史更名数据
qff save save_delist	保存退市股票的日数据和分钟数据

Note:

1. 如果您打算使用 `qff save xxxx` 分项下载数据，则在第一次初始化过程时，务必首先执行 `qff save init_info` 命令。
2. 市值数据是基于股票日数据、财报数据以及除权除息数据计算生成，注意按顺序下载更新。
3. 如果您需要查询股票历史更名数据，需手工执行 `qff save init_name`，否则将使用最近的股票名称，影响回测时 ST 股的判断。

3.2.4 数据自动更新

通过配置数据自动更新服务，QFF 可以自动下载更新每日股市收盘后最新数据。指令 `qff save all` 具有数据更新功能，您需要在您使用的操作系统中，自行配置定时任务，定时任务开始时间建议 16:00，执行周期即每个工作日。

我们强烈推荐您使用 **QFF Docker 镜像** 进行数据自动更新。

3.2.5 数据库维护

1. 您可以通过执行 `qff dbinfo` 命令查看 `qff` 数据库当前信息，包括数据集合名称、记录数量、集合大小、存储容量、索引信息以及最后一次更新日期等。
2. 您也可以通过执行 `qff drop` 命令删除指定的数据表，再通过 `qff save` 命令生成新的数据集合。

说明

鉴于 QFF 的安装涉及数据库及数据运维软件的安装配置，导致用户难以在短时间部署，为此我们专门设计了 Docker 镜像，方便您直接拉取使用。

4.1 安装 Docker

Docker 是一个开源的应用容器引擎，可以让开发者打包他们的应用以及依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的 Linux 机器上，也可以实现虚拟化。

4.1.1 安装指导

1. Docker 安装教程
2. 建议 Windows 7 和 8 的用户升级到 Windows 10/11 系统进行安装
3. Windows 镜像下载地址

4.1.2 配置国内镜像

1. Docker 国内镜像加速教程
2. 请在国内使用的用户务必进行该项配置，从而加速获取镜像的速度。

4.2 部署 MongoDB 镜像

Important: 由于 QFF 数据运维需每天定时更新证券数据，强烈建议您配置自己的云服务器或家庭NAS 服务器, 并将 MongoDB 和 QFF 镜像部署到您的服务器上。

Docker 安装 MongoDB 教程

4.2.1 拉取 mongo 镜像

选择合适的 mongo 镜像版本，本教程选择镜像体积较小且稳定版本 4.2.2，可访问https://hub.docker.com/_/mongo了解更多信息。

```
docker pull mongo:4.2.2
```

4.2.2 运行 mongo 容器

1. 普通运行:

```
docker run -d -p 27017:27017 mongo:4.2.2
```

2. 指定数据保存路径如果为方便以后数据库迁移，可以指定容器内数据库文件映射到外部的文件路径:

```
docker run -d -v /root/db /data/db -p 27017:27017 mongo:4.2.2
```

其中/root/db 是外部服务器保存数据目录的路径

3. 如果您的数据库不是仅在局域网内使用，基于安全考虑，必须设置账户密码。设置方法如下:

```
docker run -d -v MONGO_INITDB_ROOT_USERNAME admin -v MONGO_INITDB_ROOT_PASSWORD_
↵xxxxxxx mongo:4.2.2
```

其中 xxxxxx 为您设置的密码。

4.3 部署 QFF 镜像

此镜像会在每次 QFF 更新版本时自动更新

4.3.1 拉取 QFF 镜像

```
docker pull haijiangxu/qff
```

4.3.2 运行 QFF 容器

1. 普通运行:

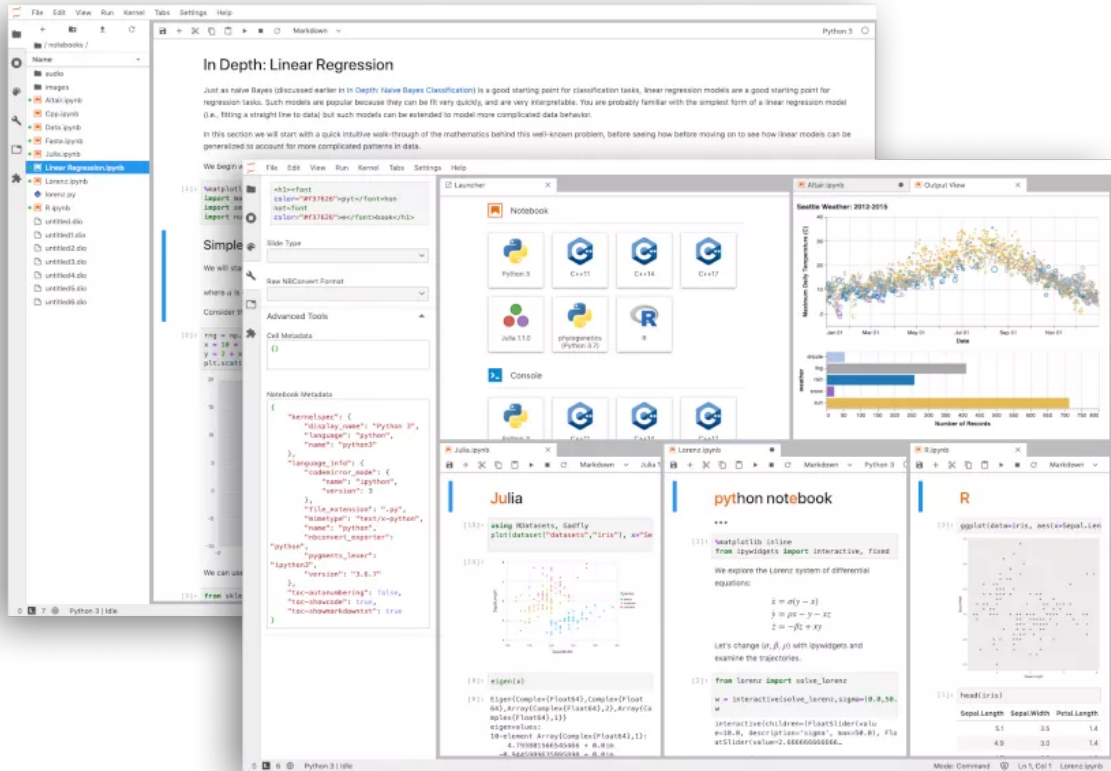
```
docker run -d -p 8765:8765 qff
```

2. 如果 MongoDB 设置了密码, 则需做以下配置:

```
docker run -d -p 8765:8765 -v /root/work:/root/work -e MONGODB_URI="mongodb://  
↪admin:xxxxxx@localhost/?authSource=admin&authMechanism=SCRAM-SHA-256" qff
```

4.3.3 镜像使用

- QFF 镜像基于 Ubuntu20.04 版本 OS, 安装了 QFF 及所有依赖库, 并安装了 cron 定时服务, 镜像已配置了自动数据更新服务, 启动 QFF 容器后, 您不用再关心数据更新问题。
- QFF 镜像还安装了 JupyterLab, 方便用户直接在浏览器使用。打开本地浏览器输入地址: `http://服务器地址:8765`, 显示密码框提示, 输入密码: `qff` 后, 将进入 JupyterLab 界面。



Important: 由于 QFF 目前还处于开发阶段，版本不稳定，在使用 docker 镜像时，可以在 JupyterLab 的终端中运行 `pip install -U qff` 升级 QFF 库版本。

QFF 框架读取用户编写的策略文件，根据输入的执行参数进行回测或模拟交易，并在执行完成后自动生成相应的策略评价报告。

说明

我们假设 QFF 已经正确安装在您的系统中，并且已经完成了相应数据库安装和股票数据的下载更新，如果有任何安装相关的问题，请首先查看[安装指南](#)

5.1 开始编写第一个策略

QFF 采用了函数式编程，以 API 的方式提供给策略研发者用于编写策略，从而避免陷入过多的技术细节，而非金融程序建模本身。

5.1.1 下载策略模板

在命令行终端中，创建并进入项目目录，然后运行以下指令：

```
$ qff create
>>> 请输入待创建的策略文件名称[默认:example.py]:test
>>> 策略文件 test.py 创建成功!
```

5.1.2 编写自己的策略代码

使用 Python 开发工具 PyCharm, VsCode, Sublime 等打开下载的策略模板文件, 并添加自己的策略代码。

下面是一个简单而又完整的策略:

```
1  # 导入函数库
2  from qff import *
3
4
5  # 初始化函数, 设定基准等等
6  def initialize(context):
7      # 输出内容到日志 log.info()
8      log.info('初始函数开始运行且全局只运行一次')
9
10     # 设定沪深 300 作为基准
11     set_benchmark('000300')
12
13     # 股票相关设定 ###
14     # 股票类每笔交易时的手续费是: 买入时佣金万分之三, 卖出时佣金万分之三加千分之一印花
    税, 每笔交易佣金最低扣 5 块钱
15     set_order_cost(open_tax=0,
16                     close_tax=0.001,
17                     open_commission=0.0003,
18                     close_commission=0.0003,
19                     min_commission=5)
20
21     # 开盘前运行
22     run_daily(before_market_open, run_time='before_open')
23     # 开盘时运行
24     run_daily(market_open, run_time='every_bar')
25     # 收盘后运行
26     run_daily(after_market_close, run_time='after_close')
27
28
29 # 开盘前运行函数
30 def before_market_open(context):
31     # 输出运行时间
32     log.info('函数运行时间 (before_market_open): ' + context.current_dt)
33
```

(continues on next page)

(continued from previous page)

```

34     # 要操作的股票：平安银行（g. 为全局变量）
35     g.security = '000001'
36
37
38 # 开盘时运行函数
39 def market_open(context, data):
40     log.info('函数运行时间 (market_open):'+context.current_dt)
41     security = g.security
42     # 获取股票的收盘价
43     close_data = attribute_history(security, count=5, unit='1d', fields=['close'])
44     # 取得过去五天的平均价格
45     MA5 = close_data['close'].mean()
46     # 取得上一时间点价格
47     current_price = close_data['close'][-1]
48     # 取得当前的现金
49     cash = context.portfolio.available_cash
50
51     # 如果上一时间点价格高出五天平均价 1%，则全仓买入
52     if (current_price > 1.01*MA5) and (cash > 0):
53         # 记录这次买入
54         log.info("价格高于均价 1%%, 买入 %s" % security)
55         # 用所有 cash 买入股票
56         order_value(security, cash)
57     # 如果上一时间点价格低于五天平均价，则空仓卖出
58     elif current_price < MA5 and \
59         security in context.portfolio.positions.keys() and\
60         context.portfolio.positions[security].closeable_amount > 0:
61         # 记录这次卖出
62         log.info("价格低于均价, 卖出 %s" % security)
63         # 卖出所有股票，使这只股票的最终持有量为 0
64         order_target(security, 0)
65
66
67 # 收盘后运行函数
68 def after_market_close(context):
69     log.info(str('函数运行时间 (after_market_close):'+context.current_dt))
70
71

```

一个完整策略只需要两步：

- 设置初始化函数:[initialize], 上面的例子中, 注册每个交易日盘前、盘中、盘后运行的函数。
- 编写函数代码, 实现突破 5 日均线买入, 下穿 5 日均线卖出的策略。

5.2 回测自己编写的策略

5.2.1 命令行运行

在命令行终端内, 执行 `qff run -h` 可以查看命令参数说明, 然后执行以下语句:

```
$ qff run test.py -s 2022-06-01 -e 2022-10-31
```

5.2.2 集成开发环境中运行

调用 `run_file()` 函数执行当前策略, 在策略文件尾部添加以下代码, 您可以直接在终端运行 `python test.py` 或在集成开发环境中运行。

```
1 if __name__ == '__main__':  
2     run_file(__file__, start="2022-06-01", end="2022-10-31")
```

5.2.3 查看策略回测结果

策略回测结束后, 将在 `~\.qff\output\back_test\` 目录下保存回测结果, 生成两个文件:

1. 一个 `pkl` 文件, 包含策略运行过程中 `context` 对象的所有信息
2. 一个 `html` 文件, 即策略运行报告。



5.3 实盘模拟编写的策略

5.3.1 启动实盘模拟

在命令行终端执行以下指令，将启动策略的实盘模拟，设置初始资金 10 万 (默认值 100 万)。

```
$ qff sim test.py --cash 100000
```

5.3.2 执行过程中交互

实盘模拟运行后，将在真实交易时间执行您的策略代码，命令行终端内将连续打印策略的日志信息。您可以通过输入 `log` 命令开关日志信息，并输入命令与 QFF 进行交互，首先输入 `help` 可以查看所有交互命令。

```
-----欢迎进入QFF交互环境(log命令可开关日志, help命令获取帮助)-----
QFF>
QFF>
QFF>
QFF> help

Possible commands are:
info ----- 查询策略运行数据
ks ----- 查询交易股票的K线图
kg ----- 显示全局对象g中name属性对应的数据和K线图
kshow ----- 查询股票K线图
time ----- 查询策略时间
fn ----- 运行自定义语句
shell ----- 运行外部程序
log ----- 设置日志输出级别
pause ----- 暂停回测任务
resume ----- 恢复回测任务
cancel ----- 回测任务取消, 退出交互环境
quit ----- 退出交互环境, 回测仍在运行
exit ----- 程序完全退出
ls ----- 列出当前路径
QFF> 66> 0000-00-01 14:55:00 INFO 1 1 1 函数运行
```

通过 `info` 命令可以随时查看策略当前实盘模拟的运行效果。

说明

QFF 交互环境在回测过程同样可以使用。

一个完整的策略包括选股、择时、止盈、止损、风险控制、仓位控制等几部分内容，QFF 框架为您实现策略提供了所有的支持。我们从最基础的策略入手，推荐以下编写策略文件思路：

1. 在开盘前执行复杂耗时的策略代码，比如针对历史行情或基本面数据筛选当日的股票池，或者判断当前的大盘环境以决策当日的仓位等。
2. 在盘中阶段，如果策略执行频率是分钟或者 **tick**，应该尽量减少策略逻辑的复杂度，比如：可以根据待选股票当日的走势判断决策是否买入，或者进行止盈止损操作等。
3. 收盘后由于此时当日行情数据还未保存至数据库，只能分析和输出当日的操作结果。

以下是实现策略代码的关键 API 说明：

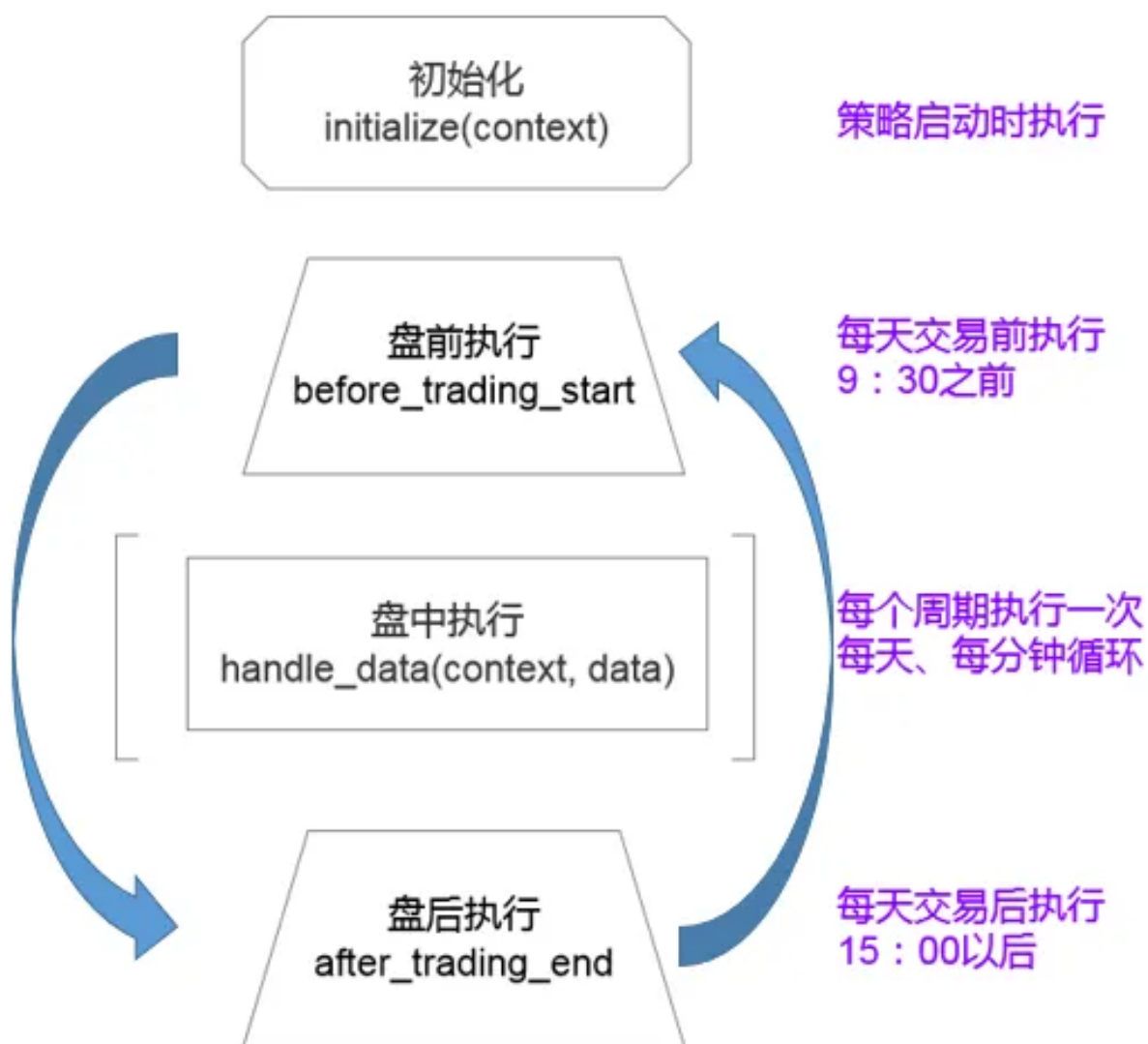
6.1 框架接口函数

用户编写的策略文件中必须包含（或部分包含）以下约定的框架接口函数，并根据策略逻辑将代码分别放置不同的函数来实现。QFF 框架正确载入策略文件后，会自动在相应的时间调用这些函数。

接口名称	描述
<i>initialize</i>	初始化函数 (必须)
<i>handle_data</i>	盘中策略运行函数 (可选)
<i>before_trading_start</i>	开盘前运行策略 (可选)
<i>after_trading_end</i>	收盘后运行策略 (可选)
<i>on_strategy_end</i>	策略运行结束时调用 (可选)
<i>process_initialize</i>	每次程序启动时运行函数 (可选)

Warning: 注意: 如果是使用框架接口函数, 请严格按照 API 中的名称参数进行定义。如果要自定义函数, 请避免和框架接口函数重名, 否则将引起报错。

6.1.1 框架流程图



6.1.2 运行时间

策略代码什么时候运行，主要由设置策略频率（天、分钟或者 tick）与框架接口函数共同完成。

- **before_trading_start**: 与策略频率无关，每个交易日开盘前运行，这里设计为 09:00 运行。
- **after_trading_end**: 与策略频率无关，每个交易日收盘后运行，这里设计为 15:30 运行。
- **handle_data**: 此函数实现策略逻辑的核心代码，是在交易日盘中运行，具体运行时间取决于策略运行频率。

6.1.3 运行频率

策略运行频率可以理解为函数的调用频率，也就是盘中阶段您多久去看一眼 K 线图。从整个策略层面上讲，QFF 有三种不同粒度的频率，分别为（天，分钟，tick）：

1. 天级别: **handle_data** 的执行时间为 09:30 时执行一次，此时刻的 ohcl 数据均为开盘价。
2. 分钟级别: **handle_data** 在股票的交易时间段中每分钟执行一次。
3. tick 级别: **handle_data** 在股票的交易时间段中每 3 秒执行一次。**tick** 级别仅支持模拟交易中使用，回测不能使用

Note: 注意：每日分钟 k 线数据 9:30-11:00 13:01-15:00，共 241 条数据

4. 自定义运行频率：一般来讲，自定义运行频率可以使用三种方法，一种就是通过定时器去判断执行次数，在一定的执行次数下再执行后边需要执行的代码；第二种是判断时间，当当前时间的分钟是某个数字的倍数时再运行，比如现在是 09:50,50 是 10 的倍数，执行，否则跳过；第三种是直接设置多个定时运行。

示例：

```

1
2  def initialize(context): # 这里选择策略频率为分钟，所以 func 函数每分钟都会执行一次
3      g.run_num = -1 # 初始化计数器，不要在盘中开启策略
4
5  def handle_data(context, data):
6      g.run_num += 1

```

(continues on next page)

(continued from previous page)

```

7     if g.run_num!=15:  # 如果这次运行的次数不是 15，打断此次运行
8         return
9     else:
10        g.run_num=0  # 条件满足，计数器归零，继续后边的代码
11
12    print('后边的代码将每隔 15 分钟运行一次')
```

6.1.4 定时运行

用户可以通过在 `initialize(context)` 函数中调用 `run_daily()` 函数设置定时运行的策略代码。

```

1
2    def initialize(context):
3        # 设置每个交易日 09:40 运行策略函数 my_fun1()
4        run_daily(my_fun1, run_time='09:40')
5        # 设置每个交易日 10:40 运行策略函数 my_fun1()
6        run_daily(my_fun1, run_time='10:40')
7        # 设置每个交易日 14:10 运行策略函数 my_fun2()
8        run_daily(my_fun2, run_time='10:40')
9
10
11    def my_fun1(context):
12        # 打印运行的时间
13        print('my_fun1 运行{}'.format(context.current_time))
14
15    def my_fun2(context):
16        # 打印运行的时间
17        print('my_fun2 运行{}'.format(context.current_time))
18
```

- 如果策略文件中自定义的函数有通过 `run_daily` 进行注册，则根据注册时的参数，系统会自动以固定的时间调用注册的函数。
- 通过设置 `run_daily` 的 `run_time` 参数，可以替代其他框架约定接口函数，具体参考 [run_daily API](#)。
- 定时运行的时间必须在盘中时间，即 9:31-11:00 13:01-15:00。
- 可以在相同的时间点设置不同的策略函数，也可以在不同的时间点设置相同的策略函数。

数，但要避免与框架接口函数重名。

- 如果用户自定义的函数没有进行注册，且函数名不属于框架接口函数，则系统不会主动调用。

Note: 一个策略中尽量不要同时使用 `run_daily` 和 `handle_data`，更不能使用 `run_daily(handle_data, “xx:xx”)`，建议使用 `run_daily`。

6.2 数据获取函数

理解策略程序框架，您就可以开始编写策略了，但策略的实现需要基于股票数据的分析，以决定股票买卖的信号。QFF 提供了丰富的数据查询 API, 包括股票、指数、基金的行情数据、基本面数据以及融资融券数据等，具体参考[数据 API](#)

6.2.1 历史数据

策略文件一般在开盘前（`before_trading_start` 函数中）进行选股或大盘择时等策略的实现，此时需获取策略运行当日之前的历史数据，在回测中，历史数据的获取非常容易用到未来函数，从而造成策略结果失真。例如开盘前获取当天的收盘价、财务数据、技术指标等。为了避免未来函数，我们提供的大多数函数支持获取历史任意时刻信息。

以下是常用到的一些函数说明：

接口名称	功能	备注
<code>get_price()</code>	获取历史数据, 可查询多个标的多个数据字段	避免在策略中使用, 容易产生未来函数
<code>history()</code>	获取历史数据, 可查询多个标的单个数据字段 (回测专用)	无未来函数
<code>attribute_his</code>	获取历史数据, 可查询单个标的多个数据字段 (回测专用)	无未来函数
<code>get_bars()</code>	获取历史数据 (包含快照数据), 可查询多个标的多个数据字段 (回测专用)	无未来函数
<code>get_all_secur</code>	获取平台支持的所有股票信息	给参数 <code>date</code> 赋值为 <code>context.previous_date</code>
<code>get_stock_nan</code>	获取股票给定日期的名称	给参数 <code>date</code> 赋值为 <code>context.previous_date</code>
<code>get_index_stc</code>	获取一个指数给定日期的成分股列表	给参数 <code>date</code> 赋值为 <code>context.previous_date</code>
<code>get_financial</code>	查询多只股票给定日期的财务数据	给参数 <code>date</code> 赋值为 <code>context.previous_date</code>
<code>get_fundament</code>	根据 <code>mongodb</code> 语法查询财务数据	注意查询日期赋值
<code>query_valuati</code>	查询满足条件的市值信息数据	注意查询日期赋值

其他函数参考数据 [API](#)

6.2.2 当前数据

在 `handle_data` 函数实现的盘中策略逻辑中, 需根据股票当前时刻价格进行判断, 以决定股票买卖的信号。

QFF 提供两种途径获取回测或模拟实盘中当前时刻标的的数据, `handle_data()` 函数自带 `data` 参数和 `get_current_data()` 函数, `data` 参数对象实际是对 `get_current_data` 函数的封装, 返回数据内容参见 `SecurityUnitData` 类。

示例:

```

1  def initialize(context):
2      g.stock = '000001'
3
4  def handle_data(context, data):
5      if data[g.stock].last_price / data[g.stock].pre_close - 1 > 0.05:

```

(continues on next page)

(continued from previous page)

```

6         log.info("股票{}当前涨幅大于 5%! ".format(g.stock))
7
8     # 或者
9     d = get_current_data(g.stock)
10    if d.last_price / d.pre_close - 1 > 0.05:
11        log.info("股票{}当前涨幅大于 5%! ".format(g.stock))
12
13

```

注意：**get_current_data** 函数只能够在盘中运行的策略中使用

6.2.3 全局对象

策略代码编写时，会经常使用到下面两个全局变量对象，其他所有的类和对象，请参考[基础 API](#)

1. 全局对象 **g**:

g 对象用来存储用户的各类可被 **pickle.dumps** 函数序列化的全局数据，即需在不同策略函数中使用的变量，可定义成 **g** 对象的属性，这些变量在策略暂停后，会自动保存，并在策略恢复运行后，变量值能够恢复。

示例：

```

1
2     def initialize(context):
3         g.stock = '000001'
4
5     def handle_data(context, data):
6         log.info("g.stock 值为{}".format(g.stock))
7

```

如果不想 **g** 中的某个变量被序列化，可以让变量以 ‘__’ 开头，这样，这个变量在序列化时就会被忽略。

2. 全局对象 **context**:

context 对象保存了策略运行过程中的上下文环境。包括策略的账户信息、运行参数以及当前运行状态。对象数据内容参见 [Context](#) 类。

用户可以在策略代码中直接读取 **context** 相关属性，但注意不能直接修改 **context** 的任何属性值。

示例:

```
1 def handle_data(context, data):
2     log.info("账户初始金额为{}".format(context.portfolio.starting_cash))
3     log.info("账户当前可用金额为{}".format(context.portfolio.available_cash))
4
```

6.3 交易执行函数

至此, 我们已经能够获取股票数据, 并可通过代码实现买卖信号逻辑, 那么接下来就需要调用交易接口来进行交易了。

6.3.1 交易函数接口

QFF 提供了多种交易接口, 以方便不同的使用需求:

<code>order</code>	按股票数量下单
<code>order_value</code>	按股票价值下单
<code>order_target</code>	按股票目标数量下单
<code>order_target_value</code>	按股票目标价值下单
<code>order_cancel</code>	撤回已下的订单
<code>get_orders</code>	获取当天的所有订单
<code>get_open_orders</code>	获得当天的所有未完成的订单

6.3.2 订单交易流程

1. 从委托到成交的流程: 订单创建-> 订单检查-> 报单-> 确认委托-> 撮合, 在订单检查时未通过则订单取消。
2. 为简化操作, QFF 未启用盘口撮合, 撮合时不考虑成交量, 一个订单一次成交记录。
 - 市价单买入时按当前价格 + 滑点价格, 转成限价单。如果当前价格为涨停价格, 则订单取消。
 - 市价单卖出时按当前价格-滑点价格, 转成限价单。如果当前价格为跌停价格, 则订单取消。
3. 订单撮合规则:

- 如果运行频率为天，则下单后立即撮合，读取剩余的分钟数据曲线，判断最高价是否大于委托价，是则成交。
- 如果运行频率为分钟（或 **tick**），则下单后，每分钟（或 **tick**）都判断该订单是否符合成交条件。

4. 对未成交的订单，在本交易日结束后撤销。

7.1 回测和模拟

7.1.1 回测

Note: 回测是让 QFF 框架根据一段时间的历史数据模拟执行您编写的策略，根据结果评价并持续改进策略，以获得满意的结果。

回测过程如下：

1. 准备好您的策略文件。
2. 选定一个回测开始和结束日期，选择初始资金、执行频率（每天还是每分钟），开始回测。
3. QFF 根据您的策略文件，按照设置的执行频率，模拟历史日期和时间，循环调用您的策略函数。
4. 当您下单后，QFF 会根据接下来时间的实际交易情况，撮合处理您的订单。
5. 下单后您可以调用 `get_open_orders` 取得所有未完成的订单，调用 `order_cancel` 取消订单
6. 您可以在任何时候调用 `log.info/debug/warn/error` 函数来打印一些日志。
7. 回测结束后我们会生成策略分析报告，画出您的收益和基准（参见 `set_benchmark()`）收益的曲线，列出每日持仓，每日交易和一系列风险数据。

7.1.2 模拟交易

Note: 模拟交易是在回测的基础上创建的，连接实时的市场数据，根据真实的市场环境进行策略模拟。

1. 模拟交易无需设置开始和结束日期，只需选择初始资金和执行频率。从策略执行启动开始，模拟交易就开始运行。除非收到结束命令或者外部中断，模拟交易才会结束。
2. 如果执行频率设置为‘每天’，并且启动时已经开盘，则在当前时间点之前运行的函数不再运行，比如：今天 10:00 创建了按天的模拟交易，代码中实现了 `handle_data` 和 `after_trading_end`，则 `handle_data` 今天不运行，而 `after_trading_end` 会在 15:30 运行。
3. 模拟交易每天收盘后 15:30，会生成截止当日的策略分析报告，并且会自动保存运行环境，类似执行暂停操作。
4. 模拟交易运行过程中可以人工干预暂停，暂停时会自动保存 `context` 对象和 `g` 对象，后期策略重启将会完全恢复暂停前的状态，模拟交易将继续执行。

7.2 多种方式运行策略

在策略开发过程中，每个人都有各自的需求。例如，他们可能需要在不同的时间周期进行回测，或者希望能够将回测好的策略直接用于实盘交易。有些人可能更喜欢基于回测数据进行编程分析，或者只是想快速查看收益结果。

为了满足这些不断变化的需求，设计 QFF API 时，将这部分需要以参数方式进行配置的严格剥离出代码层面。这样一来，同一份策略代码可以通过传入不同的参数来实现完全不同的策略开发、风控、运行和调优等功能。

7.2.1 命令行运行

在命令行模式中，我们预先定义了常用的运行参数，您可以直接在控制台输入参数来配置策略的运行，但并不是所有的参数都需要配置，此处列出一些常用参数选项，您可以执行 `qff run -h` 查询。

参数缩写	参数全称	说明
无	strategy-file	策略文件名称路径
-n	-name	策略名称, 默认为策略文件名
-rt	-run-type	策略运行类型, bt 为回测, sim 为模拟交易, 默认为 bt
-f	-freq	策略执行频率, 可选 (day, min, tick), 默认 day
-c	-cash	账户初始资金, 默认 ¥1,000,000
-s	-start	回测开始日期, 默认为结束日期前 60 个交易日
-e	-end	回测结束日期, 默认为当日之前最近一个交易日
-f	-freq	策略运行结果输出到指定目录, 默认 ./qff/output/
-t	-trace	策略运行过程中是否进行交互, 模拟交易时自动生效
无	-resume	恢复前期暂停的策略运行, 一般用于模拟交易中
-l	-log-level	设置控制台日志输出的级别, 可选 (verbose, info, warning, error), 默认 info

示例:

```
# 以回测模式执行当前目录下 test.py 策略, 回测周期 2022-06-01 至 2022-10-31, 执行频率天,
初始资金 1000000
$ qff run test.py -s 2022-06-01 -e 2022-10-31
```

7.2.2 在代码中运行

QFF 的一个重要的优势在于支持本地编写和运行用户的策略文件, 这样您可以使用 Py-Charm 等集成开发环境提供的如自动代码补全、代码检查、实时错误高亮等丰富功能, 提升策略编写效率, 减少代码错误概率。

QFF 支持在集成开发环境中运行、调试策略文件的功能, 如下所示, 在策略文件尾部添加以下代码, 调用 `run_file()` 函数可执行 (或调试) 当前策略。

```
1 if __name__ == '__main__':
2     run_file(__file__, start="2022-06-01", end="2022-10-31")
```

您也可以直接在命令行策略文件当前目录下输入 `python test.py` 命令, 执行策略文件。

7.3 运行过程中交互

QFF 提供在策略运行过程中的人机交互功能接口 **trace**，模拟交易时在控制台自动打开。

您可以通过输入 **log** 命令开关控制台日志信息（日志文件保存不受影响），并输入命令与 QFF 进行交互，首先输入 **help** 可以查看所有交互命令。

7.3.1 命令总览

命令	说明
info	查询策略运行数据
ks	查询交易股票的 K 线图
kg	显示全局对象 g 中 name 属性对应的数据和 K 线图
kshow	查询股票 K 线图
trader	操作同花顺下单软件客户端
time	查询策略时间
fn	运行自定义语句
shell	运行外部程序
log	设置日志输出级别
pause	暂停策略运行
resume	恢复策略运行
cancel	策略运行取消，退出交互环境
quit	退出交互环境，回测仍在运行
exit	程序完全退出
ls	列出当前路径

Note: 回测时需通过参数 **-t** 打开 **trace** 功能, 如果日志输出太频繁, 可能无法通过 **log** 命令关闭, 可在启动时添加参数 **-l warning**

7.3.2 查询策略运行数据

info 命令可以查看策略当前运行的状态和数据，直接输入 info 命令将显示以下子命令信息：

子命令	说明
info 0	输出回测框架/模拟实盘运行基本参数
info 1	输出当前股票账户资金状况及当前账户持仓情况详情
info 2	输出策略运行历史账户资产数据
info 3	输出策略运行历史订单交易数据
info 4	输出策略运行当日订单委托信息
info 5	输出策略运行分析报告
info 6	输出策略风险分析指标数据
info 7	输出策略绩效分析指标数据

示例：

```
QFF>
```

```
QFF>info 0
```

```
+-----+
| 回测框架/模拟实盘运行基本参数 |
+-----+-----+
| 策略名称 |          dbma          |
| 框架类型 |          回测          |
| 当前状态 |          运行中        |
| 运行频率 |          day           |
| 开始日期 | 2021-05-13             |
| 结束日期 | 2022-05-12             |
| 回测周期 |          242           |
| 当前日期 | 2021-07-20 09:30:00    |
| 运行天数 |          48            |
| 基准指数 |          000300        |
| 初始资金 |          1000000       |
+-----+-----+
```

```
QFF>info 1
```

```
+-----+
|      账户当前资金情况      |
+-----+
```

(continues on next page)

(continued from previous page)

日期	2021-06-25
现金资产	443304.66
持仓资产	713630.0
账户总资产	1156934.66
累计收益额	156934.66
累计收益率	0.1569
当日盈亏金额	740.0
当日涨幅	0.0006
仓位	0.6168
基准总资产	1041076.01
基准收益率	0.0411

-----+

前持仓情况

日期	股票代码	股票名称	持仓数量	今开数量	可用数量	平均成本	当前价格	浮动盈亏	浮动盈亏率	当日盈亏	当日盈亏率	持仓天数	当日市值	仓位占比
2021-06-28	000009	中国宝安	25000	0	25000	9.81	16.65	171000.0	69.72%	7500.0	1.83%	23	416250.0	35.98%
2021-06-28	000007	*ST 全新	74000	0	74000	3.79	4.11	23680.0	8.44%	0.0	0.00%	4	304140.0	26.29%

```
QFF> info 6
```

(continues on next page)

(continued from previous page)

当前日期:2021-12-14 09:30:00

+-----+			
	策略风险系数风险		
+-----+			
	策略收益	7.62%	
	策略年化收益	13.62%	
	基准收益	0.78%	
	超额收益	6.79%	
	阿尔法	0.106	
	贝塔	0.384	
	夏普率	0.628	
	索提诺比率	0.927	
	信息比率	1.090	
	卡玛比率	0.556	
	omega 比率	0.002	
	策略最大回撤	24.49%	
	超额最大回撤	-20.99%	
	日均超额收益	0.05%	
	超额夏普率	0.561	
	基准年化收益	1.36%	
	策略波动率	0.254	
	日胜率	51.72%	
	资金使用率	48.38%	
	最大回撤起点	2021-07-22	
	最大回撤终点	2021-11-26	
+-----+			

QFF> info 7

当前日期:2021-12-28 09:30:00

+-----+			
	策略绩效分析		
+-----+			
	交易净利润	31566.02	
	总盈利金额	383567.62	
	总亏损金额	-352001.6	
	总盈亏比	1.09	
	盈利次数	8	
	亏损次数	31	
	胜率	20.51%	

(continues on next page)

(continued from previous page)

```

| 平均盈利金额 | 47945.95 |
| 平均亏损金额 | -11354.89 |
| 平均盈亏比 | 4.22 |
| 最大单笔盈利 | 263750.0 |
| 最大单笔亏损 | -23807.0 |
| 最大盈亏比 | 11.08 |
| 交易标的数量 | 39 |
| 平均持仓周期 | 7.18 |
| 交易费用总额 | 15063.09 |
| 交易费用占比 | 47.72% |

```

```

+-----+

```

```

+-----+

```

```

↪-----+

```

```

|                                     账户交易配对情况                                     |

```

```

↪-----+

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

↪-----+-----+-----+-----+-----+

```

```

| index | code | name | buy_date | buy_price | sell_date | sell_price |

```

```

↪amount | pnl_return | pnl_money | hold_gap |

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

↪-----+-----+-----+-----+-----+

```

```

| 0 | 000007 | *ST 全新 | 2021-05-17 | 3.8 | 2021-05-18 | 3.5 |

```

```

↪65900 | -7.89% | -19770.0 | 2 |

```

```

| 1 | 000006 | 深振业 A | 2021-05-28 | 4.98 | 2021-05-31 | 4.95 |

```

```

↪49100 | -0.60% | -1473.0 | 2 |

```

```

| 2 | 000007 | *ST 全新 | 2021-06-01 | 3.71 | 2021-06-03 | 3.63 |

```

```

↪66700 | -2.16% | -5336.0 | 3 |

```

```

| 3 | 000006 | 深振业 A | 2021-06-23 | 4.95 | 2021-06-24 | 4.88 |

```

```

↪56600 | -1.41% | -3962.0 | 2 |

```

```

| 4 | 000001 | 平安银行 | 2021-06-28 | 23.01 | 2021-06-29 | 22.1 |

```

```

↪12600 | -3.95% | -11466.0 | 2 |

```

```

| 5 | 000001 | 平安银行 | 2021-07-02 | 22.37 | 2021-07-05 | 21.23 |

```

```

↪13100 | -5.10% | -14934.0 | 2 |

```

```

| 6 | 000007 | *ST 全新 | 2021-06-23 | 3.79 | 2021-07-27 | 4.31 |

```

```

↪74000 | 13.72% | 38480.0 | 25 |

```

```

| 7 | 000009 | 中国宝安 | 2021-05-26 | 9.81 | 2021-07-29 | 20.36 |

```

```

↪25000 | 107.54% | 263750.0 | 46 |

```

```

| 8 | 000007 | *ST 全新 | 2021-07-30 | 4.41 | 2021-08-03 | 4.11 |

```

```

↪70700 | -6.80% | -21210.0 | 3 |

```

(continues on next page)

(continued from previous page)

9	000007	*ST 全新	2021-08-10	4.36	2021-08-11	4.26	下
↪73400	-2.29%	-7340.0	2				
10	000006	深振业 A	2021-08-12	4.55	2021-08-13	4.51	下
↪69700	-0.88%	-2788.0	2				
11	000001	平安银行	2021-08-13	19.48	2021-08-18	19.26	下
↪16000	-1.13%	-3520.0	4				
12	000006	深振业 A	2021-08-17	4.53	2021-08-18	4.42	下
↪60700	-2.43%	-6677.0	2				
13	000001	平安银行	2021-08-19	20.17	2021-08-23	19.09	下
↪15500	-5.35%	-16740.0	3				
14	000002	万科 A	2021-08-12	20.55	2021-08-26	19.81	下
↪15400	-3.60%	-11396.0	11				
15	000007	*ST 全新	2021-08-26	4.23	2021-08-27	4.16	下
↪75900	-1.65%	-5313.0	2				
16	000006	深振业 A	2021-08-26	4.52	2021-08-30	4.4	下
↪71100	-2.65%	-8532.0	3				
17	000007	*ST 全新	2021-08-30	4.22	2021-08-31	4.15	下
↪74200	-1.66%	-5194.0	2				
18	000002	万科 A	2021-09-03	19.91	2021-09-06	19.48	下
↪15500	-2.16%	-6665.0	2				
19	000009	中国宝安	2021-07-30	21.4	2021-09-06	22.06	下
↪14500	3.08%	9570.0	27				
20	000001	平安银行	2021-09-08	18.95	2021-09-10	18.49	下
↪15400	-2.43%	-7084.0	3				
21	000002	万科 A	2021-09-14	19.94	2021-09-15	18.76	下
↪14600	-5.92%	-17228.0	2				
22	000006	深振业 A	2021-09-02	4.5	2021-09-16	4.54	下
↪67000	0.89%	2680.0	11				
23	000001	平安银行	2021-09-13	20.05	2021-09-17	18.5	下
↪14500	-7.73%	-22475.0	5				
24	000002	万科 A	2021-09-24	19.25	2021-09-27	18.9	下
↪14500	-1.82%	-5075.0	2				
25	000006	深振业 A	2021-09-24	4.56	2021-09-27	4.41	下
↪61200	-3.29%	-9180.0	2				
26	000002	万科 A	2021-09-29	20.1	2021-10-19	19.24	下
↪13600	-4.28%	-11696.0	10				
27	000007	*ST 全新	2021-09-28	4.07	2021-10-21	4.08	下
↪67700	0.25%	677.0	13				
28	000002	万科 A	2021-10-25	19.63	2021-10-26	19.31	下

(continues on next page)

(continued from previous page)

[illegible]

7.3.3 查看股票 K 线图

trace 模块提供了三种查看 k 线图命令，方便您在模拟交易（或回测运行）中随时生成并查看股票的 K 线。

1. kshow 命令

kshow 命令输入需观察的股票代码，在临时目录 `~/ .qff/temp` 下生成 **k** 线图文件 `kline chart.html`，如果是 **windows** 系统，将自动打开默认浏览器。**kshow** 命令语法如下：

kshow 000001	从当前日期（或回测运行）查询指定股票代码的日 K 线，默认 500 条数据
kshow 000001 200	从当前日期（或回测运行）查询指定股票代码的日 K 线，显示设置的数据条数
kshow 000001 - 20	在回测状态下，查询当前日期前 500 条到后 20 条数据。
kshow index 000001	从当前日期（或回测运行）查询指数代码的日 K 线，默认 500 条数据
kshow index 000001 200	从当前日期（或回测运行）查询指数代码的日 K 线，显示设置的数据条数

2. ks 命令

ks 命令用于查看历史成交记录中股票的 K 线图，并标注买卖点。用于分析策略买点和卖点的正确性。

ks 命令语法：

- **ks**: 不带参数运行将输出当前已经配对的成交记录列表。
- **ks n**: 生成以上成交记录列表中第 **n** 条记录对应的股票 **k** 线图，并标注买入卖出点。

K线图-600658 电子城



3. kg 命令

kg 命令用于分析保存在 **g** 全局变量中定义股票池中的股票，这些股票被您的策略代码选中，但未被成功买入。此时，您可能希望掌握选中和未交易成功的原因，**trace** 模块提供了 **kg** 命令能够让您快速查看。

kg 命令语法:

- **kg name:** 显示全局对象 **g** 中 **name** 属性对应列表中的数据。**g.name** 为列表对象, **g.name** 列表中数据项如需进行 K 线显示, 需满足以下格式:
[股票代码, 标注点 1 日期, 标注点 1 价格, 标注点 2 日期, 标注点 2 价格, 其他信息...]
- **kg name n:** 显示全局对象 **g** 中 **name** 属性对应列表中股票的日 K 线, 并标注信息点, **n** 代表第 **n** 条数据。

示例:

```
QFF>
```

```
QFF>kg lost_stocks
```

```
# 您需要在您的策略代码中, 将股票池中未成交或丢弃的股票添加至 g.lost_stocks 中
```

7.4 暂停和恢复

模拟交易时, 策略根据真实的市场环境, 启动后将持续运行。您可能希望策略只在交易时间内运行, 或者执行一段时间后希望少量优化您的代码, 此时您需要暂停和恢复操作。

7.4.1 暂停

暂停方法有以下两种:

1. 在 **trace** 人机交互环境中, 输入 **pause** 命令, 策略将启动暂停操作, 此时提示您输入备份文件名称, 并提供默认文件名 (策略文件名 **_sim.pkl**), 直接回车则选择默认文件名称。QFF 会通过 **pickle.dumps** 函数序列化 **context** 和 **g** 两个全局变量写入备份文件中, 然后策略退出运行。备份文件保存在 **~/.qff/cache** 目录下, 您在 **trace** 中输入 **exit** 命令, 则完全退出进程。
2. 模拟交易每天 15:30 会自动保存运行环境, 并以默认文件名称保存备份文件在 **~/.qff/cache** 目录下, 您可以在 **after_trading_end** 接口函数中, 添加 **sys.exit(1)** 退出策略模拟交易进程。

Note: 暂停时使用默认文件名称, 会覆盖以前的备份文件, 即只保存最新的策略运行环境。

7.4.2 恢复

通过命令行参数，有两种方法进行恢复操作：

1. 如果您使用默认文件作为备份文件名，则您可以通过 `qff run < 您的策略文件名称> --resume` 命令，恢复最后一次暂停时备份的运行环境。
2. 如果您使用自己命名的备份文件名称，则您可以通过 `qff resume < 您的备份文件名称>` 命令，恢复您指定 `pkl` 文件所保存的运行环境。

Note: 如果您希望只在交易时间运行模拟交易，可以将 `qff run < 您的策略文件名称> --resume` 命令设置在您的定时任务中，启动时间需设置在交易日 09:00 前。

8.1 策略评价报告

策略评价报告在下列情况下生成：

1. 回测运行完成后，自动生成策略评价报告。
2. 模拟交易每个交易日 15:30，会生成从模拟交易起始日期到当前的策略评价报告。
3. 在回测或模拟交易的 `trace` 环境中，输入 `info 5` 命令，会产生当时的评价报告

Note: 评价报告以 `html` 文件格式展现，保存在 `~/qff/output` 目录下。

8.1.1 收益概述页面

策略运行效果在此页面非常友好展现，策略收益率、基准收益、超额收益以及每日仓位占比等信息以曲线图形展示，一目了然。同时计算了策略运行的各种风险指标。



8.1.3 交易详情页面

交易详情页面只是简单罗列策略运行期间的每笔交易，供您进行策略分析。

策略名称dbl_amount

状态100.0% 已完成

参数回测，2021-08-27到2022-03-25，¥1000000，day

策略收益

交易分析

交易详情

持仓详情

账户详情

日志输出

策略启动时间

2023-02-17 21:56:07

运行总耗时

0天 0时 37分 36秒

交易详情

交易时间	股票代码	股票名称	交易方向	成交数量	成交单价	成交金额	交易费用
2021-08-27 14:55	600398	海澜之家	买入	14400股	¥ 6.94	¥99955.99	¥ 19.99
2021-08-27 14:55	603031	安德利	买入	3100股	¥ 31.38	¥97266.45	¥ 19.45
2021-08-30 14:55	600658	电子城	买入	24400股	¥ 4.1	¥100060.01	¥ 20.01
2021-08-31 14:55	600463	空港股份	买入	15200股	¥ 6.62	¥100644.12	¥ 20.12
2021-08-31 14:55	002729	好利来	买入	5600股	¥ 18.0	¥100820.16	¥ 20.16
2021-08-31 14:55	600215	长春经开	买入	14200股	¥ 7.1	¥100840.16	¥ 20.16
2021-09-01 14:55	601766	中国中车	买入	15000股	¥ 6.6	¥99019.8	¥ 19.8
2021-09-01 14:55	000596	古井贡酒	买入	400股	¥ 219.28	¥87713.54	¥ 17.54
2021-09-01 14:55	600018	上港集团	买入	17700股	¥ 5.59	¥98962.79	¥ 19.79
2021-09-01 14:55	300168	万达信息	买入	7500股	¥ 13.14	¥98569.71	¥ 19.71
2021-09-01 14:55	600690	海尔智家	买入	500股	¥ 28.43	¥14215.0	¥ 5.0
2021-09-02 14:50	600658	电子城	卖出	-24400股	¥ 4.29	¥-104794.1	¥ 125.9

8.1.4 持仓详情页面

持仓详情页面只是简单罗列策略运行期间每日仓位情况，供您进行策略分析。

策略名称

dbl_amount

状态

100.0% 已完成

参数

回测，2021-08-27到 2022-03-25，¥1000000，day

策略收益

交易分析

交易详情

持仓详情

账户详情

日志输出

策略启动时间

2023-02-17 21:56:07

运行总耗时

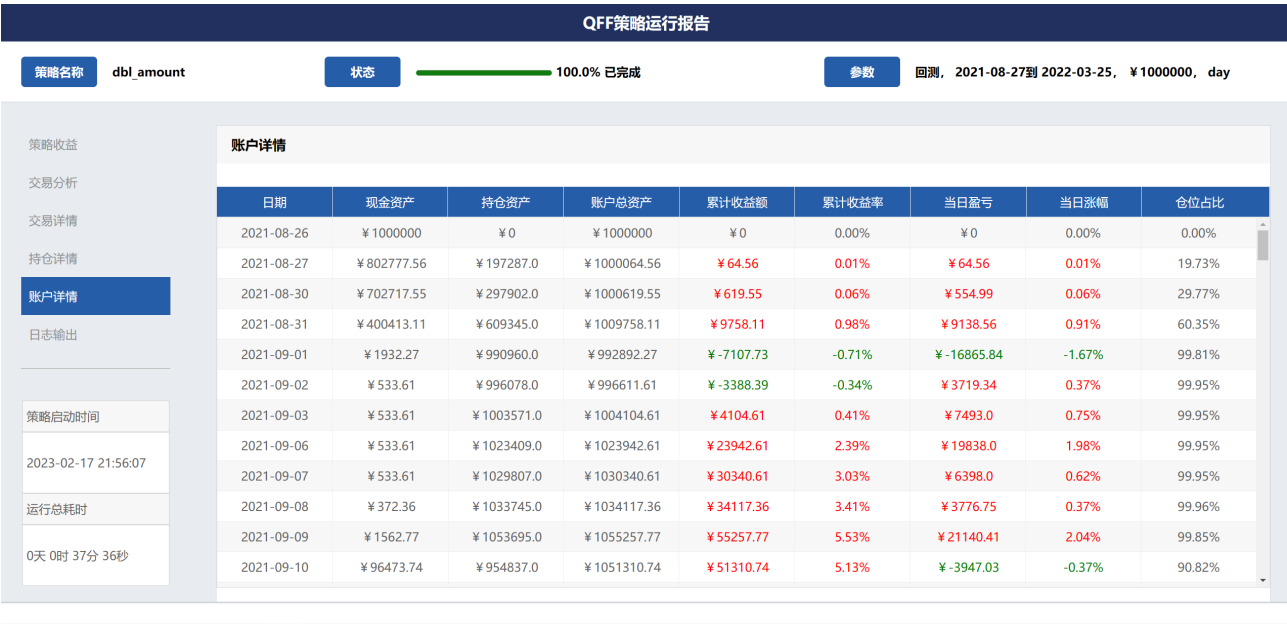
0天 0时 37分 36秒

持仓详情

交易日期	股票	持仓数量	平均成本	当日收盘	当日市值	持有天数	仓位占比	浮动盈亏	当日盈亏
2021-08-27	海澜之家(600398)	14400	¥ 6.94	¥ 6.93	¥ 99792.0	1	9.98%	-0.14%	-0.14%
2021-08-27	安德利(603031)	3100	¥ 31.38	¥ 31.45	¥ 97495.0	1	9.75%	0.22%	0.22%
2021-08-30	海澜之家(600398)	14400	¥ 6.94	¥ 6.89	¥ 99216.0	2	9.92%	-0.72%	-0.58%
2021-08-30	安德利(603031)	3100	¥ 31.38	¥ 31.9	¥ 98890.0	2	9.88%	1.66%	1.43%
2021-08-30	电子城(600658)	24400	¥ 4.1	¥ 4.09	¥ 99796.0	1	9.97%	-0.24%	-0.24%
2021-08-31	海澜之家(600398)	14400	¥ 6.94	¥ 6.77	¥ 97488.0	3	9.65%	-2.45%	-1.74%
2021-08-31	安德利(603031)	3100	¥ 31.38	¥ 32.27	¥ 100037.0	3	9.91%	2.84%	1.16%
2021-08-31	电子城(600658)	24400	¥ 4.1	¥ 4.49	¥ 109556.0	2	10.85%	9.51%	9.78%
2021-08-31	空港股份(600463)	15200	¥ 6.62	¥ 6.61	¥ 100472.0	1	9.95%	-0.15%	-0.15%
2021-08-31	好利来(002729)	5600	¥ 18.0	¥ 17.98	¥ 100688.0	1	9.97%	-0.11%	-0.11%
2021-08-31	长春经开(600215)	14200	¥ 7.1	¥ 7.12	¥ 101104.0	1	10.01%	0.28%	0.28%
2021-09-01	海澜之家(600398)	14400	¥ 6.94	¥ 6.69	¥ 96336.0	4	9.70%	-3.60%	-1.18%

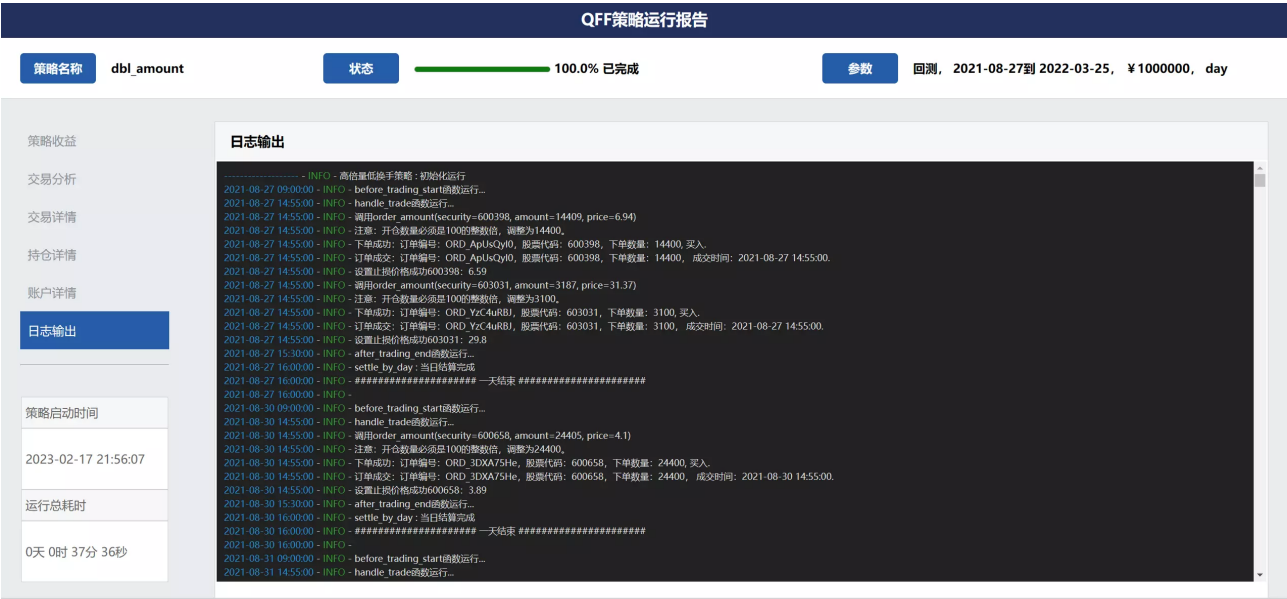
8.1.5 账户详情页面

账户详情页面简单罗列策略运行期间每日账户资产、当日和累计盈亏、当日仓位占比等信息，供您进行策略分析。



8.1.6 日志输出页面

日志输出页面调用日志文件，将完整的日志信息在此页面上展示，并对告警和错误信息进行颜色标注。



8.2 风险指标说明

风险指标数据有利于您对策略进行一个客观的评价, 包含以下风险指标:

名称	描述
<i>Total Returns</i>	策略收益
<i>Total Annualized</i>	年化收益率
<i>Max Drawdown</i>	最大回撤
<i>Alpha</i>	阿尔法
<i>Beta</i>	贝塔
<i>Sharpe Ratio</i>	夏普比率
<i>Sortino Ratio</i>	索提诺比率
<i>Calmar Ratio</i>	卡玛比率
<i>Omega Ratio</i>	Omega 比率
<i>Information Ratio</i>	信息比率
<i>Algorithm Volatility</i>	策略波动率
<i>Benchmark Volatility</i>	基准波动率

8.2.1 策略收益

$$Total\ Returns = (P_{end} - P_{start}) / P_{start} * 100\%$$

P_{end} = 策略最终股票和现金的总价值

P_{start} = 策略开始股票和现金的总价值

8.2.2 年化收益率

年化收益率是用来衡量资产或投资组合的收益率的一种指标, 表示资产或投资组合在一年内所获得的平均收益率。

$$Total\ Annualized\ Returns = R_p = ((1 + P)^{\frac{250}{n}} - 1) * 100\%$$

P = 策略收益

n = 策略执行天数

年化收益率通常用于与其他投资组合或指标进行比较, 以评估投资组合的表现。年化收益率还可以用来估算投资组合的未来表现, 但需要注意的是, 过去的表现并不一定代表未来的表现, 投资者应该谨慎使用。

8.2.3 最大回撤

描述策略可能出现的最糟糕的情况，最极端可能的亏损情况。

$$\text{Max Drawdown} = \text{Max}((P_x - P_y) / P_x)$$

P_x, P_y = 策略某日股票和现金的总价值 $y > x$

8.2.4 阿尔法

投资中面临着系统性风险（即 **Beta**）和非系统性风险（即 **Alpha**），**Alpha** 是投资者获得与市场波动无关的回报。比如投资者获得了 15% 的回报，其基准获得了 10% 的回报，那么 **Alpha** 或者价值增值的部分就是 5%。

$$\text{Alpha} = R_p - [R_f + \rho(R_m - R_f)]$$

R_p = 策略年化收益率
 R_m = 基准年化收益率
 R_f = 无风险利率（默认 0.04）
 ρ = 策略 **beta** 值

Alpha 值	解释
$\alpha > 0$	策略相对于风险，获得了超额收益
$\alpha = 0$	策略相对于风险，获得了适当收益
$\alpha < 0$	策略相对于风险，获得了较少收益

8.2.5 贝塔

表示投资的系统性风险，反映了策略对大盘变化的敏感性。例如一个策略的 **Beta** 为 1.5，则大盘涨 1% 的时候，策略可能涨 1.5%，反之亦然；如果一个策略的 **Beta** 为 -1.5，说明大盘涨 1% 的时候，策略可能跌 1.5%，反之亦然。

$$\text{Beta} = \rho = \frac{\text{Cov}(D_p, D_m)}{\text{Var}(D_m)}$$

D_p = 策略每日收益
 D_m = 基准每日收益
 $\text{Cov}(D_p, D_m)$ = 策略每日收益与基准每日收益的协方差
 $\text{Var}(D_m)$ = 基准每日收益的方差

Beta 值	解释
$\beta < 0$	投资组合和基准的走向通常反方向，如做空类
$\beta = 0$	投资组合和基准的走向没有相关性，如固定收益类
$0 < \beta < 1$	投资组合和基准的走向相同，但是比基准的移动幅度更小
$\beta = 1$	投资组合和基准的走向相同，并且和基准的移动幅度贴近
$\beta > 1$	投资组合和基准的走向相同，但是比基准的移动幅度更大

8.2.6 夏普比率

夏普比率（Sharpe Ratio）是一种常用的风险调整收益的指标，用于衡量投资组合每承担一单位风险所获得的超额收益。夏普比率越高，表明每承担一单位风险所获得的超额收益越高，投资组合的风险调整收益能力越强。

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p}$$

R_p = 策略年化收益率
 R_f = 无风险利率（默认 0.04）
 σ_p = 策略收益波动率

夏普比率衡量投资组合的风险调整收益能力，可以帮助投资者评估投资组合的表现，但需要注意的是，夏普比率存在一定的局限性，主要表现在以下几个方面：

- 假设收益呈正态分布：夏普比率假设收益呈正态分布，但实际上收益可能存在非正态分布，因此可能会影响夏普比率的准确性。
- 无法区分波动率的来源：夏普比率无法区分波动率的来源，因此可能会低估投资组合的整体风险水平。
- 对收益的定义有局限性：夏普比率只考虑了收益的平均值，而没有考虑收益的分布情况，因此可能会忽略收益的非对称分布。

综上所述，夏普比率是一种常用的风险调整收益指标，可以帮助投资者评估投资组合的风险调整收益能力，但在实际应用中需要综合考虑其他因素，并注意其局限性。

8.2.7 索提诺比率

表示每承担一单位的下行风险，将会获得多少超额回报。

$$\text{Sortino Ratio} = \frac{R_p - R_f}{pd}$$

R_p = 策略年化收益率
 R_f = 无风险利率（默认 0.04）
 pd = 策略下行波动率

索提诺比率的值越大，代表单位风险所获得的收益越高，也就是说，该投资组合相对于风险而言获得的收益越多，因此索提诺比率越高，代表该投资组合的风险调整收益能力越强。

判断一个投资组合的索提诺比率是否好，需要考虑以下几个因素：

- 索提诺比率的大小：索提诺比率越大，代表投资组合的风险调整收益能力越强，因此索提诺比率越大越好。
- 目标收益率的选择：目标收益率的选择应该根据投资者的需求和市场状况来确定。对于风险偏好较高的投资者，可以选择较高的目标收益率；对于风险偏好较低的投资者，则可以选择较低的目标收益率。同时，目标收益率也需要根据市场状况进行调整。
- 投资组合下行风险的标准差：下行风险指的是收益率低于目标收益率的风险，因此，下行风险的标准差越小，代表投资组合的风险越低，因此索提诺比率越高。
- 历史数据的时间跨度：索提诺比率的计算需要使用历史数据，因此历史数据的时间跨度也会对计算结果产生影响。一般而言，历史数据的时间跨度越长，计算出来的索提诺比率越有参考价值。

需要注意的是，索提诺比率只是一种风险调整收益的指标，不能完全代表一个投资组合的优劣。在实际应用中，还需要综合考虑其他因素，如投资组合的风险水平、投资策略的风格、资产配置等。

8.2.8 卡玛比率

卡玛比率（Calmar Ratio）是另一种衡量风险调整收益能力的指标，它将投资组合的年化收益率与最大回撤之比作为风险调整收益的指标，用来评估投资者在承担风险的情况下，获得的收益是否合理。卡玛比率越高，说明投资组合在承担相同的风险下，获得的收益越高，反之亦然。

卡玛比率的公式为：

$$\text{Calmar Ratio} = \frac{\text{年化收益率}}{\text{最大回撤}}$$

其中，年化收益率指的是投资组合在一定时间内的平均年化收益率，最大回撤指的是投资组合在某一段时间内最大的下跌幅度。一般情况下，最大回撤是指从高点到低点的跌幅。

与索提诺比率类似，卡玛比率也是一种风险调整收益的指标，可以帮助投资者评估投资组合的风险调整收益能力。与索提诺比率不同的是，卡玛比率将最大回撤考虑在内，更加注重保护资金安全，因此可以帮助投资者更好地控制风险。不过，卡玛比率也有一些局限性，需要综合考虑其他因素，如投资组合的风险水平、资产配置等。

8.2.9 欧米茄比率

欧米茄比率（Omega Ratio）是一种金融风险指标，它是一个基于收益率分布的风险调整收益指标，主要用于评估投资组合的风险和收益。

Omega 比率的计算公式如下：

$$\text{Omega Ratio} = \frac{P(R > k)}{P(R < k)}$$

其中， $P(R > k)$ 表示收益率大于某个阈值 k 的概率， $P(R < k)$ 表示收益率小于等于阈值 k 的概率。

Omega 比率的值越大，代表单位风险所获得的收益越高，也就是说，该投资组合相对于风险而言获得的收益越多，因此 Omega 比率越高，代表该投资组合的风险调整收益能力越强。

判断一个投资组合的 Omega 比率是否好，需要考虑以下几个因素：

- **Omega 比率的大小：**Omega 比率越大，代表投资组合的风险调整收益能力越强，因此 Omega 比率越大越好。
- **阈值 k 的选择：** k 是关键参数，不同的阈值会对 Omega 比率的计算结果产生影响。一般而言，阈值的选择应该根据投资者的风险偏好来确定。对于风险偏好较高的投资者，可以选择较高的阈值；对于风险偏好较低的投资者，则可以选择较低的阈值。
- **历史数据的时间跨度：**Omega 比率的计算需要使用历史数据，因此历史数据的时间跨度也会对计算结果产生影响。一般而言，历史数据的时间跨度越长，计算出来的 Omega 比率越有参考价值。

需要注意的是，Omega 比率只是一种风险调整收益的指标，不能作为独立的评价指标。在评估投资组合时，还需要综合考虑其他指标，如夏普比率、索提诺比率等。

8.2.10 信息比率

衡量单位超额风险带来的超额收益。信息比率越大，说明该策略单位跟踪误差所获得的超额收益越高，因此，信息比率较大的策略的表现要优于信息比率较低的基准。合理的投资目标应该是在承担适度风险下，尽可能追求高信息比率。

$$Information\ Ratio = \frac{R_p - R_m}{\sigma_t}$$

R_p = 策略年化收益率
 R_m = 基准年化收益率
 σ_t = 策略与基准每日收益差值的年化标准差

8.2.11 策略波动率

用来测量策略的风险性，波动越大代表策略风险越高。

$$Algorithm\ Volatility = \sigma_p = \sqrt{\frac{250}{n-1} \sum_{i=0}^n (r_p - \bar{r}_p)^2}$$

r_p = 策略每日收益率
 $\bar{r}_p$ = 策略每日收益率的平均值 = $\frac{1}{n} \sum_{i=0}^n r_{pi}$
 n = 策略执行天数

8.2.12 基准波动率

用来测量基准的风险性，波动越大代表基准风险越高。

$$Benchmark\ Volatility = \sigma_m = \sqrt{\frac{250}{n-1} \sum_{i=0}^n (r_m - \bar{r}_m)^2}$$

r_m = 基准每日收益率
 $\bar{r}_m$ = 基准每日收益率的平均值 = $\frac{1}{n} \sum_{i=0}^n r_{mi}$
 n = 基准执行天数

QFF 实盘操作基于通用同花顺下单软件，通过 `pywinauto` 库自动化接管客户端，实现股票实际账号的买入、卖出、撤单、账户查询、委托以及成交查询等功能。

Note: QFF 实盘交易仅支持 Windows 系统，其功能在 0.5.15 版本后开始支持。

9.1 安装和设置

9.1.1 客户端安装

登录 [同花顺官网](#)，选择同花顺免费版下载并安装。如果最新版本操作出现错误，可从网盘下载兼容的历史版本 `THS_9.20.40_20230613.exe`，提取码 `qff0`。

9.1.2 客户端设置

- 添加客户端快捷方式。软件位置在同花顺安装目录下 `xiadan.exe`，找到该文件后鼠标右键点击，选择发送到-> 桌面快捷方式。在运行 `qff` 实盘操作前，一般需先启动下单软件。
- 执行 `qff` 命令行指令，将下单软件路径保存在 `qff` 配置文件中。如何 `xiadan.exe` 程序已运行，`qff` 实盘可直接接管，否则，将通过 `qff` 配置文件信息，自动启动下单软件。

```
$ qff config set THS.path c:\ths\xiadan.exe
```

- 需要先手动登录一次下单软件。添加券商，填入账户号、密码、验证码，勾选“保存密码”。第一次登录后，上述信息被缓存。

委托下单

证券公司: 国盛证券 添加

站点列表: 联通站点2(171.34.206.!) 连最快站点

资金帐户: 88888888

交易密码: *****

加密协议: 验证码

验证码: 4489 4 4 8 9

☒ 保存密码 ☐ 自动登录

登录

- 需要对客户端按以下设置，不然会导致客户端超时锁定。系统设置 > 界面设置：界面不操作超时时间设为 0。

系统设置

系统设置 交易设置 界面设置 快速交易

登录后默认页面	3.资金股票
下单窗口总在最前面	否
买入双击股票切换到卖出	是
是否在行情分时图上显示买卖标记	是
查询界面自动刷新时间(秒)	0
界面不操作超过多久锁屏(分)	0
解锁密码连续几次错误程序退出	0
最小化标题显示	默认
下单界面字体大小	正常字体
买卖界面自动刷新时间(秒)	0
隐藏盘口红蓝色柱体	否
点击委托客户端右上角“X”按钮	最小化下单
显示悬浮工具栏	是
行情代码切换时联动委托代码	否
是否隐藏个人信息	是
修改价格显示对应涨跌幅	是

确定

取消

9.2 命令行指令

QFF 提供了用于操作同花顺客户端的命令行指令。在进行实盘交易前，可通过这些命令行指令进行测试验证。您可以执行 `qff trader -h` 查询 `trader` 命令用法：

```
$ qff trader -h

usage:
  qff trader <subcommand> [options]
eg:
  qff trader connect
  qff trader balance
  qff trader position
  qff trader entrust
  qff trader deal
  qff trader buy stock_code amount price
  qff trader sell stock_code amount price
  qff trader buy stock_code amount
  qff trader sell stock_code amount
  qff trader cancel entrust_no
```

操作同花顺下单软件客户端，用于策略实盘运行前，测试能否正确对交易软件进行操作。

子命令：

- connect:	连接交易软件客户端
- balance:	获取账户资金股票信息
- position:	获取账户持仓股票列表
- entrust:	获取当日委托订单列表
- deal:	获取当日成交订单列表
- buy stock amount price:	买入指定股票，eg: <code>qff trader buy 000001 100 10.5</code>
- sell stock amount price:	卖出指定股票。eg: <code>qff trader sell 000001 100 10.</code>

↩9

- buy stock amount:	市价买入指定股票，eg: <code>qff trader buy 000001 100</code>
- sell stock amount:	市价卖出指定股票。eg: <code>qff trader sell 000001 100</code>
- cancel entrust_no:	撤销委托订单， <code>entrust_no</code> 为委托订单编号，如果不输

入，则撤销所有委托订单

positional arguments:

subcommand 实盘操作子命令

(continues on next page)

options 子命令所需参数

optional arguments:

-h, --help 显示当前帮助信息

示例:

- 执行 `qff trader connect` 进行连接同花顺客户端测试

```
$ qff trader connect
qff>> ----- - INFO - 同花顺交易接口: 下单程序未启动, 正在自动加载...
qff>> ----- - INFO - 同花顺交易接口: 连接下单程序成功!
```

- 执行 `qff trader balance` 获取账户资金股票信息

```
$ qff trader balance
qff>> ----- - INFO - 同花顺交易接口: 连接下单程序成功!

+-----+
|      账户资金股票      |
+-----+-----+
| 资金余额 | 358.64 |
| 冻结金额 | 13.78  |
| 可用金额 | 372.42 |
| 可取金额 | 358.64 |
| 股票市值 | 123456.0 |
| 总资产  | 123456.0 |
| 持仓盈亏 | 10000.0 |
+-----+-----+
```

9.3 实盘交易函数

QFF 提供了以下函数接口, 用于在用户策略中直接进行实盘交易。目前 QFF 还未提供实盘运行框架, 您可以在模拟运行的策略中, 直接调用实盘交易函数, 以实现实盘交易功能。

<code>trader_connect</code>	连接交易软件客户端
<code>trader_balance</code>	查询当前账户资金股票信息
<code>trader_position</code>	查询当前账户股票持仓信息
<code>trader_today_entrusts</code>	查询当日下单委托记录
<code>trader_today_deal</code>	查询当日成交记录
<code>trader_order</code>	实盘下单交易股票
<code>trader_cancel</code>	撤销当日委托

9.4 trace 交互接口

QFF 人机交互功能接口 `trace` 中增加了实盘操作接口命令，运行 `trader` 命令可获取使用帮助，其用法和命令行接口一致。用户也可在策略模拟运行中，通过 `trace` 命令手工进行股票实盘交易。

QFF> trader

Usage:

以下命令仅支持 windows 系统下运行

```

-----
命令格式: trader connect           : 连接交易软件客户端
命令格式: trader balance           : 获取账户资金股票信息
命令格式: trader position          : 获取账户持仓股票列表
命令格式: trader entrust           : 获取当日委托订单列表
命令格式: trader deal              : 获取当日成交订单列表
命令格式: trader buy stock amount price : 买入指定股票, eg: qff
trader buy 000001 100 10.5
命令格式: trader sell stock amount price : 卖出指定股票. eg: qff
trader sell 000001 100 10.9
命令格式: trader buy stock amount      : 市价买入指定股票, eg: qff
trader buy 000001 100
命令格式: trader sell stock amount      : 市价卖出指定股票. eg: qff
trader sell 000001 100
命令格式: trader cancel entrust_no      : 撤销委托订单, entrust_no
为委托订单编号, 如果不输入, 则撤销所有委托订单
-----

```

示例: 执行 `trader balance` 获取实盘账户资金股票信息

```
QFF> trader balance
```

```
+-----+
|      账户资金股票      |
+-----+-----+
| 资金余额 |   372.42   |
| 冻结金额 | 296015.89  |
| 可用金额 | 296388.31  |
| 可取金额 |   372.42   |
| 股票市值 | 1095553.0   |
| 总资产   | 1391941.31  |
| 持仓盈亏 | 3928391.25  |
+-----+-----+
```

9.5 实盘交易注意事项

9.5.1 实盘与模拟交易的区别

- 市价单冻结资金规则上不同：实盘按照“涨停价 + 千分之五手续费”计算可买数量，并冻结资金，这个可能引起实盘下单量偏小；模拟交易的市价单也没有这个规则；
- 手续费不同：模拟交易可以自己设置手续费，实盘下单时没有计算手续费；
- 滑点的影响：模拟交易可以自己设置滑点，实盘成交结果以交易所成交结果为准；
- 对手影响：实盘的话得有对手方才会成交，如果下市价单会减少部分的影响，但是也有缺点就是成交价格会劣于限价单；
- 下单数量的影响：实盘一次下单数量太大可能会引起市场冲击，模拟交易不会引起市场冲击；
- 权限的影响：模拟交易没有这个限制，实盘时需要开通一定的权限才能交易，例如股票的市价单、创业板，期货一些品种也需要单独开通（例如股指期货）；

9.5.2 股票实盘下单失败的可能原因

- 同花顺客户端版本错误或未成功登录下单软件；
- 没有开通市价单，但使用市价单下单；
- 购买了创业板或科创板标的，但是没有开通相应权限；
- 资金不足；
- 涨停时购买股票，或者跌停时卖出-> 委托废单；
- 五档成交，剩余部分被撤单-> 内部取消；
- 集合竞价不能进行市价委托-> 委托废单；
- 在非交易时间下单；
- 下单数量太大了：股票、基金交易单笔申报最大数量应当不超过 100 万股（份）

在下面我们列举一些常用的算法范例，您可以通过 QFF 运行，方便您快速学习和掌握 QFF 框架。

10.1 双均线策略

金叉死叉策略其实就是双均线策略。策略思想是：当短期均线上穿长期均线时，形成金叉，此时买入股票。当短期均线下穿长期均线时，形成死叉，此时卖出股票。研究表明，双均线系统虽然简单，但只要严格执行，也能长期盈利。

```
1
2 # 双均线策略，当五日均线位于十日均线上方则买入，反之卖出。
3 from qff import *
4
5
6 # 初始化函数，设定要操作的股票、基准等等
7 def initialize(context):
8     # 定义一个全局变量，保存要操作的股票
9     # 000002(股票：万科 A)
10    g.security = '000002'
11    # 设定沪深 300 作为基准
12    set_benchmark('000300')
13    # 运行函数
14    run_daily(trade, 'every_bar')
15
16
17 # 交易程序
18 def trade(context, data):
```

(continues on next page)

(continued from previous page)

```

19     security = g.security
20     # 设定均线窗口长度
21     n1 = 5
22     n2 = 10
23     # 获取股票的收盘价
24     close_data = attribute_history(security, n2+2, '1d', ['close'])
25     # 取得过去 ma_n1 天的平均价格
26     ma_n1 = close_data['close'][-n1:].mean()
27     # 取得过去 ma_n2 天的平均价格
28     ma_n2 = close_data['close'][-n2:].mean()
29     # 取得当前的现金
30     cash = context.portfolio.available_cash
31
32     # 如果当前有余额, 并且 n1 日均线大于 n2 日均线
33     if ma_n1 > ma_n2:
34         # 用所有 cash 买入股票
35         order_value(security, cash)
36         # 记录这次买入
37         log.info("Buying %s" % security)
38
39     # 如果 n1 日均线小于 n2 日均线, 并且目前有头寸
40     elif ma_n1 < ma_n2 and\
41         security in context.portfolio.positions.keys() and\
42         context.portfolio.positions[security].closeable_amount > 0:
43         # 全部卖出
44         order_target(security, 0)
45         # 记录这次卖出
46         log.info("Selling %s" % security)
47
48
49 if __name__ == '__main__':
50     run_file(__file__)
51
52
53

```

10.2 MACD 策略

以下是一个我们使用 TALib 编写的单股票 MACD 算法示例。

1. **macd** 是长短均线的差值，**signal** 是 **macd** 的均线，使用 **macd** 策略有几种不同的方法，我们这里采用 **macd** 线突破 **signal** 线的判断方法。
2. **talib** 是 **python** 的技术指标库，其中包含了很多 150 多种量化指标，所以 **talib** 是非常值得我们学习和使用的。**talib** 使用 C 语言实现，执行速度非常快，其安装方法也比较特殊，请自行搜索安装方法。

```

1  from qff import *
2  import talib as tl
3  import numpy as np
4
5
6  def initialize(context):
7
8      # 设置指数基准
9      set_benchmark(security="000300")
10     # 定义一个全局变量，保存要操作的股票
11     g.s1 = "000001"
12     # 定义运行函数，每日 9 点 50 分运行
13     run_daily(market_open, run_time='09:50')
14
15     log.info("initialize : 初始化运行")
16
17
18  def market_open(context):
19     log.info("market_open 函数，每天运行一次...")
20     # 读取历史数据，前 100 天的收盘价
21     close = history(100, '1d', 'close', g.s1).values
22     # 获取当前价格
23     current_price = get_current_data(g.s1).last_price
24     # 将当前价格与历史价格合并
25     close = np.append(close, current_price)
26
27     # 用 Talib 计算 MACD 取值，得到三个时间序列数组，
28     macd, signal, hist = tl.MACD(close, 12, 26, 9)
29
30     # 如果 macd 从上往下跌破 macd_signal

```

(continues on next page)

(continued from previous page)

```

31     if macd[-1] < signal[-1] and macd[-2] > signal[-2]:
32         # 进行清仓
33         if g.s1 in context.portfolio.positions.keys():
34             order_target(g.s1, 0)
35
36         # 如果短均线从下往上突破长均线，为入场信号
37         if macd[-1] > signal[-1] and macd[-2] < signal[-2]:
38             # 满仓买入
39             order_value(g.s1, context.portfolio.available_cash)
40
41
42 if __name__ == '__main__':
43     run_file(__file__, start="2021-08-27", end="2022-03-25")
44
45

```

10.3 小市值策略

筛选出市值介于 20-30 亿的股票，选取其中市值最小的三只股票，每天开盘买入，持有五个交易日，然后调仓。等权重买入，无单只股票仓位上限控制、无止盈止损。小市值策略曾经在 15 年期间有非常好的收益，未来有可能还能重现。

```

1  from qff import *
2
3
4  # 初始化函数，设定要操作的股票、基准等等
5  def initialize(context):
6      # 设定沪深 300 作为基准
7      set_benchmark('000300')
8      # 持仓数量
9      g.stock_num = 3
10     # 交易日计时器
11     g.days = 0
12     # 调仓频率
13     g.refresh_rate = 5
14
15

```

(continues on next page)

(continued from previous page)

```

16 def before_trading_start(context):
17     log.info("before_trading_start 函数运行...")
18
19
20 def check_stocks(context):
21     # 选出小市值股票
22
23     filter = {'date': context.previous_date, 'market_cap': {'$gt': 20, '$lt': 30}}
24     projection = {'market_cap': 1}
25     df = query_valuation(filter, projection)
26     df = df.sort_values('market_cap').reset_index()
27     buy_list = list(df['code'])[:g.stock_num*2]
28
29     # 过滤停牌股票
30     paused_code = get_paused_stock(buy_list, context.previous_date)
31     filter_paused = [x for x in buy_list if x not in paused_code]
32
33     return filter_paused[:g.stock_num]
34
35
36 # 交易函数
37 def handle_data(context, data):
38     if g.days % g.refresh_rate == 0:
39
40         # 获取持仓列表
41         sell_list = list(context.portfolio.positions.keys())
42         # 如果有持仓, 则卖出
43         if len(sell_list) > 0:
44             for stock in sell_list:
45                 order_target_value(stock, 0)
46
47         # 分配资金
48         if len(context.portfolio.positions) < g.stock_num:
49             Num = g.stock_num - len(context.portfolio.positions)
50             Cash = context.portfolio.available_cash / Num
51         else:
52             Cash = 0
53
54         # 选股

```

(continues on next page)

(continued from previous page)

```

55     stock_list = check_stocks(context)
56
57     # 买入股票
58     for stock in stock_list:
59         if len(context.portfolio.positions.keys()) < g.stock_num:
60             order_value(stock, Cash)
61
62     # 天计数加一
63     g.days = 1
64     else:
65         g.days += 1
66
67
68 if __name__ == '__main__':
69     run_file(__file__, start="2021-08-27", end="2022-03-25")
70

```

10.4 海龟策略

海龟交易系统是非常经典的一种策略，类似的成熟策略体系还有很多种，例如羊驼，鳄鱼等等。关于海龟策略的原理介绍可以参照 [这篇帖子](#)。

```

1
2 # 海归策略
3 # 2012-01-01 到 2016-03-10, ¥1000000, 分钟
4
5 # 海龟策略
6
7 from qff import *
8 import numpy as np
9
10 # =====
11 # 总体回测前
12 # =====
13
14
15 # 总体回测前要做的事情
16 def initialize(context):

```

(continues on next page)

(continued from previous page)

```

17     set_params()  # 1 设置策略参数
18     set_variables()  # 2 设置中间变量
19     set_backtest()  # 3 设置回测条件
20
21
22 # 1
23 # 设置策略参数
24 def set_params():
25     g.security = '000001'
26     # 系统 1 入市的 trailing date
27     g.short_in_date = 20
28     # 系统 2 入市的 trailing date
29     g.long_in_date = 55
30     # 系统 1 exiting market trailing date
31     g.short_out_date = 10
32     # 系统 2 exiting market trailing date
33     g.long_out_date = 20
34     # g.dollars_per_share 是标的股票每波动一个最小单位, 1 手股票的总价格变化量。
35     # 在国内最小变化量是 0.01 元, 所以就是  $0.01 \times 100 = 1$ 
36     g.dollars_per_share = 1
37     # 可承受的最大损失率
38     g.loss = 0.1
39     # 若超过最大损失率, 则调整率为:
40     g.adjust = 0.8
41     # 计算 N 值的天数
42     g.number_days = 20
43     # 最大允许单元
44     g.unit_limit = 4
45     # 系统 1 所配金额占总金额比例
46     g.ratio = 0.8
47
48
49 # 2
50 # 设置中间变量
51 def set_variables():
52     # 初始单元
53     g.unit = 1000
54     # A list storing info of N
55     g.N = []

```

(continues on next page)

(continued from previous page)

```

56     # Record the number of days for this trading system
57     g.days = 0
58     # 系统 1 的突破价格
59     g.break_price1 = 0
60     # 系统 2 的突破价格
61     g.break_price2 = 0
62     # 系统 1 建的仓数
63     g.sys1 = 0
64     # 系统 2 建的仓数
65     g.sys2 = 0
66     # 系统 1 执行且系统 2 不执行
67     g.system1 = True
68
69
70 # 3
71 # 设置回测条件
72 def set_backtest():
73     # 作为判断策略好坏和一系列风险值计算的基准
74     set_benchmark(g.security)
75     log.set_level('info') # 设置报错等级
76
77
78 '''
79 =====
80 每天开盘前
81 =====
82 '''
83
84
85 # 每天开盘前要做的事情
86 def before_trading_start(context):
87     set_slip_fee(context)
88
89
90 # 4 根据不同的时间段设置滑点与手续费
91 def set_slip_fee(context):
92     # 将滑点设置为 0
93     set_slippage(0)
94     # 根据不同的时间段设置手续费

```

(continues on next page)

(continued from previous page)

```

95     dt = context.current_dt
96
97     if dt > '2013-01-01':
98         set_order_cost(open_commission=0.0003, close_commission=0.0013, min_
↪ commission=5)
99     elif dt > '2011-01-01':
100         set_order_cost(open_commission=0.001, close_commission=0.002, min_
↪ commission=5)
101     elif dt > '2009-01-01':
102         set_order_cost(open_commission=0.002, close_commission=0.003, min_
↪ commission=5)
103     else:
104         set_order_cost(open_commission=0.003, close_commission=0.004, min_
↪ commission=5)
105
106 # =====
107 # 每天交易时
108 # =====
109
110
111 # 按分钟回测
112 def handle_data(context, data):
113     dt = context.current_dt # 当前日期
114     if dt[:10] == '2020-04-20':
115         log.info(dt)
116     data = get_current_data(g.security)
117     current_price = data.last_price # 当前价格 N
118     if dt[11:15] == '09:30':
119         g.days += 1
120         calculate_N() # 计算 N 的值
121     if g.days > g.number_days:
122         # 当前持有的股票和现金的总价值
123         value = context.portfolio.total_assets
124         # 可花费的现金
125         cash = context.portfolio.available_cash
126         if g.sys1 == 0 and g.sys2 == 0:
127             # 若损失率大于 g.loss, 则调整 (减小) 可持有现金和总价值
128             if value < (1 - g.loss) * context.portfolio.starting_cash:
129                 cash *= g.adjust

```

(continues on next page)

(continued from previous page)

```

130         value *= g.adjust
131
132     # 计算美元波动的价格
133     dollar_volatility = g.dollars_per_share * g.N[-1]
134     # 依本策略, 计算买卖的单位
135     g.unit = value * 0.01 / dollar_volatility
136
137     # 系统 1 的操作
138     g.system1 = True
139     if g.sys1 == 0:
140         market_in(current_price, g.ratio * cash, g.short_in_date)
141     else:
142         stop_loss(current_price)
143         market_add(current_price, g.ratio * cash, g.short_in_date)
144         market_out(current_price, g.short_out_date)
145
146     # 系统 2 的操作
147     g.system1 = False
148     if g.sys2 == 0:
149         market_in(current_price, (1 - g.ratio) * cash, g.long_in_date)
150     else:
151         stop_loss(current_price)
152         market_add(current_price, (1 - g.ratio) * cash, g.long_in_date)
153         market_out(current_price, g.long_out_date)
154
155     # 5
156
157
158 # 计算当前 N 的值
159 # 输入: none
160 # 输出: N 的值的更新列表-list 类型
161 def calculate_N():
162     # 如果交易天数小于等于 20 天
163     if g.days <= g.number_days:
164         price = attribute_history(g.security, g.days+1, '1d', ['high', 'low', 'close
165         ↪'])
166         price['pre_close'] = price['close'].shift(1)
167         lst = []
168         for i in range(0, g.days):

```

(continues on next page)

(continued from previous page)

```

168     h_l = price['high'][i] - price['low'][i]
169     h_c = price['high'][i] - price['pre_close'][i]
170     c_l = price['pre_close'][i] - price['low'][i]
171     # 计算 True Range
172     True_Range = max(h_l, h_c, c_l)
173     lst.append(True_Range)
174     # 计算前 g.days (小于等于 20) 天的 True_Range 平均值, 即当前 N 的值:
175     current_N = np.mean(np.array(lst))
176     g.N.append(current_N)
177
178     # 如果交易天数超过 20 天
179     else:
180         price = attribute_history(g.security, 2, '1d', ['high', 'low', 'close'])
181         price['pre_close'] = price['close'].shift(1)
182         h_l = price['high'][-1] - price['low'][-1]
183         h_c = price['high'][-1] - price['pre_close'][-1]
184         c_l = price['pre_close'][-1] - price['low'][-1]
185         # Calculate the True Range
186         True_Range = max(h_l, h_c, c_l)
187         # 计算前 g.number_days (大于 20) 天的 True_Range 平均值, 即当前 N 的值:
188         current_N = (True_Range + (g.number_days - 1) * g.N[-1]) / g.number_days
189         g.N.append(current_N)
190
191
192 # 6
193 # 入市: 决定系统 1、系统 2 是否应该入市, 更新系统 1 和系统 2 的突破价格
194 # 海龟将所有资金分为 2 部分: 一部分资金按系统 1 执行, 一部分资金按系统 2 执行
195 # 输入: 当前价格-float, 现金-float, 天数-int
196 # 输出: none
197 def market_in(current_price, cash, in_date):
198     # Get the price for the past "in_date" days
199     price = attribute_history(g.security, in_date, '1d', ['close'])
200     # Build position if current price is higher than highest in past
201     if current_price > max(price['close']):
202         # 计算可以买该股票的股数
203         num_of_shares = cash / current_price
204         if num_of_shares >= g.unit:
205             print("买入")
206             print(current_price)

```

(continues on next page)

(continued from previous page)

```

207     print(max(price['close']))
208     if g.system1:
209         if g.sys1 < int(g.unit_limit * g.unit):
210             order(g.security, int(g.unit))
211             g.sys1 += int(g.unit)
212             g.break_price1 = current_price
213         else:
214             if g.sys2 < int(g.unit_limit * g.unit):
215                 order(g.security, int(g.unit))
216                 g.sys2 += int(g.unit)
217                 g.break_price2 = current_price
218
219
220 # 7
221 # 加仓函数
222 # 输入: 当前价格-float, 现金-float, 天数-int
223 # 输出: none
224 def market_add(current_price, cash, in_date):
225     if g.system1:
226         break_price = g.break_price1
227     else:
228         break_price = g.break_price2
229     # 每上涨 0.5N, 加仓一个单元
230     if current_price >= break_price + 0.5 * g.N[-1]:
231         num_of_shares = cash / current_price
232         # 加仓
233         if num_of_shares >= g.unit:
234             print("加仓")
235             print(g.sys1)
236             print(g.sys2)
237             print(current_price)
238             print(break_price + 0.5 * g.N[-1])
239
240         if g.system1:
241             if g.sys1 < int(g.unit_limit * g.unit):
242                 order(g.security, int(g.unit))
243                 g.sys1 += int(g.unit)
244                 g.break_price1 = current_price
245         else:

```

(continues on next page)

(continued from previous page)

```

246         if g.sys2 < int(g.unit_limit * g.unit):
247             order(g.security, int(g.unit))
248             g.sys2 += int(g.unit)
249             g.break_price2 = current_price
250
251
252 # 8
253 # 离场函数
254 # 输入: 当前价格-float, 天数-int
255 # 输出: none
256 def market_out(current_price, out_date):
257     # Function for leaving the market
258     price = attribute_history(g.security, out_date, '1d', ['close'])
259     # 若当前价格低于前 out_date 天的收盘价的最小值, 则卖掉所有持仓
260     if current_price < min(price['close']):
261         print("离场")
262         print(current_price)
263         print(min(price['close']))
264         if g.system1:
265             if g.sys1 > 0:
266                 order(g.security, -g.sys1)
267                 g.sys1 = 0
268             else:
269                 if g.sys2 > 0:
270                     order(g.security, -g.sys2)
271                     g.sys2 = 0
272
273
274 # 9
275 # 止损函数
276 # 输入: 当前价格-float
277 # 输出: none
278 def stop_loss(current_price):
279     # 损失大于 2N, 卖出股票
280     if g.system1:
281         break_price = g.break_price1
282     else:
283         break_price = g.break_price2
284     # If the price has decreased by 2N, then clear all position

```

(continues on next page)

(continued from previous page)

```
285     if current_price < (break_price - 2 * g.N[-1]):
286         print("止损")
287         print(current_price)
288         print(break_price - 2 * g.N[-1])
289         if g.system1:
290             order(g.security, -g.sys1)
291             g.sys1 = 0
292         else:
293             order(g.security, -g.sys2)
294             g.sys2 = 0
295
296
297 # =====
298 # 每天收盘后
299 # =====
300
301 # 每日收盘后要做的的事情（本策略中不需要）
302 def after_trading_end(context):
303     return
304
305
```

11.1 市值数据

每日更新

通过函数 `get_valuation()` 和 `query_valuation()` 获取

数据格式如下：

字段名	含义
code	股票代码
date	日期
quantity_ratio	量比
capitalization	总股本 (万股)
circulating_cap	流通股本 (万股)
market_cap	总市值 (亿元)
cir_market_cap	流通市值 (亿元)
turnover_ratio	换手率 (%)
pe_ttm	市盈率 (PE, TTM)
pe_lyr	市盈率 (PE)s
pe_dyn	市盈率 (动态)
pb_ratio	市净率 (PB)

11.2 融资融券数据

每日更新

通过函数`get_mtss()` 获取

数据格式如下：

字段名	含义
date	日期
sec_code	股票代码
fin_value	融资余额（元）
fin_buy_value	融资买入额（元）
fin_refund_value	融资偿还额（元）
sec_value	融券余量（股）
sec_sell_value	融券卖出量（股）
sec_refund_value	融券偿还股（股）

11.3 年报 (季报) 数据

财务报表数据按季度更新

财报数据通过财务 [API](#)函数进行查询，为股票基本面分析提供数据支持。

11.3.1 1. 每股指标

字段名	含义
f001	基本每股收益
f002	扣非经常性损益每股收益
f003	每股未分配利润
f004	每股净资产’
f005	每股资本公积金
f006	净资产收益率
f007	每股经营现金流量

11.3.2 2. 资产负债表 BALANCE SHEET

2.1 资产

2.1.1 流动资产

字段名	含义
f008	货币资金
f009	交易性金融资产
f010	应收票据
f011	应收账款
f012	预付款项
f013	其他应收款
f014	应收关联公司款
f015	应收利息
f016	应收股利
f017	存货
f018	其中：消耗性生物资产
f019	一年内到期的非流动资产
f020	其他流动资产
f021	流动资产合计

2.1.2 非流动资产

字段名	含义
f022	可供出售金融资产
f023	持有至到期投资
f024	长期应收款
f025	长期股权投资
f026	投资性房地产
f027	固定资产
f028	在建工程
f029	工程物资
f030	固定资产清理
f031	生产性生物资产
f032	油气资产
f033	无形资产
f034	开发支出
f035	商誉
f036	长期待摊费用
f037	递延所得税资产
f038	其他非流动资产
f039	非流动资产合计
f040	资产总计

2.2 负债

2.2.1 流动负债

字段名	含义
f041	短期借款
f042	交易性金融负债
f043	应付票据
f044	应付账款
f045	预收款项
f046	应付职工薪酬
f047	应交税费
f048	应付利息
f049	应付股利
f050	其他应付款
f051	应付关联公司款
f052	一年内到期的非流动负债
f053	其他流动负债
f054	流动负债合计

2.2.2 非流动负债

字段名	含义
f055	长期借款
f056	应付债券
f057	长期应付款
f058	专项应付款
f059	预计负债
f060	递延所得税负债
f061	其他非流动负债
f062	非流动负债合计
f063	负债合计

2.3 所有者权益

字段名	含义
f064	实收资本（或股本）
f065	资本公积
f066	盈余公积
f067	减：库存股
f068	未分配利润
f069	少数股东权益
f070	外币报表折算价差
f071	非正常经营项目收益调整
f072	所有者权益（或股东权益）合计
f073	负债和所有者（或股东权益）合计

11.3.3 3. 利润表

字段名	含义
f074	其中：营业收入
f075	其中：营业成本
f076	营业税金及附加
f077	销售费用
f078	管理费用
f079	勘探费用
f080	财务费用
f081	资产减值损失
f082	加：公允价值变动净收益
f083	投资收益
f084	其中：对联营企业和合营企业的投资收益
f085	影响营业利润的其他科目
f086	三、营业利润
f087	加：补贴收入
f088	营业外收入
f089	减：营业外支出
f090	其中：非流动资产处置净损失
f091	加：影响利润总额的其他科目
f092	四、利润总额
f093	加：影响净利润的其他科目
f095	减：所得税
f094	五、净利润
f096	归属于母公司所有者的净利润
f097	少数股东损益

11.3.4 4. 现金流量表

字段名	含义
.	一、经营活动。
f098	销售商品、提供劳务收到的现金
f099	收到的税费返还

continues on next page

表 1 – continued from previous page

字段名	含义
f100	收到其他与经营活动有关的现金
f101	经营活动现金流入小计
f102	购买商品、接受劳务支付的现金
f103	支付给职工以及为职工支付的现金
f104	支付的各项税费
f105	支付其他与经营活动有关的现金
f106	经营活动现金流出小计
f107	经营活动产生的现金流量净额
.	二、投资活动。
f108	收回投资收到的现金
f109	取得投资收益收到的现金
f110	处置固定资产、无形资产和其他长期资产收回的现金净额
f111	处置子公司及其他营业单位收到的现金净额
f112	收到其他与投资活动有关的现金
f113	投资活动现金流入小计
f114	购建固定资产、无形资产和其他长期资产支付的现金
f115	投资支付的现金
f116	取得子公司及其他营业单位支付的现金净额
f117	支付其他与投资活动有关的现金
f118	投资活动现金流出小计
f119	投资活动产生的现金流量净额
.	三、筹资活动
f120	吸收投资收到的现金
f121	取得借款收到的现金
f122	收到其他与筹资活动有关的现金
f123	筹资活动现金流入小计
f124	偿还债务支付的现金
f125	分配股利、利润或偿付利息支付的现金
f126	支付其他与筹资活动有关的现金
f127	筹资活动现金流出小计
f128	筹资活动产生的现金流量净额
f129	四、汇率变动对现金的影响
f130	其他原因对现金的影响
f131	五、现金及现金等价物净增加额
f132	期初现金及现金等价物余额

continues on next page

表 1 – continued from previous page

字段名	含义
f133	期末现金及现金等价物余额
.	六、现金流量附表项目
.	六（1）将净利润调节为经营活动现金流量
f134	净利润
f135	资产减值准备
f136	固定资产折旧、油气资产折耗、生产性生物资产折旧
f137	无形资产摊销
f138	长期待摊费用摊销
f139	处置固定资产、无形资产和其他长期资产的损失
f140	固定资产报废损失
f141	公允价值变动损失
f142	财务费用
f143	投资损失
f144	递延所得税资产减少
f145	递延所得税负债增加
f146	存货的减少
f147	经营性应收项目的减少
f148	经营性应付项目的增加
f149	其他
f150	经营活动产生的现金流量净额 2
.	六（2）不涉及现金收支的投资和筹资活动
f151	债务转为资本
f152	一年内到期的可转换公司债券
f153	融资租入固定资产
.	六（3）现金及现金等价物净增加情况
f154	现金的期末余额
f155	现金的期初余额
f156	现金等价物的期末余额
f157	现金等价物的期初余额
f158	现金及现金等价物净增加额

11.3.5 5. 偿债能力分析

字段名	含义
f159	流动比率
f160	速动比率
f161	现金比率 (%)
f162	利息保障倍数
f163	非流动负债比率 (%)
f164	流动负债比率 (%)
f165	现金到期债务比率 (%)
f166	有形资产净值债务率 (%)
f167	权益乘数 (%)
f168	股东的权益/负债合计 (%)
f169	有形资产/负债合计 (%)
f170	经营活动产生的现金流量净额/负债合计 (%)
f171	EBITDA/负债合计 (%)

11.3.6 6. 经营效率分析

字段名	含义
f172	应收帐款周转率
f173	存货周转率
f174	运营资金周转率
f175	总资产周转率
f176	固定资产周转率
f177	应收帐款周转天数
f178	存货周转天数
f179	流动资产周转率
f180	流动资产周转天数
f181	总资产周转天数
f182	股东权益周转率

存货周转率 =(存货周转天数 + 应收帐款周转天数-应付帐款周转天数 + 预付帐款周转天数-预收帐款周转天数)/365

11.3.7 7. 发展能力分析

字段名	含义
f183	营业收入增长率 (%)
f184	净利润增长率 (%)
f185	净资产增长率 (%)
f186	固定资产增长率 (%)
f187	总资产增长率 (%)
f188	投资收益增长率 (%)
f189	营业利润增长率 (%)

11.3.8 8. 获利能力分析

字段名	含义
f193	成本费用利润率 (%)
f194	营业利润率
f195	营业税金率
f196	营业成本率
f197	净资产收益率
f198	投资收益率
f199	销售净利率 (%)
f200	总资产报酬率
f201	净利润率
f202	销售毛利率 (%)
f203	三费比重
f204	管理费用率
f205	财务费用率
f206	扣除非经常性损益后的净利润
f207	息税前利润 (EBIT)
f208	息税折旧摊销前利润 (EBITDA)
f209	EBITDA/营业总收入

11.3.9 9. 资本结构分析

字段名	含义
f210	资产负债率 (%)
f211	流动资产比率
f212	货币资金比率
f213	存货比率
f214	固定资产比率
f215	负债结构比
f216	归属于母公司股东权益/全部投入资本 (%)
f217	股东的权益/带息债务 (%)
f218	有形资产/净债务 (%)

11.3.10 10. 现金流量分析

字段名	含义
f219	每股经营性现金流 (元)
f220	营业收入现金含量 (%)
f221	经营活动产生的现金流量净额/经营活动净收益 (%)
f222	销售商品提供劳务收到的现金/营业收入 (%)
f223	经营活动产生的现金流量净额/营业收入
f224	资本支出/折旧和摊销
f225	每股现金流量净额 (元)
f226	经营净现金比率 (短期债务)
f227	经营净现金比率 (全部债务)
f228	经营活动现金净流量与净利润比率
f229	全部资产现金回收率

11.3.11 11. 单季度财务指标

字段名	含义
f230	营业收入现金及现金等价物净增加额
f231	营业利润现金及现金等价物净增加额
f232	归属于母公司所有者的净利润
f233	扣除非经常性损益后的净利润
f234	经营活动产生的现金流量净额
f235	投资活动产生的现金流量净额
f236	筹资活动产生的现金流量净额
f237	现金及现金等价物净增加额

11.3.12 12. 股本股东

字段名	含义
f238	总股本
f239	已上市流通 A 股
f240	已上市流通 B 股
f241	已上市流通 H 股
f242	股东人数 (户)
f243	第一大股东的持股数量
f244	十大流通股东持股数量合计 (股)
f245	十大股东持股数量合计 (股)

11.3.13 13. 机构持股

字段名	含义
f246	机构总量 (家)
f247	机构持股总量 (股)
f248	QFII 机构数
f249	QFII 持股量
f250	券商机构数

continues on next page

表 2 – continued from previous page

字段名	含义
f251	券商持股量
f252	保险机构数
f253	保险持股量
f254	基金机构数
f255	基金持股量
f256	社保机构数
f257	社保持股量
f258	私募机构数
f259	私募持股量
f260	财务公司机构数
f261	财务公司持股量
f262	年金机构数
f263	年金持股量
f264	十大流通股东中持有 A 股合计 (股)[注 1]
f265	第一大流通股东持股量 (股)
f266	自由流通股 (股)[注 2]
f267	受限流通 A 股 (股)
f268	一般风险准备 (金融类)
f269	其他综合收益 (利润表)
f270	综合收益总额 (利润表)
f271	归属于母公司股东权益 (资产负债表)
f272	银行机构数 (家)(机构持股)
f273	银行持股量 (股)(机构持股)
f274	一般法人机构数 (家)(机构持股)
f275	一般法人持股量 (股)(机构持股)
f276	近一年净利润 (元)
f277	信托机构数 (家)(机构持股)
f278	信托持股量 (股)(机构持股)
f279	特殊法人机构数 (家)(机构持股)
f280	特殊法人持股量 (股)(机构持股)
f281	加权净资产收益率 (每股指标)
f282	扣非每股收益 (单季度财务指标)
f283	最近一年营业收入 ()
f284	国家队持股数量 (万股)[注 3]

[注 1]：季度报告中，若股东同时持有非流通 A 股性质的股份（如同时持有流通 A 股和流通 B 股），取的是包含同时持有非流通 A 股性质的流通股数

[注 2]：

1. 自由流通股 = 已流通 A 股 - 十大流通股东 5% 以上的 A 股；

2. 季度报告中，若股东同时持有非流通 A 股性质的股份（如同时持有流通 A 股和流通 H 股），5% 以上的持股取的是不包含同时持有非流通 A 股性质的流通股数，结果可能偏大；

3. 指标按报告期展示，新股在上市日的下个报告期才有数据

[注 3]：本指标统计包含汇金公司、证金公司、外汇管理局旗下投资平台、国家队基金、国开、养老金以及中科汇通等国家队机构持股数量

11.3.14 14. 预告快报数据

字段名	含义
f285	业绩预告-本期净利润同比增幅下限%
f286	业绩预告-本期净利润同比增幅上限%
f287	业绩快报-归母净利润
f288	业绩快报-扣非净利润
f289	业绩快报-总资产
f290	业绩快报-净资产
f291	业绩快报-每股收益
f292	业绩快报-摊薄净资产收益率
f293	业绩快报-加权净资产收益率
f294	业绩快报-每股净资产
f313	业绩预告公告日期
f314	财报公告日期
f315	业绩快报公告日期
f317	业绩预告-本期净利润下限（万）
f318	业绩预告-本期净利润上限（万）

[注：展示未来一个报告期的数据。例，3 月 31 日至 6 月 29 日这段时间内展示的是中报的数据；如果最新的财务报告后面有多个报告期的业绩预告/快报，只能展示最新的财务报告后面的一个报告期的业绩预告/快报]

11.3.15 15. 增补数据

字段名	含义
f295	应付票据及应付账款 (资产负债表)
f296	应收票据及应收账款 (资产负债表)
f297	递延收益 (资产负债表)
f298	其他综合收益 (资产负债表)
f299	其他权益工具 (资产负债表)
f300	其他收益 (利润表)
f301	资产处置收益 (利润表)
f302	持续经营净利润 (利润表)
f303	终止经营净利润 (利润表)
f304	研发费用 (利润表)
f305	其中, # 利息费用 (利润表-财务费用)
f306	其中, # 利息收入 (利润表-财务费用)
f307	近一年经营活动现金流净额
f308	近一年归母净利润
f309	近一年扣非净利润
f310	近一年现金净流量
f311	基本每股收益 (单季度)
f312	营业总收入 (单季度)
f316	近一年投资活动现金流净额
f319	营业总收入 TTM
f320	员工总数 (人)
f321	每股企业自由现金流
f322	每股股东自由现金流

11.3.16 16. 资产负债表新增指标

字段名	含义
f401	专项储备
f402	结算备付金
f403	拆出资金
f404	发放贷款及垫款

continues on next page

表 3 – continued from previous page

字段名	含义
f405	衍生金融资产
f406	应收保费
f407	应收分保账款
f408	应收分保合同准备金
f409	买入返售金融资产
f410	划分为持有待售的资产
f411	发放贷款及垫款
f412	向中央银行借款
f413	吸收存款及同业存放
f414	拆入资金
f415	衍生金融负债
f416	卖出回购金融资产款
f417	应付手续费及佣金
f418	应付分保账款
f419	保险合同准备金
f420	代理买卖证券款
f421	代理承销证券款
f422	划分为持有待售的负债
f423	预计负债
f424	递延收益
f425	其中, # 优先股
f426	永续债非流动负债科目
f427	长期应付职工薪酬
f428	其中, # 优先股
f429	永续债所有者权益科目
f430	债权投资
f431	其他债权投资
f432	其他权益工具投资
f433	其他非流动金融资产
f434	合同负债
f435	合同资产
f436	其他资产
f437	应收款项融资
f438	使用权资产
f439	租赁负债

11.3.17 17. 未分类财务数据

字段名	含义
f501	稀释每股收益
f502	营业总收入
f503	汇兑收益
f504	其中, 归属于母公司综合收益
f505	其中, 归属于少数股东综合收益
f506	利息收入
f507	已赚保费
f508	手续费及佣金收入
f509	利息支出
f510	手续费及佣金支出
f511	退保金
f512	赔付支出净额
f513	提取保险合同准备金净额
f514	保单红利支出
f515	分保费用
f516	其中, # 非流动资产处置利得
f517	信用减值损失
f518	净敞口套期收益
f519	营业总成本
f520	信用减值损失
f521	资产减值损失
f561	其他原因对现金的影响 2
f562	客户存款和同业存放款项净增加额
f563	向中央银行借款净增加额
f564	向其他金融机构拆入资金净增加额
f565	收到原保险合同保费取得的现金
f566	收到再保险业务现金净额
f567	保户储金及投资款净增加额
f568	处置以公允价值计量且其变动计入当期损益的金融资产净增加额
f569	收取利息、手续费及佣金的现金
f570	拆入资金净增加额

continues on next page

表 4 – continued from previous page

字段名	含义
f571	回购业务资金净增加额
f572	客户贷款及垫款净增加额
f573	存放中央银行和同业款项净增加额
f574	支付原保险合同赔付款项的现金
f575	支付利息、手续费及佣金的现金
f576	支付保单红利的现金
f577	其中, 子公司吸收少数股东投资收到的现金
f578	其中, 子公司支付给少数股东的股利利润
f579	投资性房地产的折旧及摊销
f580	信用减值损失

12.1 策略程序接口

Note: 用户编写的策略文件需实现以下部分策略程序接口函数，才能被 QFF 框架识别并正确执行。

12.1.1 initialize - 策略初始化

`initialize(context)`

初始化函数 - 在回测和实时模拟交易只会在启动的时候触发一次，用于初始一些全局变量。作为策略文件的入口，本方法必须实现。

在初始化函数中，除了初始化全局变量，用户可以设置基准、交易费率、固定滑点等参数。另外通过调用 `run_daily()`，还可以配置定时运行的策略函数，用以替代其他的框架函数。

Parameters

`context` (*Context* 对象) - 策略上下文，存放有当前的账户/股票持仓信息

Example

```
def initialize(context):  
  
    # 设置指数基准  
    set_benchmark(security="000300")  
    # 定义一个全局变量，保存要操作的股票  
    g.security = ['000001', '000002']
```

(continues on next page)

(continued from previous page)

```
# 定义运行函数
run_daily(market_open, run_time='09:50')

log.info("initialize : 初始化运行")
```

12.1.2 handle_data - 盘中策略运行函数 (可选)

`handle_data(context, data)`

盘中策略运行函数 - 该函数每个单位时间会调用一次, 如果按天回测, 则每天调用一次, 如果按分钟, 则每分钟调用一次。如果在 `initialize()` 函数中 `run_daily` 设置了盘中策略函数, 则本函数可以不实现。

该函数运行的时间是股票的交易时间, 即 **9:30 - 15:00**。

该函数在回测中的非交易日是不会触发的 (如回测结束日期为 2021 年 1 月 5 日, 则程序在 2022 年 1 月 1 日-3 日时, `handle_data` 不会运行, 4 日继续运行)。

对于使用日级回测或模拟, 策略内获取到逻辑时间 (`context.current_dt`) 是 9:30。

Parameters

- **context** (*Context* 对象) - 策略上下文, 存放有当前的账户/股票持仓信息
- **data** (*Dict[SecurityUnitData]*) - 一个字典 (dict), key 是股票代码, value 是当时的 *SecurityUnitData* 对象, 存放当前时间的股票数据, 与使用 `get_current_data()` 功能相同。注意: 为了加速, data 里面的数据是按需获取的, 每次 `handle_data` 被调用时, data 是空的 dict, 当你使用 `data[security]` 时该 security 的数据才会被获取。

Example

```
def handle_data(context, data):
    for code in g.security:
        # 获取当前时间点的股票数据
        last_price = data[code].last_price
        average_price = data[code].mavg(20, 'close')
        # 获取账户剩余资金
        cash = context.portfolio.available_cash
        if last_price > average_price:
```

(continues on next page)

(continued from previous page)

```

        # 下单买入指定金额的股票
        order_value(code, cash)
    elif last_price < average_price:
        # 卖出指定股票
        order_target(code, 0)

# 在交易时间段内的策略函数中, 使用 :func:`get_current_data` 来获取当前时间点的数
据, 此数据只在这一个时间点有效, 请不要存起来到下一个 handle_data 再用。

# :class:`.Context` 为全局变量, 在策略函数中可以访问, 但不能对 context 对象属性进
行修改和赋值。

```

12.1.3 before_trading_start - 开盘前运行策略 (可选)

`before_trading_start(context)`

开盘前运行函数 - 该函数会在每天开始交易前被调用一次, 您可以在这里添加一些每天都要初始化的东西。比如根据历史量价数据或基本面数据, 选择符合策略条件的股票加入股票池, 再根据当日开盘后的表现, 进行买入或卖出。需要注意, 不能在这个函数中发送订单。

该函数依据的时间是股票的交易时间, 即该函数启动时间为 **09:00**。

Parameters

`context` (*Context* 对象) - 策略上下文, 存放有当前的账户/股票持仓信息

Example

```

def before_trading_start(context):
    log.info(context.current_dt)

```

12.1.4 after_trading_end - 收盘后运行策略 (可选)

`after_trading_end(context)`

收盘后运行函数 - 该函数会在每天结束交易后被调用一次, 您可以在这里添加一些每天收盘后要执行的内容。这个时候所有未完成的订单已经取消。

该函数依据的时间是股票的交易时间, 即该函数启动时间为 **15:30**。

Parameters

`context` (*Context* 对象) - 策略上下文, 存放有当前的账户/股票持仓信息

Example

```
def after_trading_end(context):  
    log.info(str(context.current_dt))
```

12.1.5 on_strategy_end - 策略运行结束时调用函数 (可选)

`on_strategy_end(context)`

策略运行结束时调用函数, 在回测、模拟交易正常结束时被调用, 失败或取消不会被调用。

Parameters

`context` (*Context* 对象) - 策略上下文, 存放有当前的账户/股票持仓信息

Example

```
def on_strategy_end(context):  
    print('回测结束')
```

12.1.6 process_initialize - 策略恢复启动时运行函数 (可选)

`process_initialize(context)`

每次程序启动时运行函数, 该函数会在每次模拟盘/回测任务恢复运行时执行, 一般用来初始化一些不能持久化保存的内容。恢复运行时, 函数 `initialize` 不再执行。

Parameters

`context` (*Context* 对象) - 策略上下文, 存放有当前的账户/股票持仓信息

Example

```
def process_initialize(context):  
    print('回测重启运行!')
```

12.2 策略执行函数

12.2.1 run_file - 执行指定的策略文件

```
qff.frame.api.run_file(strategy_file, run_type='bt', resume=False, freq='day',
                        cash=1000000, start=None, end=None, name=None,
                        output_dir=None, log_level='info', trace=False)
```

运行策略文件，并初始化环境参数。

Parameters

- **strategy_file** (str) -待运行策略文件路径
- **run_type** (str) -指定策略运行方式。**bt**-回测；**sim**-实盘模拟, 默认值为 **bt**
- **resume** (bool) - 是否执行恢复运行, **True**-恢复以前的策略执行, **False**-重新开始执行策略, 默认 **False**.
- **freq** (str) -策略执行频率, 有效值为 'day', 'min', 'tick', 默认值为 'day'
- **cash** (int) -账户初始资金, 默认值 1000000
- **start** (Optional[str]) -回测开始日期, 默认为结束日期前 60 个交易日
- **end** (Optional[str]) -回测结束日期, 默认为上一个交易日
- **name** (Optional[str]) -策略名称
- **output_dir** (Optional[str]) -指定结果数据输出目录
- **log_level** (str) -控制台日志输出的级别, 有效值为 'debug', 'info', 'warning', 'error', 默认值为 'info'
- **trace** (bool) -策略运行过程中是否进行交互, 模拟交易时自动有效

Returns

None

Example

使用方法：一般在策略文件中的尾部加入以下代码

```
if __name__ == '__main__':
    run_file(__file__, 0, start='2022-06-01', end='2022-08-31')
```

12.3 策略设置函数

12.3.1 run_daily - 定时运行策略设置函数 (可选)

`qff.frame.api.run_daily(func, run_time='every_bar', append=True)`

设置定时运行的策略函数。回测环境/模拟专用 **API**

指定每天要运行的函数, 可以在具体交易日的某一分钟执行。

Parameters

- **func** (Callable) - 一个自定义的函数, 此函数必须接受 **context** 参数。例如自定义函数名 `market_open(context)`。
- **run_time** (str) - 具体执行时间, 一个字符串格式的时间:
 - 交易时间内的任意时间点, 上午 “09:30-11:30”, 下午 “13:01-15:00”, 例如 “10:00”, “14:00”;
 - **every_bar**, 运行时间和您设置的运行频率一致, 按天会在交易日的开盘时调用一次, 按分钟会在交易时间每分钟运行。执行后将替代 `handle_data()` 策略框架函数。
 - **before_open**, 设置 **func** 在开盘前运行, 执行后将替代 `before_trading_start()` 策略框架函数。
 - **after_close**, 设置 **func** 在收盘后运行, 执行后将替代 `after_trading_end()` 策略框架函数。
- **append** (bool) - 如果 **run_time** 已注册运行函数, 新函数是在原函数前面运行还是后面运行。**append=True**: 则新函数是在原函数后面运行。

Note: 一个策略中尽量不要同时使用 `run_daily` 和 `handle_data`, 更不能使用 `run_daily(handle_data, “xx:xx”)` `run_daily` 中的函数只能有一个参数 **context**, 具体示例如下:

Example

```
def initialize(context):
    run_daily(func, run_time='10:00')
```

(continues on next page)

(continued from previous page)

```
def func(context):
    print(context.current_dt)
    print('-'*50)

# 参数 func 必须是一个全局的函数，不能是类的成员函数，并且 func 是可
# 重入函数，即可重复调用。
```

Return type

None

12.3.2 set_benchmark - 设置指数基准

```
qff.frame.api.set_benchmark(security)
```

设置基准

默认我们选定了沪深 300 指数的每日价格作为判断您策略好坏和一系列风险值计算的基准。您也可以使用 `set_benchmark` 指定其他指数

Parameters`security (str)` - 指数基准**Returns**

None

12.3.3 set_order_cost - 设置交易税费

```
qff.frame.api.set_order_cost(open_tax=0, close_tax=0.001,
                             open_commission=0.0002,
                             close_commission=0.0002, min_commission=5)
```

设置佣金和印花税率。

指定每笔交易要收取的手续费，系统会根据用户指定的费率计算每笔交易的手续费

Parameters

- `open_tax (float)` - 买入时印花税，默认值为 0
- `close_tax (float)` - 卖出时印花税，默认值为千分之一
- `open_commission (float)` - 买入时佣金，默认值为万分之二
- `close_commission (float)` - 卖出时佣金，默认值为万分之二

- `min_commission` (float) -最低佣金，不包含印花税，默认值为 5

Return type

None

12.3.4 `set_slippage` - 设置固定滑点

```
qff.frame.api.set_slippage(slippage=0.00246)
```

设置固定滑点

当您下单后, 真实的成交价格与下单时预期的价格总会有一定偏差, 因此我们加入了滑点模式来帮您更好的模拟真实市场的表现. 我们暂时只支持固定滑点。同时, 我们也支持为交易品种和特定的交易标的设置滑点。

Parameters

`slippage` (float) -固定滑点值，默认 0.00246

Return type

None

12.3.5 `set_universe` - 定股票值 (history 函数专用)

```
qff.frame.api.set_universe(security_list)
```

设定股票值 (history 函数专用)

设置或者更新此策略要操作的股票池 `context.universe`. 请注意: 该函数现在只用于设定 **history** 函数的默认 **security_list**, 除此之外并无其他用处。

Parameters

`security_list` (list) -证券标的列表

Return type

None

12.3.6 `pass_today` - 跳过当日 (回测专用)

```
qff.frame.api.pass_today()
```

跳过当日 (回测专用)

在分钟执行策略中, 调用此函数可用跳过当日剩余的每分钟策略运行, 以提高回测效率。

Return type

None

12.4 股票交易函数

12.4.1 order - 按股票数量下单

```
qff.frame.order.order(security, amount=100, price=None, callback=None)
```

按股票数量下单

调用成功后, 您将可以调用 `get_open_orders()` 取得所有未完成的交易, 也可以调用 `order_cancel()` 取消交易

Parameters

- **security** (str) - 标的代码
- **amount** (int) - 交易数量, 正数表示买入, 负数表示卖出
- **price** (Optional[float]) - 下单价格, 下单价格为空, 则认为是市价单, 按当前最新价格挂单, 否则认为是限价单
- **callback** (Optional[Callable]) - 回调函数, 订单成交/取消后调用执行, `callback(status)`

Return type

Optional[str]

Returns

成功返回 order 对象 id, 失败返回 None

订单撮合规则:

1. 为简化操作, 撮合时不考虑成交量, 一个订单一次成交记录
2. 市价单买入时按当前价格 + 滑点价格, 转成限价单。如果当前价格为涨停价格, 则订单取消。
3. 市价单卖出时按当前价格-滑点价格, 转成限价单。如果当前价格为跌停价格, 则订单取消。
4. 如果运行频率为天, 则下单后立即撮合, 读取剩余的分钟数据曲线, 判断最高价是否大于委托价, 是则成交。
5. 如果运行频率为分钟, 则下单后, 每分钟都判断该订单是否符合成交条件。

6. 对未成交的订单，在本交易日结束后撤销。

12.4.2 order_value - 按股票价值下单

```
qff.frame.order.order_value(security, value, price=None, callback=None)
```

按股票价值下单

Parameters

- **security** (str) - 股票代码
- **value** (float) - 股票价值， $\text{value} = \text{最新价} * \text{手数} * \text{乘数}$ （股票为 100）
- **price** (Optional[float]) - 下单价格，市价单可不填价格，按当前最新价格挂单
- **callback** (Optional[Callable]) - 回调函数，订单成交/取消后调用执行，`callback(status)`

Return type

Optional[str]

Returns

Order 对象 id 或者 None, 如果创建委托成功, 则返回 Order 对象 id, 失败则返回 None

Example

```
# 卖出价值为 10000 元的平安银行股票
order_value('000001', -10000)
```

12.4.3 order_target - 按股票目标数量下单

```
qff.frame.order.order_target(security, amount, price=None, callback=None)
```

按股票目标数量下单

使最终标的的数量达到指定的 amount。注意使用此接口下单时若指定的标的有未完成的订单，则先前未完成的订单将会被取消

Parameters

- **security** (str) - 股票代码

- **amount** (int) -期望的标的最终持有的股票数量
- **price** (Optional[float]) -下单价格，市价单可不填价格，按当前最新价格挂单
- **callback** (Optional[Callable]) -回调函数，订单成交/取消后调用执行，callback(status)

Return type

Optional[str]

Returns

Order 对象 id 或者 None, 如果创建委托成功, 则返回 Order 对象 id, 失败则返回 None

12.4.4 order_target_value - 按股票目标价值下单

```
qff.frame.order.order_target_value(security, value, price=None,
                                    callback=None)
```

按股票目标价值下单

调整标的仓位到 value 价值，注意使用此接口下单时若指定的标的有未完成的订单，则先前未完成的订单将会被取消

Parameters

- **security** (str) -股票代码
- **value** (float) -期望的标的最终价值，value = 最新价 * 手数 * 乘数（股票为 100）
- **price** (Optional[float]) -下单价格，市价单可不填价格，按当前最新价格挂单
- **callback** (Optional[Callable]) -回调函数，订单成交/取消后调用执行，callback(status)

Return type

Optional[str]

Returns

Order 对象 id 或者 None, 如果创建委托成功, 则返回 Order 对象 id, 失败则返回 None

Example

```
# 卖出价值为 10000 元的平安银行股票
order_value('000001', -10000)
# 卖出平安银行所有股票
order_target_value('000001', 0)
# 调整平安银行股票仓位到 10000 元价值
order_target_value('000001', 10000)
```

12.4.5 order_cancel - 撤回已下的订单

```
qff.frame.order.order_cancel(order_id)
```

撤回已下的订单

Parameters

`order_id` (str) - 订单编号

Return type

bool

Returns

成功返回 True, 失败返回 False

12.4.6 get_orders - 获取当天的所有订单

```
qff.frame.order.get_orders(order_id=None, security=None, status=None)
```

获取当天的所有订单

Parameters

- `order_id` (Optional[str]) - 订单 id
- `security` (Optional[str]) - 标的代码, 可以用来查询指定标的的所有订单
- `status` (Optional[*ORDER_TYPE*]) - 查询特定订单状态的所有订单

Return type

Dict[str, *Order*]

Returns

返回一个 dict, key 是 `order_id`, value 是 *Order* 对象

12.4.7 get_open_orders - 获得当天的所有未完成的订单

```
qff.frame.order.get_open_orders()
```

获得当天的所有未完成的订单

Returns

返回一个 dict, key 是 order_id, value 是 [Order] 对象

12.5 实盘操作函数

12.5.1 trader_connect - 连接交易软件客户端

```
qff.trader.ths.trader_connect()
```

连接交易软件客户端

调用本函数连接已启动的交易软件客户端，本函数需在其他实盘操作函数前运行。为使 qff 成功连接交易软件，需要提前启动并登录交易软件，对于通用版同花顺下单程序，需要先手动登录一次：添加券商，填入账户号、密码、验证码，勾选“保存密码”，并在 config 文件中配置下单程序的完整路径。

Return type

bool

Returns

成功返回 True，失败返回 False

12.5.2 trader_balance - 获取当前账户资金股票信息

```
qff.trader.ths.trader_balance()
```

查询当前账户资金股票信息

通过操作交易软件客户端，获取当前账户的资金股票信息。

Return type

Optional[dict]

Returns

返回字典类型，包括‘资金余额’，‘冻结金额’，‘可用金额’，‘可取金额’，‘股票是在’，‘总资产’，‘持仓盈亏’等信息。

12.5.3 trader_position - 获取当前账户股票持仓信息

```
qff.trader.ths.trader_position()
```

查询当前账户股票持仓信息

通过操作交易软件客户端，获取当前账户的股票持仓信息。

Return type

Optional[DataFrame]

Returns

返回 DataFrame 类型，字段包括：‘证券代码’，‘证券名称’，‘股票余额’，‘可用余额’，‘冻结数量’，‘成本价’，‘市价’，‘盈亏’，‘盈亏比例 (%)’，‘市值’，‘当日买入’，‘当日卖出’，‘交易市场’，‘持股天数’ 等信息。

12.5.4 trader_today_entrusts - 获取当日委托记录

```
qff.trader.ths.trader_today_entrusts()
```

查询当日下单委托记录

通过操作交易软件客户端，获取当前账户的当日委托记录信息。

Return type

Optional[DataFrame]

Returns

成功返回委托记录，字段包括：‘委托时间’，‘证券代码’，‘证券名称’，‘操作’，‘备注’，‘委托数量’，‘成交数量’，‘委托价格’，‘成交均价’，‘撤单数量’，‘合同编号’，‘交易市场’ 等信息。

12.5.5 trader_today_deal - 获取当日成交记录

```
qff.trader.ths.trader_today_deal()
```

查询当日成交记录

通过操作交易软件客户端，获取当前账户的当日成交记录信息。

Return type

Optional[DataFrame]

Returns

成功返回委托记录，字段包括：' 成交时间' , ' 证券代码' , ' 证券名称' , ' 操作' , ' 成交数量' , ' 成交均价' , ' 成交金额' , ' 合同编号' , ' 成交编号' 等信息。

12.5.6 trader_order - 实盘交易股票

`qff.trader.ths.trader_order(security, amount, price=None)`

实盘下单交易股票

通过操作同花顺下单软件客户端，买入或卖出指定的股票。

Parameters

- **security** (str) -六位股票代码
- **amount** (int) -交易数量, 正数表示买入, 负数表示卖出
- **price** (Optional[float]) -下单价格, 下单价格为空, 则按市价买入或卖出, 市价委托类型, 默认客户端默认选择; 否则认为是限价单。

Return type

Optional[int]

Returns

成功委托返回合同编号，失败则返回 None

12.5.7 trader_cancel - 撤销当日委托

`qff.trader.ths.trader_cancel(entrust_no=None)`

撤销当日委托

通过操作交易软件客户端，撤销当日提交的委托订单。

Parameters

entrust_no (Optional[int]) -委托合同编号, 如果参数为 None, 则撤销当前所有委托

Return type

bool

Returns

成功返回 True, 失败返回 False

12.6 策略分析函数

12.6.1 stats_report - 策略运行结果分析

```
class qff.frame.stats.stats_report(ctx=None, file_name=None)
```

分析策略运行结果，并输出分析报告。QFF 回测运行完成后会自动调用，模拟交易中每天收盘后也会自动调用。

Parameters

- **ctx** (Optional[str]) - 策略运行的上下文环境，默认为当前运行的策略。也可以通过 *pickle.dump* 函数读取以前策略的 *pkl* 文件获取 context.
- **file_name** (Optional[str]) - 策略分析报告输出路径，默认为 *.qff/output/backtest/< 策略名称 > 策略分析报告.html*

Return type

None

Returns

None

12.6.2 strategy_eval - 择时策略评价

```
class qff.frame.evaluation.strategy_eval(get_signal_fun, name=None,
                                         desc=None, hold_gaps=None,
                                         start='2010-01-04',
                                         end='2022-05-06', security=None,
                                         csv=None)
```

策略评价函数，对每个股票的历史曲线数据进行择时，分析策略运行效果，并生成策略评价报告 *word* 文件。

Parameters

- **get_signal_fun** (Callable) - 择时函数，获取买点信号，输入参数为 (code, kline 数据), 返回值 List[date_str]
- **name** (Optional[str]) - 策略名称
- **desc** (Optional[str]) - 策略描述

- **hold_gaps** (Optional[List[str]]) -持仓周期, 评估策略在不同持仓周期下的表现, 默认 [1, 3, 5, 10, 20]
- **start** (str) -评价测试开始日期
- **end** (str) -评价测试结束日期
- **security** (Optional[List[str]]) - 评价测试使用的股票集合, 为 None 表示当前所有上市股票
- **csv** (Optional[str]) -保存 pnl 的数据文件, 如果存在该文件, 则直接使用该文件数据, 否则运行策略函数, 并将运行结果保存至 csv 文件中。

Return type

None

Returns

None

12.7 日期时间函数

12.7.1 get_real_trade_date - 获取真实的交易日期

```
class qff.tools.date.get_real_trade_date(date, towards=-1)
```

根据给定日期, 获取真实的交易日期

Parameters

- **date** -给定日期 [str,date]
- **towards** -方向, -1 -> 向前, 1 -> 向后 int

Returns

返回计算后的日期 str

12.7.2 is_trade_day - 判断是否交易日期

```
class qff.tools.date.is_trade_day(date)
```

判断是否交易日期

Parameters

date (str) - 需判断的日期

Return type

bool

Returns

True: 是交易日期

12.7.3 get_date_gap - 获取向前或向后间隔天数的交易日日期

```
class qff.tools.date.get_date_gap(date, gap, methods)
```

返回指定日期向前或向后间隔天数的交易日日期

Parameters

- **date** (str) - 字符串起始日
- **gap** (int) - 间隔多数个交易日
- **methods** (str) - 方向 [“gt-> 大于”, “gte-> 大于等于”, “小于->lt”, “小于等于->lte”, “等于->===”]

Return type

Optional[str]

Returns

返回计算后的日期

12.7.4 get_next_trade_day - 获取下一个交易日的日期

```
class qff.tools.date.get_next_trade_day(date, n=1)
```

获取下一个交易日的日期

Parameters

- **date** (str) - 字符串起始日
- **n** (int) - 间隔多数个交易日

Return type

str

Returns

返回计算后的日期

12.7.5 get_pre_trade_day - 获取前几个交易周期的日期

```
class qff.tools.date.get_pre_trade_day(date, n=1, freq='day')
```

获取前几个交易周期的日期

Parameters

- `date (str)` - 字符串起始日
- `n (int)` - 间隔多数个交易日
- `freq (str)` - 间隔频率, 支持 ['day' , ' 1min' , ' 5min' , ' 15min' , ' 30min' , ' 60min']

Return type

str

Returns

返回计算后的日期

12.7.6 get_trade_days - 获取指定范围交易日

```
class qff.tools.date.get_trade_days(start, end)
```

获取指定范围交易日

Parameters

- `start (str)` - 开始日期
- `end (str)` - 结束日期

Return type

Optional[list]

Returns

返回交易日期列表

12.7.7 get_trade_gap - 获取两个交易日之间的交易天数

```
class qff.tools.date.get_trade_gap(start, end)
```

返回 start 到 end 中间有多少个交易日，算首尾

Parameters

- start (str) -开始日期
- end (str) -结束日期

Return type

int

Returns

交易日数量

12.7.8 run_time - 计算函数运行时间的装饰函数

```
class qff.tools.date.run_time(func)
```

计算函数运行时间的装饰函数

Parameters

func (Callable) -函数名称，在待计时函数上方输入 @run_time 即可

Return type

float

Returns

函数运行的时间

12.8 其他函数

12.8.1 log - 日志 log 信息

```
class qff.tools.logs.Log
```

分级别输出日志，跟 python 的 logging 模块一致 print 输出的结果等同于 log.info

- log.error(content) 输出错误日志
- log.warn(content) 输出报警日志
- log.info(content) 输出信息日志

- `log.debug(content)` 输出调试日志

```
class qff.tools.logs.Log.set_level(self, level)
```

设置不同种类的 log 的级别, 低于这个级别的 log 不会输出. 所有 log 的默认级别是 info

Parameters

`level` - 字符串, 必须是 'debug', 'info', 'warning', 'error' 中的一个, 级别: `debug < info < warning < error`

Returns

None

12.8.2 kshow - 生成网页版 K 线图

```
class qff.tools.kshow.kshow(df, code, mp=None, index=False)
```

通过 echart 生成 K 线图, 并打开浏览器查看

Parameters

- `df` (DataFrame) - dataframe 股票数据
- `code` (str) - 股票代码
- `mp` (Optional[Dict]) - 标注点, 用于标注买入信号或卖出信号, 格式为

```
{
    "标注名称 1": [日期, 价格],
    "标注名称 2": [日期, 价格],
}
```

- `index` (bool) - 是否指数代码

Return type

Grid

Returns

None

12.8.3 send_message - 发送消息提醒

```
class qff.tools.email.send_message(message)
```

给用户自己发送消息提醒, 暂时只支持邮件通知.

Parameters

message -消息内容. 字符串.

Returns

True/False, 表示是否发送成功. 当发送失败时, 会在日志中显示错误信息.

- 此功能只能在 **实时运行模拟交易**中使用, 回测中使用会直接忽略, 无任何提示;
- 要使用模拟交易发送消息提醒功能, 必须在使用前正确设置邮箱配置参数;
- 参数文件 `~/qff/setting/config.ini` 按以下格式配置参数:

```
[EMAIL]
from_email = your_email@example.com
from_email_password = 授权密码
smtp_server = smtp.example.com
smtp_port = 587
to_email = to_receive_email@example.com
```

其中授权密码可通过访问 <https://www.jiuanweb.com/jz/show-165933770.html> 获取生成方法。

12.9 类和对象

12.9.1 g - 全局变量对象

```
class qff.frame.context.GlobalVar
```

全局对象 **g**, 用来存储用户的各类可被 `pickle.dumps` 函数序列化的全局数据

在模拟盘中, 如果中途进程中断, 我们会使用 `[pickle.dumps]` 序列化所有的 **g** 下面的变量内容, 保存到磁盘中, 再启动的时候模拟盘就不会有任何数据影响。如果没有用 **g** 声明, 会出现模拟盘重启后, 变量数据丢失的问题。

如果不想 **g** 中的某个变量被序列化, 可以让变量以 ‘`_`’ 开头, 这样, 这个变量在序列化时就会被忽略

示例:

```
def initialize(context):
    g.security = "000001"
    g.count = 1
    g.flag = 0

def process_initialize(context):
    # 保存不能被序列化的对象，进程每次重启都初始化，更多信息，请看 \[process\_
    ↪initialize\]
    g.__q = ["000001", "000002"]

def handle_data(context, data):
    log.info(g.security)
    log.info(g.count)
    log.info(g.flag)
```

12.9.2 context - 策略上下文全局对象

```
class qff.frame.context.Context
```

策略上下文

系统全局变量 `context` 保存策略运行信息，包含账户、策略、时间、运行参数等。用户可以直接读取 `context` 相关属性, 但 注意不能直接修改

属性	类型	说明
strategy_name	str	策略名称
run_type	<i>RUN_TYPE</i>	当前框架运行模式，回测还是模拟
status	<i>RUN_STATU</i>	策略当前运行状态
run_freq	str	策略运行频率包括” day” ,” tick” 和 “min”
start_date	str	回测开始日期
end_date	str	回测结束日期
current_dt	str	策略执行的当前时间 “yyyy-mm-dd HH:MM:SS”
previous_date	str	策略执行的当前时间的前一天” yyyy-mm-dd”
benchmark	str	基准指数代码
portfolio	<i>Portfolio</i>	交易账户对象
order_list	Dict	当日的所有订单列表,key 为 order_id, value 为 <i>Order</i>
order_hists	List	历史订单列表, 以 List 保存 <i>Order</i> 对象的 message 属性
positions_hists	List	历史仓位列表, 以 List 保存 <i>Position</i> 对象的 message 属性
asset_hists	List	历史账户资产列表,以 List 保存 <i>Portfolio</i> 对象的 message 属性
strategy_file	str	策略文件名称及路径
log_file	str	日志文件名称及路径
run_start	str	回测开始时间，格式” yyyy-mm-dd HH:MM:SS”
run_end	str	回测运行结束时间（用于计算回测耗时）
output_dir	str	策略运行结果文件输出路径

12.9.3 Portfolio - 股票账户

```
class qff.frame.portfolio.Portfolio(starting_cash)
```

股票账户

账户当前的资金，标的信息，即所有标的操作仓位的信息汇总。

属性	类型	说明
starting_cash	float	初始资金
available_cash	float	可用资金, 用来购买证券的资金
locked_cash	float	冻结资金, 挂单锁住资金
total_assets	float	账户总资产, 包括现金, 仓位 (股票) 的总价值, 用来计算收益
positions_assets	float	持仓资产价值
bench-mark_assets	float	基准市值
income	float	账户累计收益
returns	float	账户收益率
day_income	float	当日盈亏金额
day_returns	float	当日涨幅
positions	Dict	当前仓位, key 值为股票代码, value 是 <i>Position</i> 对象
bench-mark_returns	float	基准收益率

12.9.4 Position - 持仓

```
class qff.frame.position.Position(security, security_name, init_time, amount, avg_cost)
```

持仓标的信息

持有的某个标的的信息

属性	类型	说明
security	str	股票代码
security_name	str	股票名称
init_time	str	建仓时间
avg_cost	float	当前持仓成本，只有在开仓/加仓时会更新
acc_avg_cost	float	累计的持仓成本，在清仓/减仓时也会更新，该持仓累积的收益都会用于计算成本
transact_time	str	最后交易时间
locked_amount	int	挂单冻结仓位
closeable_amou	int	可卖出的仓位，不包括挂单冻结仓位，建仓当天不能卖出
to-day_open_amou	int	今天开仓的仓位
to-day_open_price	float	今日开仓价格
total_amount	int	总仓位, 等于 locked_amount+closeable_amount+today_open_a
latest_price	float	最新行情价格
valuation	float	标的市值，计算方法是: price * total_amount
income	float	浮动盈亏
income_rate	float	浮动盈亏比率
today_income	float	当日盈亏
to-day_income_rat	float	当日盈亏比率
hold_days	int	持仓天数

12.9.5 Order - 订单对象

```
class qff.frame.order.Order(security, amount, price=None,
                             style=ORDER_TYPE.MARKET, callback=None)
```

订单对象

属性	类型	说明
id	str	订单编号
security	str	股票代码
is_buy	boolean	买入还是卖出, True-买入, False-卖出
amount	int	委托数量
status	<i>ORDER_STATUS</i>	订单状态
add_time	str	订单委托时间
deal_time	str	订单成交时间
cancel_time	str	订单取消时间
style	<i>ORDER_TYPE</i>	订单类型, 市价单还是限价单
order_price	float	委托价格, 当订单为限价单时
trade_amount	int	成交数量
trade_money	float	成交金额 (含交易费用)
trade_price	float	成交均价, 等于成交金额除以成交数量, 包含了交易费用分摊
commission	float	交易费用 (佣金、税费等)
gain	float	订单收益股票卖出时计算该值

12.9.6 SecurityUnitData - 当前数据对象

```
class qff.price.cache.SecurityUnitData(code, market='stock')
```

当前时刻标的的数据快照对象

通过 `get_current_data()` 函数获取, 只能在回测或模拟交易中使用

属性	类型	说明
code	str	标的代码
name	str	标的名称
last_price	float	当前股票价格
day_open	float	当日开盘价
high_all_day	float	当日之前时间段最高价
low_all_day	float	当日之前时间段最低价
pre_close	float	昨日收盘价
high_limit	float	当日涨停价
low_limit	float	当日跌停价
last_high	float	当前 Bar 最高价
last_low	float	当前 Bar 最低价
paused	bool	当日是否停牌
min_data_before	DataFrame	今日当前时刻之前的分钟曲线
min_data_frequency	str	当日缓存分钟曲线的频率 ('1min' , '5min' , '15min' , '30min' , '60min')
high_limit_time	int	当天涨停时间长度
block	list	股票所属板块
ticks	dict	当前时刻 Tick 值，实盘专用

12.9.7 Tick - tick 对象

```
class qff.Tick
```

一个 tick 所包含的信息。tick 中的信息是在 tick 事件发生时，盘面的一个快照。

字段名	含义
price	当前价格
last_close	昨日收盘价
open	当日开盘价
high	截至到当前时刻的日内最高价
low	截至到当前时刻的日内最低价
vol	截至到当前时刻的日内总手数
cur_vol	当前 tick 成交笔数
amount	截至到当前时刻的日内总成交额
s_vol	内盘
b_vol	外盘
bid1~bid5	买一到买五价格
ask1~ask5	卖一到卖五价格
bid_vol1~bid_vol5	买一到买五挂单手数
ask_vol1~ask_vol5	卖一到卖五挂单手数

12.10 枚举常量

12.10.1 RUN_TYPE - 策略运行模式

```
class qff.frame.const.RUN_TYPE(value)
```

策略运行模式

BACK_TEST

历史数据回测

SIM_TRADE

实盘模拟交易

12.10.2 RUN_STATUS - 策略运行状态

```
class qff.frame.const.RUN_STATUS(value)
```

策略运行状态

NONE

未开始

RUNNING

正在进行

DONE

完成

FAILED

失败

CANCELED

取消

PAUSED

暂停

12.10.3 ORDER_TYPE - 订单类型

```
class qff.frame.const.ORDER_TYPE(value)
```

订单类型

MARKET

市价单

LIMIT

限价单

12.10.4 ORDER_STATUS - 订单状态

```
class qff.frame.const.ORDER_STATUS(value)
```

订单状态

OPEN

已委托未成交

DEAL

已成交

CANCELLED

已撤单

13.1 行情

13.1.1 `get_price` - 获取历史数据, 可查询多个标的多个数据字段

```
qff.price.query.get_price(security, start=None, end=None, freq='daily',  
                           fields=None, skip_paused=False, fq='pre',  
                           count=None, market='stock')
```

获取历史数据, 可查询一支或者多个标的的行情数据, 按天或者按分钟, 返回数据格式为 `DataFrame`

Parameters

- **`security`** (*str or list*) - 单只标的代码或标的代码列表, 标的为股票、指数、ETF 等, 本参数不能为空.
 - **`count`** (*int*) - 返回的结果集的行数, 即表示获取 `end_date` 之前几个 `freq` 的数据, 与 `start` 二选一, 不可同时使用.
 - **`start`** (*str、datetime.datetime or datetime.date*) - 查询开始时间, 与 `count` 二选一, 不可同时使用.
1. 如果 `count` 和 `start_date` 参数都没有, 则 `start` 等于 `end`.
 2. 如果 `count` 和 `start_date` 参数都设置, 则 `start` 生效, 返回结果会截取最后 `count` 数量的记录.
 3. 当取分钟数据时, 时间可以精确到分钟, 比如: 传入 `datetime.datetime(2015, 1, 1, 10, 0, 0)` 或者 `'2015-01-01 10:00:00'`

4. 当取分钟数据时, 如果只传入日期, 则日内时间是当日的 09:30:00 - 15:00:00

5. 当取天数据时, 传入的日内时间会被忽略

- `end(str, datetime.datetime or datetime.date)` - 查询结束时间, 默认是当前日期, **注意:** 当取分钟数据时, 如果 `end` 只有日期, 则日内时间等同于 15:00:00.
- `freq(str)` - 单位时间长度, 现在支持 ['daily' , '1d' , 'day' , '1min' , '5min' , '15min' , '30min' , '60min' , '1m' , '5m' , '15m' , '30m' , '60m'],
- `fields(str or list)` - 选择要获取的行情数据字段, 默认是 None, 表示 ['open' , 'close' , 'high' , 'low' , 'vol' , 'amount'] 这几个标准字段, 指数另外支持 ['up_count' , 'down_count']。
- `skip_paused(bool)` - 是否跳过不交易日期 (包括停牌, 未上市或者退市后的日期). 如果不跳过, 停牌时会使用停牌前收盘价数据填充。
关于停牌: 因为此 API 可以获取多只股票的数据, 可能有的股票停牌有的没有, 为了保持时间轴的一致, 我们默认没有跳过停牌的日期
- `fq(str or None)` - 复权选项: 'pre' , 前复权; None, 不复权, 返回实际价格; 'post' , 后复权
- `market(str)` - 市场类型, 目前支持 ["stock" , "index" , "ETF"], 默认 "stock" .

Returns

请注意, 为了方便比较一只股票的多个属性, 同时也满足对比多只股票的一个属性的需求, 我们在 `security` 参数是一只股票和多只股票时返回的结构完全不一样.

- 如果是一支股票, 则返回 [pandas.DataFrame] 对象, 行索引是 `date`(分钟级别数据为 `datetime`), 列索引是行情字段名字.
- 如果是多支股票, 则返回 [pandas.DataFrame] 对象, 行索引是 ['date' , 'code'], 或 ['datetime' , 'code']

Return type

DataFrame or None

Example

```

# 获取股票 000001 当天的日数据
df = get_price('000001')

# 获得 000001 的 2020 年 01 月的分钟数据, 只获取 open+close 字段
df = get_price('000001', start='2020-01-01', end='2020-01-31 23:00:00',
    frequency='1m', fields=['open', 'close'])

# 获取获得 000001 在 2020 年 01 月 31 日前 2 个交易日的数据
df = get_price('000001', count = 2, end='2015-01-31', freq='daily', fields=[
    'open', 'close'])

# 获得 000001 的 2020 年 12 月 1 号 14:00-2020 年 12 月 2 日 12:00 的分钟数据
df = get_price('000001', start='2020-12-01 14:00:00', end_date='2020-12-02
    12:00:00', freq='1m')

# 获取多只股票
panel = get_price(get_index_stocks('000903')) # 获取中证 100 的所有成分股的当天
    日数据, 返回一个 [pandas.DataFrame]

```

13.1.2 history - 获取历史数据, 可查询多个标的单个数据字段

```

qff.price.query.history(count, unit='1d', field='close', security_list=None,
    skip_paused=False, fq='pre')

```

回测/模拟专用 API 获取历史数据, 可查询多个股票的单个数据字段, 返回数据格式为 DataFrame 当取天数据时, 不包括当天的, 即使是在收盘后; 分钟数据不包括当前分钟的数据, 没有未来

Parameters

- **count (int)** -数量, 返回的结果集的行数
- **unit (str)** -单位时间长度, 现在支持 ['daily' , '1d' , 'day' , '1min' , '5min' , '15min' , '30min' , '60min' , '1m' , '5m' , '15m' , '30m' , '60m']
- **field (str)** -要获取的数据类型, 只能是一个值, 包含: ['open' , 'close' , 'low' , 'high' , 'vol' , 'amount']
- **security_list (Optional[list])** -要获取数据的股票列表, None 表示查询 context.universe 中所有股票的数据

- **skip_paused** (bool) -是否跳过不交易日期 (包括停牌). 如果不跳过, 停牌时会使用停牌前的数据填充。
- **fq** (str) -复权选项.' pre': 前复权;' post' : 后复权; None: 不复权

Return type

Optional[DataFrame]

Returns

[pandas.DataFrame] 对象, 行索引是 `datetime` 字符串, 列索引是股票代码.

13.1.3 `attribute_history` - 获取历史数据, 可查询单个标的多个数据字段

```
qff.price.query.attribute_history(security, count, unit='1d', fields=None,  
                                fq='pre')
```

回测/模拟专用 API 查看某一支股票的历史数据, 可以选这只股票的多个属性, 默认跳过停牌日期. 当取天数据时, 不包括当天的, 即使是在收盘后; 分钟数据不包括当前分钟的数据, 没有未来;

Parameters

- **security** (*str or list*) -股票代码
- **count** (*int*) -数量, 返回的结果集的行数
- **unit** (*str*) -单位时间长度, 现在支持 ['daily' , '1d' , 'day' , '1min' , '5min' , '15min' , '30min' , '60min' , '1m' , '5m' , '15m' , '30m' , '60m']
- **fields** (*str or list*) -包含: ['open' , 'close' , 'low' , 'high' , 'vol' , 'amount']
- **fq** (*str or None*) -复权选项: pre-前复权 None-不复权 post-后复权

Returns

返回 [pandas.DataFrame] 对象, 行索引是 `datetime` 字符串, 列索引是属性名字.

13.1.4 get_bars - 获取历史数据 (包含快照数据), 可查询多个标的多个数据字段

```
qff.price.query.get_bars(security, count, unit='1d', fields=None,
                        include_now=False, end_dt=None,
                        fq_ref_date=None, market='stock')
```

函数暂未实现

获取各种时间周期的 bar 数据, bar 的分割方式与主流股票软件相同, 而且支持返回当前时刻所在 bar 的数据; get_bars 开盘时取的 bar 高开低收都是当天的开盘价, 成交量成交额为 0; get_bars 没有跳过停牌选项, 所获取的数据都是不包含停牌的数据, 如果 bar 个数少于 count 个, 则返回实际个数, 并不会填充。

Parameters

- **security** (list) - 标的代码或列表, 支持一个或多个标的
- **count** (int) - 大于 0 的整数, 表示获取 bar 的个数。如果行情数据的 bar 不足 count 个, 返回的长度则小于 count 个数。
- **unit** (str) - bar 的时间单位, 支持标准 bar, 包括 ['1m' , '5m' , '15m' , '30m' , '60m' , '1d']
- **fields** (Optional[list]) - 获取数据的字段, 支持如下值: ['date' , 'open' , 'close' , 'high' , 'low' , 'vol' , 'amount'], 默认为 None, 表示全部字段。
- **include_now** (bool) - 取值 True 或者 False。表示是否包含当前 bar, 比如策略时间是 9:33, unit 参数为 5m, 如果 include_now=True, 则返回 9:30-9:33 这个分钟 bar。
- **end_dt** (Optional[str]) - 查询的截止时间, 支持的类型为 None 或 str。默认值为 None
 - 在回测/模拟环境下默认为 context.current_dt
 - 在其他环境下默认为 datetime.now()
 - 由于 bar 的最小单位是一分钟, 所以 end_dt 的秒没有什么意义, 会被替换为 0, 例如: "2019-11-22 9:35:23" 和 "2019-11-22 9:35:00" 是一样的。
- **fq_ref_date** (Optional[str]) - 复权基准日期, 支持的类型为 str 或 None, 为 None 时为不复权数据。
 - 回测/模拟环境中默认为 context.current_dt

- 在投研环境下默认为 `datetime.now()`
- 如果输入 `fq_ref_date = None`, 则获取到的是不复权的数据
- 如果想获取后复权的数据, 可以将 `fq_ref_date` 指定为一个股票 IPO 之前很早的日期, 比如 `datetime.date(1990, 1, 1)`
- 定点复权, 以某一天价格点位为参照物, 进行的前复权或后复权。
- 设置为 `datetime.datetime.now()` 即返回前复权数据。
- 设置为 `context.current_dt` 返回动态复权数据,
- `market (str)` - 市场类型, 目前支持 [“stock”, “index”, “ETF”], 默认 “stock”。

Return type

`Optional[DataFrame]`

Returns

- 若 `security` 为字符串格式的标的代码时, 返回 `pandas.DataFrame`, `dataframe` 的 `index` 是一个日期字符串
- 若 `security` 为 list 格式的标的代码时, 返回 `pandas.DataFrame`, `dataframe` 的 `index` 是一个 `MultiIndex`

Example

```
# 获取平安银行最近 5 天数据, 包括 context.current_date
df = get_bars('000001', 5, unit='1d', fields=['open', 'close'], include_
    ↪now=True)

# 设置复权基准日为 2018-01-05, 取得的最近 5 条包括 end_dt 的天数据
get_bars('600507', 5, unit='1d', fields=['date', 'open', 'high', 'low', 'close'],
    ↪include_now=True, end_dt='2018-01-05 11:00:00', fq_ref_date='2018-01-05')
```

13.1.5 get_current_data - 获取当前时刻标的的数据

```
qff.price.cache.get_current_data(code, market='stock')
```

获取当前时刻标的的数据

获取当前单位时间（当天/当前分钟）的涨跌停价, 是否停牌, 当天的开盘价等。回测时, 通过其他获取数据的 API 获取到的是前一个单位时间（天/分钟）的数据, 而有些数据, 我们在这个单位时间是知道的, 比如涨跌停价, 是否停牌, 当天的开盘价。我们添加了这个 API 用来获取这些数据。

Parameters

- `code (str)` - 股票代码
- `market (str)` - 标的类型，股票还是指数

Return type

`Optional[SecurityUnitData]`

Returns

一个 `SecurityUnitData` 对象，代表当前时刻的股票数据

13.1.6 fetch_ticks - 获取实时 tick 数据

```
qff.price.fetch.fetch_ticks(code, market='stock')
```

获取股票或指数列表当前时刻的 ticks 数据

Parameters

- `code (list)` - 一支股票代码或者一个指数代码列表
- `market (str)` - 市场类型，目前支持 “stock” 和 “index”，默认 “stock”。

Returns

返回 `[pandas.DataFrame]` 对象，当前股票列表对应的 tick 数据。tick 字段描述请见 `Tick - tick` 对象

13.1.7 fetch_today_transaction - 获取当日实时分笔成交信息

```
qff.price.fetch.fetch_today_transaction(code)
```

获取当日实时分笔成交信息，包含集合竞价

Parameters

`code (str)` - 一支股票代码

Returns

返回对应股票的分笔成交信息，

	price	vol	num	buyorsell
datetime				
2022-12-30 09:25	13.04	3119	153	2
2022-12-30 09:30	13.04	2965	89	0

(continues on next page)

(continued from previous page)

```
2022-12-30 09:30 13.05 10320 182 0
```

其中: buyorsell 1--sell 0--buy 2--盘前'

13.1.8 fetch_today_min_curve - 实时获取当天的 1 分钟曲线

```
qff.price.fetch.fetch_today_min_curve(code, market='stock')
```

获取当天的分钟曲线，返回当前时间前的当日 1 分钟曲线数据，用于模拟交易环境

Parameters

- `code (str)` -- 一支股票代码或者一个指数代码
- `market (str)` - 市场类型，目前支持 “stock/index/etf”，默认 “stock”。

Returns

返回 [pandas.DataFrame] 对象，行索引是 `date`(分钟级别数据为 `datetime`)，列索引是行情字段名字。

13.2 股票

13.2.1 get_all_securities- 获取平台支持的所有股票信息

```
qff.price.query.get_all_securities(date=None, market='stock', df=False)
```

获取平台支持的所有股票信息

Parameters

- `date (str)` - 查询日期，用于获取某日期还在上市的股票信息。默认值为 `None`，表示获取当日的股票信息。
 1. 特定参数 “all”，表示获取所有日期的上市股票信息。
 2. 特定参数 “delist”，表示获取所有退市股票信息
 3. 建议使用时添加上指定 `date`
- `market (str)` - 用来过滤 `securities` 的类型，目前支持的 `type` 仅有 [‘stock’ , ‘index’ , ‘etf’]

- `df (bool)` -返回格式, 若是 `True`, 返回 `[pandas.DataFrame]`, 否则返回一个仅包含标的代码的 `list`, 默认是 `False`.

Returns

`[pandas.DataFrame or list]` 返回标的列表

Example

```
get_all_securities()[2]
```

返回:

```
['000001', '000002']
```

当日市场交易的所有股票列表

```
get_all_securities(df=True)[2]
```

返回:

---	name	start	end	type
000001	平安银行	1991-04-03	2200-01-01	stock
000002	万科A	1991-01-29	2200-01-01	stock

`name`: 中文名称

`start`: 上市日期

`end`: 退市日期 (股票是最后一个交易日, 不同于摘牌日期), 如果没有退市则为 2200-01-01;

`type`: 类型, `stock`(股票), `index`(指数), `etf`(场内 ETF 基金)

13.2.2 get_security_info- 获取股票的信息

```
qff.price.query.get_security_info(code, market='stock')
```

获取股票的信息.

Parameters

- `code` (字符串) -证券代码
- `market` (字符串) -用来过滤 `code` 的类型, 目前支持的 `type` 仅有 [`'stock'`, `'index'`, `'etf'`]

Returns

一个字典对象, 有如下 `key` 值:

1. `code`: 股票代码
2. `name`: 中文名称

3. **start**: 上市日期, 字符串类型

4. **end**: 退市日期 (股票是最后一个交易日, 不同于摘牌日期), 如果没有退市则为 2200-01-01

Example

::

```
# 获取股票 000001 的上市时间 start_date = get_security_info( '000001'
).start print(start)
```

13.2.3 get_stock_name- 获取股票名称

```
qff.price.query.get_stock_name(code=None, date=None)
```

获取股票名称

Parameters

- **code** -股票代码, 支持 list, 如果为空, 则返回所有股票
- **date** -查询日期, 如果为空, 则取上一个交易日日期

Return dict

返回股票代码与股票名称的字典

13.2.4 get_block_stock- 根据板块名称检索对应的股票代码

```
qff.price.query.get_block_stock(block)
```

根据板块名称检索对应的股票代码

Parameters

block (*str*) -板块名称

Returns

板块对应的股票代码列表

Return type

list

板块名称汇总:

```
blockname =
[
```

(continues on next page)

(continued from previous page)

300ESG,300周期,300非周,3D 打印,5G 概念,BIPV 概念,C2M 概念,CIPS 概念,CXO 概念,ETC 概念

HJT 电池,IP 变现,MCU 芯片,MSCIA50,MSCI 中盘,MSCI 成份,MiniLED,NFT 概念,NMN 概念,OLED 概念

PPP 模式,PVDF 概念,QFII 新进,QFII 重仓,RCS 概念,ST 板块,一带一路,三代半导,上海自贸,上证 180

上证 380,上证 50,上证中盘,上证创新,上证治理,上证混改,上证红利,上证超大,不活跃股,专精特新

业绩预升,业绩预增,业绩预降,东数西算,两年新股,个人持股,中俄贸易,中创 100,中华 A80,中字头

中小 100,中小 300,中小银行,中盘价值,中盘成长,中证 100,中证 200,中证央企,中证红利,中证龙头

久不分红,乡村振兴,亏损股,云游戏,云科技,云计算,互联网金融,人工智能,人脑工程,人造肉

代糖概念,仿制药,低价股,低市净率,低市盈率,体育概念,保险新进,保险重仓,信创,信息安全

信托重仓,债转股,储能,元宇宙,充电桩,光伏,光刻机,免疫治疗,免税概念,养老概念

养老金,内地低碳,军民融合,农业 50,农村金融,冷链物流,分拆上市,分散染料,创业 300,创业创新

创业大盘,创业板 50,创业板指,创业蓝筹,创医药,创成长,创投概念,创新 100,创科技,创质量

券商重仓,券商金股,化肥,北上重仓,北交所,北京冬奥,区块链,区块链 50,医废处理,医美概念

半导体 50,博彩概念,卫星导航,即将解禁,参股新股,参股金融,双创 50,发可转债,口罩防护,可燃冰

台资背景,含 B 股,含 GDR,含 H 股,含可转债,员工持股,商誉减值,回购计划,固态电池,国产软件

国信价值,国开持股,国证价值,国证农业,国证基建,国证大宗,国证成长,国证服务,国证治理,国证红利

国证芯片,国资云,国防军工,土地流转,在线消费,地下管网,地摊经济,地热能,垃圾分类,培育钻石

基因概念,基金减仓,基金增仓,基金独门,基金重仓,壳资源,外资背景,大数据,大盘价值,大盘成长

大盘股,大飞机,天然气,央企 100,央企改革,央视 50,婴童概念,字节跳动,安防服务,定增股

定增预案,家庭医生,密集调研,富时 A50,小盘价值,小盘成长,小米概念,工业互联网,工业大麻,工业母机

工业气体,已高送转,幽门菌,微利股,微盘股,恒大概念,成渝特区,户数减少,户

(continues on next page)

(continued from previous page)

数增加，扣非亏损

投资时钟，抗癌，拟减持，拟增持，持续增长，换电概念，摘帽，操作系统，数字孪生，数字货币

数据中心，整体上市，新冠检测，新冠药，新型烟草，新材料，新硬件，新能源车，新进成份，新零售

无人机，无人驾驶，无线耳机，昨成交 20，昨收活跃，昨日上榜，昨日振荡，昨日涨停，昨日跌停，昨日较弱

昨日较强，昨日连板，昨日首板，昨曾涨停，昨曾跌停，昨高换手，智慧城市，智慧政务，智能交通，智能医疗

智能家居，智能机器，智能电网，智能电视，智能穿戴，最近复牌，最近多板，最近异动，最近闪拉，最近闪跌

有机硅，机构吸筹，板块趋势，核污防治，核电核能，次新开板，次新股，次新超跌，次新预增，武汉规划

民企 100，民营医院，民营银行，氟概念，氢能源，氮化镓，水产品，水利建设，污水处理，汽车拆解

汽车电子，汽车芯片，沪深 300，油气改革，泛珠三角，活跃股，海南自贸，海外业务，海峡西岸，消费 100

消费电子，涉矿概念，深次新股，深注册制，深证 100，深证 300，深证价值，深证创新，深证成指，深证成长

深证治理，深证红利，烟草概念，燃料电池，物业管理，物联网，特斯拉，特高压，猪肉，环保 50

环渤海，生物疫苗，生物质能，电商概念，电子纸，电子身份，电解液，白酒概念，百元股，百度概念

皖江区域，盐湖提锂，知识产权，石墨烯，破净资产，破发行价，碳中和，碳纤维，磷概念，社保新进

社保重仓，种业，科创 50，科创信息，科技 100，租购同权，稀土永磁，稀缺资源，空气治理，粤港澳

精选指数，绩优股，维生素，绿色建筑，绿色照明，绿色电力，网红经济，网络游戏，职业教育，聚氨酯

股东减持，股东增持，股权分散，股权激励，股权转让，股权集中，胎压监测，能源互联，腾讯概念，腾讯济安

航运概念，节能，芯片，苹果概念，草甘膦，虚拟现实，虫害防治，融资增加，融资融券，行业龙头

被举牌，装配建筑，装饰园林，要约收购，证金汇金，资源优势，赛马概念，超导概念，超清视频，超级电容

跨境电商，轮动趋势，辅助生殖，边缘计算，近已解禁，近期弱势，近期强势，近期新低，近期新高，远程办公

连续亏损，送转潜力，送转超跌，透明工厂，通达信 88，配股股，配股预案，重组股，重组预案，量子科技

(continues on next page)

(continued from previous page)

```

    金融科技, 钛金属, 钠电池, 钴金属, 铁路基建, 银河 99, 锂电池, 锂矿, 镍金属, 长
    三角
    长株潭, 阿里概念, 陆股通减, 陆股通增, 降解塑料, 雄安新区, 页岩气, 预制菜, 预
    计扭亏, 预计转亏
    预高送转, 风沙治理, 风能, 风险提示, 食品安全, 高商誉, 高市净率, 高市盈率, 高
    校背景, 高端装备
    "高股息股", "高融资盘", "高贝塔值", "高负债率", "高质押股", "鸡肉", "鸿蒙概念", "黄
    金概念"
]

```

13.2.5 get_stock_block- 根据股票代码检索所属的板块名称

```
qff.price.query.get_stock_block(code)
```

根据股票代码检索对应的 block 名称

13.2.6 get_mtss- 获取股票的融资融券信息

```
qff.price.query.get_mtss(security_list, start_date, end_date, fields=None)
```

获取股票的融资融券信息。

获取一只或者多只股票在一个时间段内的融资融券信息

Parameters

- **security_list** -一只股票代码或者一个股票代码的 list
- **start_date** -开始日期, 一个字符串
- **end_date** -结束日期, 一个字符串
- **fields** -字段名或者 list, 可选. 默认为 None, 表示取全部字段, 各字段含义如下:

字段名	含义
date	日期
sec_code	股票代码
fin_value	融资余额（元）
fin_buy_value	融资买入额（元）
fin_refund_value	融资偿还额（元）
sec_value	融券余量（股）
sec_sell_value	融券卖出量（股）
sec_refund_value	融券偿还股（股）

Returns

返回一个 `pandas.DataFrame` 对象，默认的列索引为取得的全部字段。
如果给定了 `fields` 参数，则列索引与给定的 `fields` 对应。

13.2.7 get_st_stock- 获取指定日期 ST 股票代码和名称

```
qff.price.query.get_st_stock(code=None, date=None)
```

获取 ST 股票代码和名称

Parameters

- `code` - 股票代码，支持 `list`，如果为空，则查询所有股票
- `date` - 查询日期，如果为空，则取最近交易日日期

Return dict

返回股票代码与股票名称的字典

13.2.8 get_paused_stock- 获取停牌的股票代码

```
qff.price.query.get_paused_stock(code=None, date=None)
```

获取停牌的股票代码

Parameters

- `code` - 股票代码，支持 `list`，如果为空，则查询所有股票
- `date` - 查询日期，如果为空，则取最近交易日日期

Return list

返回股票代码列表

13.3 指数

13.3.1 get_index_name- 获取指数名称

```
qff.price.query.get_index_name(code=None)
```

获取指数名称

Parameters

`code` - 指数代码, 支持 list, 如果为空, 则返回所有指数

Return dict

返回股票代码与股票名称的字典

13.3.2 get_index_stocks- 获取一个指数给定日期的成分股列表

```
qff.price.query.get_index_stocks(index, date=None)
```

获取一个指数给定日期的成分股列表. 目前仅支持:

- ‘000016’: 上证 50
- ‘000852’: 中证 1000
- ‘000905’: 中证 500
- ‘000906’: 中证 800
- ‘000300’: 沪深 300
- ‘000010’: 上证 180
- ‘000688’: 科创 50abc

Parameters

- `index` - 字符串, 一个指数代码, 如 ‘000300’
- `date` - 字符串, 查询日期, 如 ‘2015-10-15’. 默认为 None, 指当前日期

Returns

返回股票代码的 list

13.3.3 get_industry_stocks- 获取申万一级行业给定日期的成分股列表

`qff.price.query.get_industry_stocks(industry, date=None)`

获取申万一级行业给定日期的成分股列表. 目前仅支持 (申万 I 级):

- ‘801010’: 农林牧渔
- ‘801030’: 基础化工
- ‘801040’: 钢铁
- ‘801050’: 有色金属
- ‘801080’: 电子
- ‘801110’: 家用电器
- ‘801120’: 食品饮料
- ‘801130’: 纺织服饰
- ‘801140’: 轻工制造
- ‘801150’: 医药生物
- ‘801160’: 公用事业
- ‘801170’: 交通运输
- ‘801180’: 房地产
- ‘801200’: 商贸零售
- ‘801210’: 社会服务
- ‘801230’: 综合
- ‘801710’: 建筑材料
- ‘801720’: 建筑装饰
- ‘801730’: 电力设备
- ‘801740’: 国防军工
- ‘801750’: 计算机
- ‘801760’: 传媒
- ‘801770’: 通信
- ‘801780’: 银行
- ‘801790’: 非银金融

- ‘801880’: 汽车
- ‘801890’: 机械设备
- ‘801950’: 煤炭
- ‘801960’: 石油石化
- ‘801970’: 环保
- ‘801980’: 美容护理

Parameters

- **industry** -字符串，一个指数代码，如 ‘801010’
- **date** -字符串，查询日期, 如 ‘2015-10-15’ . 默认为 None, 指当前日期

Returns

返回股票代码的 list

13.4 财务

13.4.1 get_fundamentals- 根据 mongodb 语法查询财务数据

```
qff.price.finance.get_fundamentals(filter, projection=None, date=None,  
                                   report_date=None)
```

根据 mongodb 语法查询财务数据详细的财务数据表及字段描述请见[年报 \(季报\) 数据](#)

- 输入 **date** 时, 查询指定日期 **date** 收盘后所能看到的最近的数据, 我们会查找上市公司在这个日期之前 (包括此日期) 发布的数据, 不会有未来函数.
- 输入 **report_date**, 查询 **report_date** 指定的季度或者年份的财务数。
- 参数 **report_date** 和 **date** 二选一, 同时输入, 则 **report_date** 有效。
- 注意: 不支持 **2000** 年之前的查询

Parameters

- **filter** (Optional[Dict]) -查询条件字典, 按 pymongo 格式输入
- **projection** (Optional[Dict]) - 你需要查询的字段列表, 按 pymongo 格式输入

- **date** (Optional[str]) - 查询日期, 一个字符串 (格式类似 '2015-10-15'), 可以是 None, 使用默认日期. 这个默认日期在回测时, 等于 `context.current_dt` 的前一天. 在实盘时, 为当前最新日期, 一般是昨天。
- **report_date** (Optional[str]) - 财报统计的季度或者年份, 一个字符串, 有两种格式:
 - 季度: 格式是: 年 + 'q' + 季度序号, 例如: '2015q1', '2013q4'.
 - 年份: 格式就是年份的数字, 例如: '2015', '2016'.

Return type

Optional[DataFrame]

Returns

返回一个 [pandas.DataFrame], 每一行对应数据库返回的每一行, 列索引是你查询的所有字段

Example

```
# 策略在开盘前选择净利润增长率大于 30% 的股票
def before_trading_start(context):
    filter = {'f184' : {"$gt": 0.3}}
    df = get_fundamentals(filter=filter, projection={'f184': 1}, date=context.
    ↪previous_date)
    g.security = df['code'].to_list()
```

13.4.2 get_financial_data- 查询多只股票给定日期的财务数据

```
qff.price.finance.get_financial_data(code, fields=None, date=None,
                                     report_date=None)
```

查询多只股票给定日期的财务数据

- 输入 **date** 时, 查询指定日期 **date** 收盘后所能看到的最近的数据, 我们会查找上市公司在这个日期之前 (包括此日期) 发布的数据, 不会有未来函数。
- 输入 **report_date**, 查询 **report_date** 指定的季度或者年份的财务数。
- 参数 **report_date** 和 **date** 二选一, 同时输入, 则 **report_date** 有效。
- 注意: 不支持 2000 年之前的查询

Parameters

- **code** -- 一支股票代码或者一个股票代码的 list, `None` 表示所有股票代码
- **fields** - 返回的财务数据字段 list, '001' - '580', `None` 表示所有财务指标, 详细的财务数据表及字段描述请见[年报 \(季报\) 数据](#)
- **date** - 查询日期, 一个字符串 (格式类似 '2015-10-15'), 可以是 `None`, 使用默认日期. 这个默认日期在回测时, 等于 `context.current_dt` 的前一天。在实盘时, 为当前最新日期, 一般是昨天。
- **report_date** - 财报统计的季度或者年份, 一个字符串, 有两种格式:
 - 季度: 格式是: 年 + 'q' + 季度序号, 例如: '2015q1', '2013q4' .
 - 年份: 格式就是年份的数字, 例如: '2015', '2016' .

:return: 返回一个 `[pandas.DataFrame]`, 每一行对应数据库返回的每一行, 列名是输入的 `field` 字段信息

Example

```
# 获取股票 '000001' 和 '601567' 的每股收益、扣非每股收益及净利润
df = get_financial_data(['000001', '601567'], ['f001', 'f002', 'f095'], watch_
    date='2020-04-22')
```

13.4.3 get_stock_reports- 获取单个股票多个报告期发布的财务数据

```
qff.price.finance.get_stock_reports(code, fields=None, start=None,
    end=None)
```

获取单个股票多个报告期发布的财务数据

Parameters

- **code (str)** -- 一支股票代码
- **fields (Optional[list])** - 返回的财务数据字段 list, '001' - '580', `None` 表示所有财务指标, 详细的财务数据表及字段描述请见[年报 \(季报\) 数据](#)
- **start (Optional[str])** - 查询期间开始日期, 一个字符串 (格式类似 '2015-10-15'), 可以是 `None`, 代表从股票上市开始

- `end` (Optional[str]) - 查询日期, 一个字符串 (格式类似' 2015-10-15'), 可以是 `None`, 使用默认日期. 这个默认日期在回测时, 等于 `context.current_dt` 的前一天。在实盘时, 为当前最新日期, 一般是昨天。

Return type

Optional[DataFrame]

Returns

返回一个 [pandas.DataFrame]

13.4.4 `get_fundamentals_continuously`- 查询单个股票连续多日的财务数据

```
qff.price.finance.get_fundamentals_continuously(code, fields=None,  
                                                end_date=None,  
                                                count=None)
```

查询单个股票连续多日的财务数据

详细的财务数据表及字段描述请见 `financial_dict`

Parameters

- `code` (str) - 一支股票代码
- `fields` (Optional[list]) - 返回的财务数据字段 list, '001' - '580', `None` 表示所有财务指标, 详细的财务数据表及字段描述请见 [年报\(季报\) 数据](#)
- `end_date` (Optional[str]) - 查询日期, 一个字符串 (格式类似'2015-10-15'), 可以是 `None`, 使用默认日期. 这个默认日期在回测时, 等于 `context.current_dt` 的前一天。在实盘时, 为当前最新日期, 一般是昨天。
- `count` (Optional[int]) - 获取 `end_date` 前 `count` 个日期的数据

Return type

Optional[DataFrame]

Returns

返回一个 [pandas.DataFrame]

13.4.5 get_history_fundamentals- 获取多只股票多个季度（年度）的历史财务数据

```
qff.price.finance.get_history_fundamentals(code, fields, watch_date=None,
                                           report_date=None, count=1,
                                           interval='1q')
```

获取多只股票多个季度（年度）的历史财务数据可指定单季度数据，也可以指定年度数据。可以指定观察日期，也可以指定最后一个报告期的结束日期

Parameters

- **code** (Optional[str]) - 股票代码或者股票代码列表
- **fields** (Optional[list]) - 要查询的财务数据的列表, 详细的财务数据表及字段描述请见[年报 \(季报\) 数据](#)
- **watch_date** (Optional[str]) - 观察日期, 如果指定, 将返回 watch_date 日期前 (包含该日期) 发布的报表数据
- **report_date** (Optional[str]) - 财报日期, 可以是‘2019’/’2019q1’ /’ 2018q4’ 格式, 如果指定, 将返回 report_date 对应报告期及之前的历史报告期的报表数据 watch_date 和 stat_date 只能指定一个, 而且必须指定一个
- **count** (int) - 查询历史的多个报告期时, 指定的报告期数量. 如果股票历史报告期的数量小于 count, 则该股票返回的数据行数将小于 count
- **interval** (str) - 查询多个报告期数据时, 指定报告期间隔, 可选值: ‘1q’ /’ 1y’, 表示间隔一季度或者一年, 举例: report_date=’ 2019q1’, interval=’ 1q’, count=4, 将返回 2018q2,2018q3,2018q4,2019q1 的数据 report_date=’ 2019q1’, interval=’ 1y’, count=4, 将返回 2016q1,2017q1,2018q1,2019q1 的数据

Return type

Optional[DataFrame]

Returns

pandas.DataFrame, 数据库查询结果. 数据格式同 get_fundamentals. 每个股票每个报告期（一季度或者一年）的数据占用一行. 推荐用户对结果使用 pandas 的 groupby 方法来进行分组分析数据

13.4.6 get_stock_forecast- 获取股票业绩预告数据

```
qff.price.finance.get_stock_forecast(code=None, start=None, end=None)
```

获取股票业绩预告数据

Parameters

- **code** (Optional[str]) -一支股票代码或股票列表, 可以是 `None` 值, 代表整个市场
- **start** (Optional[str]) -查询期间开始日期, 一个字符串 (格式类似' 2015-10-15'), 可以是 `None`, 代表从股票上市开始
- **end** (Optional[str]) -查询日期, 一个字符串 (格式类似' 2015-10-15'), 可以是 `None`, 使用默认日期. 这个默认日期在回测时, 等于 `context.current_dt` 的前一天。在实盘时, 为当前最新日期, 一般是昨天。

Return type

Optional[DataFrame]

Returns

返回一个 [pandas.DataFrame], 包含以下数据:

字段名	含义
code	股票代码
pub_date	业绩预告公告日期
report_date	报告期
profit_min	本期利润下限
profit_max	本期利润上限
profit_ratio_min	本期利润同比增幅下限
profit_ratio_max	本期利润同比增幅上限

13.4.7 get_stock_express- 获取股票业绩快报数据

```
qff.price.finance.get_stock_express(code=None, start=None, end=None)
```

获取股票业绩快报数据

Parameters

- **code** (Optional[str]) - 一支股票代码或股票列表，可以是 None 值，代表整个市场
- **start** (Optional[str]) - 查询期间开始日期，一个字符串（格式类似 '2015-10-15'），可以是 None，代表从股票上市开始
- **end** (Optional[str]) - 查询日期结束日期，一个字符串（格式类似 '2015-10-15'），可以是 None，使用默认日期。这个默认日期在回测时，等于 `context.current_dt` 的前一天。在实盘时，为当前最新日期，一般是昨天。

Return type

Optional[DataFrame]

Returns

返回一个 [pandas.DataFrame]，包含以下数据：

字段名	含义
code	股票代码
pub_date	业绩快报公告日期
report_date	报告期
net_profit	归母公司净利润
adjusted_profit	扣非净利润
total_assets	总资产
net_assets	净资产
eps	每股收益
roe_diminish	摊薄净资产收益率
roe_weighting	加权净资产收益率
naps	每股净资产

13.4.8 get_valuation- 获取多个股票在指定交易日范围内的市值表数据

```
qff.price.finance.get_valuation(code, start=None, end=None, fields=None,
                                count=None)
```

获取多个股票在指定交易日范围内的市值表数据

Parameters

- **code** (str) - 一支股票代码或者一个股票代码的 list，None 表示所有股票代码

- **start** (Optional[str]) - 查询开始时间，不能与 **count** 共用
- **end** (Optional[str]) - 查询结束时间
- **count** (Optional[int]) - 表示往前查询每一个标的 **count** 个交易日的数据，如果期间标的停牌，则该标的返回的市值数据数量小于 **count**
- **fields** (Optional[list]) - 财务数据中市值表的字段，为 **None** 返回所有字段，可用字段如下：

Return type

Optional[DataFrame]

Returns

返回一个 **dataframe**，索引默认是 **pandas** 的整数索引，返回的结果字段描述如下：

字段名	含义
code	股票代码
date	日期
quantity_ratio	量比
capitalization	总股本 (万股)
circulating_cap	流通股本 (万股)
market_cap	总市值 (亿元)
circulating_market_cap	流通市值 (亿元)
turnover_ratio	换手率 (%)
pe_ratio	市盈率 (PE, TTM)
pe_ratio_lyr	市盈率 (PE)s
pe_ratio_dyn	市盈率 (动态)
pb_ratio	市净率 (PB)

13.4.9 query_valuation- 查询满足条件的市值信息数据

`qff.price.finance.query_valuation(filter, projection=None)`

查询满足条件的市值信息数据可用查询条件字段如下：

字段名	含义
code	股票代码
date	日期
quantity_ratio	量比
capitalization	总股本 (万股)
circulating_cap	流通股本 (万股)
market_cap	总市值 (亿元)
cir_market_cap	流通市值 (亿元)
turnover_ratio	换手率 (%)
pe_ttm	市盈率 (PE, TTM)
pe_lyr	市盈率 (PE)s
pe_dyn	市盈率 (动态)
pb_ratio	市净率 (PB)

Parameters

- **filter** (Dict) -dict 查询条件字典, 按 pymongo 格式输入
- **projection** (Optional[Dict]) - 你需要查询的字段列表, 按 pymongo 格式输入, 返回结果中总会包含 **code**、**date** 字段, 无需输入

Return type

Optional[DataFrame]

Returns

返回一个 [pandas.DataFrame], 每一行对应数据库返回的每一行, 索引是你查询的所有字段

Example

```
# 查询上一个交易日量比大于 2, 换手率小于 2% 的股票
yesterday = context.previous_date
filter = {'date': yesterday, 'quantity_ratio': {'$gt': 2},
↪ 'turnover_ratio': {'$lt': 0.02}}
projection = {'quantity_ratio' : 1, 'turnover_ratio': 1}
signal = query_valuation(filter, projection)
```


q

`qff`, [130](#)

`qff.frame.api`, [108](#)

`qff.frame.const`, [131](#)

`qff.frame.order`, [111](#)

`qff.price.finance`, [149](#)

`qff.price.query`, [133](#)

`qff.trader.ths`, [115](#)